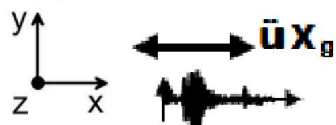
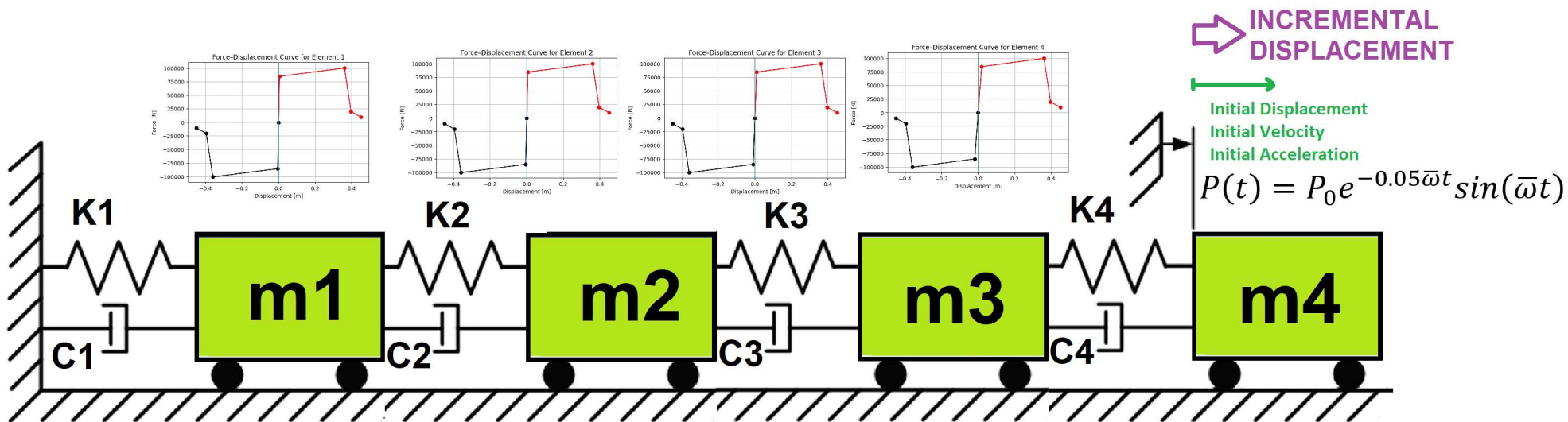


>> IN THE NAME OF ALLAH, THE MOST GRACIOUS, THE MOST MERCIFUL <<

**COMPREHENSIVE NONLINEAR
SEISMIC ASSESSMENT OF A MULTI-
DEGREE-FREEDOM (MDOF)
STRUCTURE: AN OPENSEES
FRAMEWORK FOR STATIC
PUSHOVER, CYCLIC DEGRADATION,
STATIC TIME-HISTORY AND
DYNAMIC TIME-HISTORY ANALYSIS**

THIS PROGRAM WRITTEN BY SALAR DELAVAR GHASHGHAEE (QASHQAI)



$$\text{Structural Ductility Damage Index} = \frac{\Delta_d - \Delta_y}{\Delta_u - \Delta_y}$$

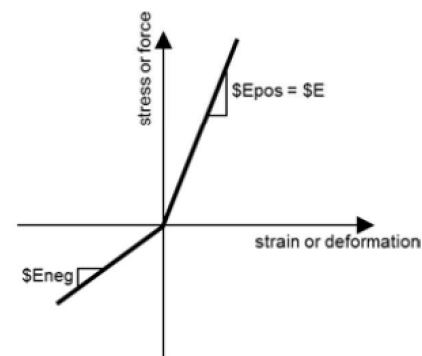
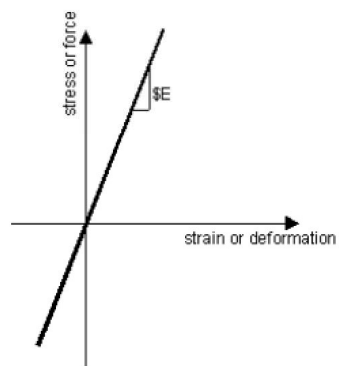
Δ_d = Lateral Displacement from Dynamic Analysis

Δ_y = Lateral Yield Displacement from Pushover Analysis

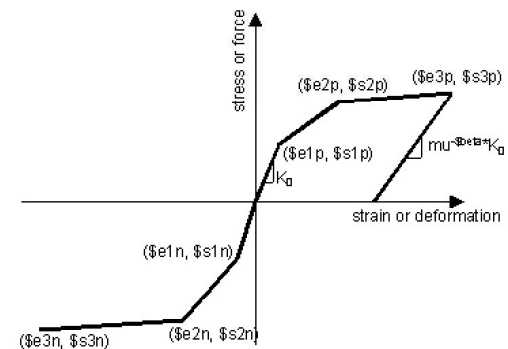
Δ_u = Lateral Ultimate Displacement from Pushover Analysis

$$P(t) = P_0 e^{-0.05\bar{\omega}t} \sin(\bar{\omega}t)$$

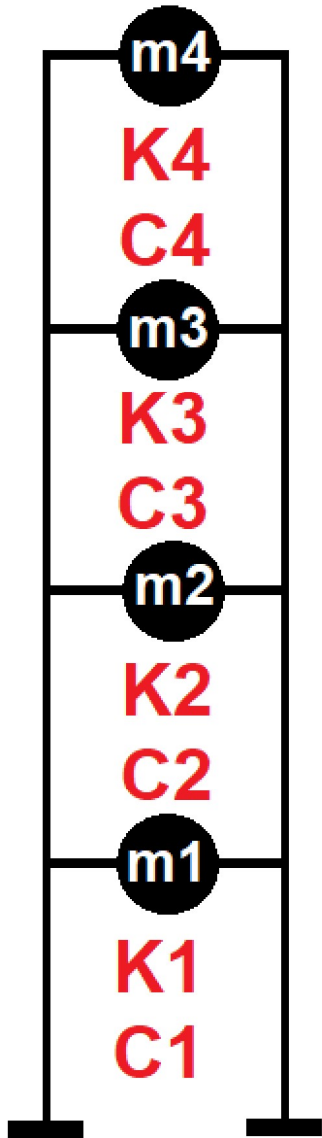
$$m\ddot{u} + c\dot{u} + ku = P(t)$$



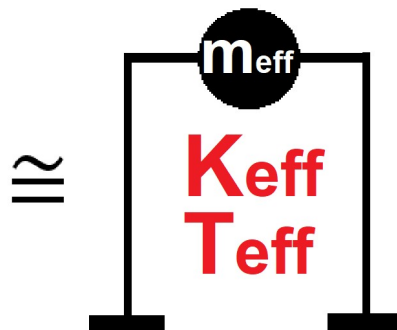
ELASTIC MATERIAL



INELASTIC MATERIAL

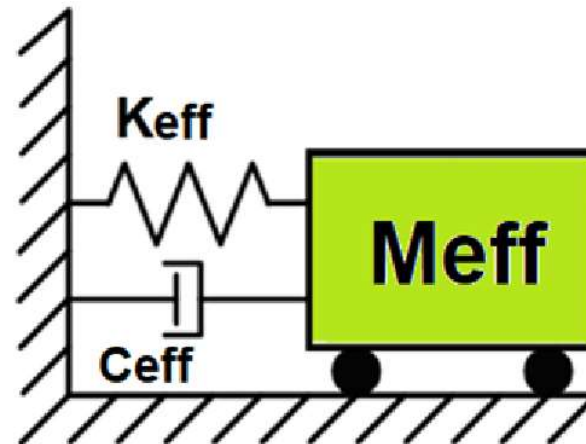


Displacement-based seismic analysis transformation, converting a multi-degree-of-freedom (MDOF) system into an equivalent single-degree-of-freedom (SDOF) system for seismic assessment



$$\Delta_d = \frac{\sum_{i=1}^n m_i \Delta_i^2}{\sum_{i=1}^n m_i \Delta_i} \quad m_{eff} = \frac{\sum_{i=1}^n m_i \Delta_i}{\Delta_d}$$

$$k_{eff} = \frac{\sum_{i=1}^n k_i \Delta_i}{\Delta_d} \quad T_{eff} = \frac{2\pi}{\sqrt{\frac{k_{eff}}{m_{eff}}}}$$



FileEditSearchSourceRunDebugConsolesProjectsToolsViewHelp

C:\Users\Dell\Desktop\OPENSEES_FILES\+EIGHT_ANALY...N_CAPACITY_INDEX\22_MDOF_8_ANA\p(t)_MDOF_8_ANA.py

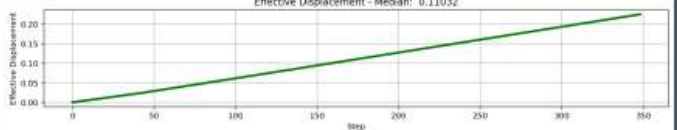
P(t)_MDOF_8_ANA.py X

COMPUTE_EFFECTIVE_PROPERTIES_FUN.py X

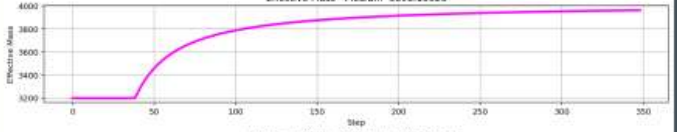
```
1 #####
2 # >> IN THE NAME OF ALLAH, THE MOST GRACIOUS, THE MOST MERCIFUL <<
3 # COMPREHENSIVE NONLINEAR SEISMIC ASSESSMENT OF A MULTI-DEGREE-FREEDOM STRUCTURE : AN OPEN
4 # FOR STATIC PUSHOVER, CYCLIC DEGRADATION, STATIC TIME-HISTORY AND DYNAMIC TIME-HISTORY
5 #-----
6 # EQUIVALENT SDOF SYSTEM DERIVATION VIA DISPLACEMENT-BASED PUSHOVER ANAL
7 #-----
8 #
9 # P(t) = P0 sin(wt)
10 # P(t) = P0 exp(-0.05wt) sin(wt)
11 #-----
12 # ASSESSMENT OF DUCTILITY DAMAGE INDICES FOR STRUCTURAL ELEMENTS AND SYSTEMS AND E
13 # OF ENERGY DISSIPATION CAPACITY INDEX
14 #-----
15 # THIS PROGRAM WRITTEN BY SALAR DELAVAR GHASHGHAEI (QASHQAI)
16 # EMAIL: salar.d.ghashghaei@gmail.com
17 #####
18 Nonlinear Seismic Performance Assessment of a MDOF:
19 An OpenSeesPy Framework for Material and Geometric Nonlinearity Under Static, Cyclic, and
20
21 This OpenSeesPy script performs rigorous nonlinear static and dynamic analysis of a MDOF
22 for performance-based earthquake engineering. The 2D model incorporates both material non
23 (elastic-perfectly plastic or hysteretic steel with strain hardening)
24 and geometric nonlinearity (corotational truss formulation) to capture P-delta effects
25 and large displacements-essential for collapse assessment.
26
27 Eight analysis protocols are implemented:
28 (1) [PERIOD] : Structural Period
29 (2) [STATIC] : Gravity load analysis establishing dead load state
30 (3) [PUSHOVER] : Displacement-controlled pushover generating full capacity curves
31 and plastic mechanism identification
32 (4) [CYCLIC_DISPLACEMENT] : Symmetric cyclic displacement protocol capturing hysteresis,
33 pinching behavior, and energy dissipation degradation
34 (5) [STATIC EXTERNAL TIME-DEPENDENT LOADING] : Static Analysis of External time-dependent
```

26 %

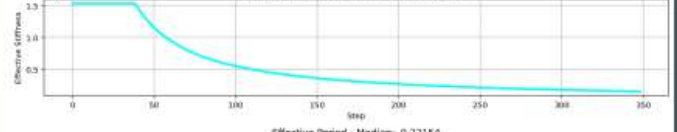
Effective Displacement - Median: 0.11032



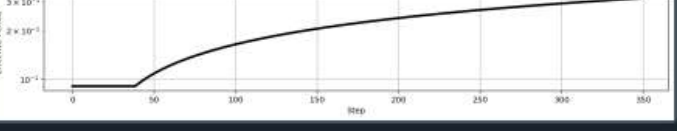
Effective Mass - Median: 3898.10026



Effective Stiffness - Median: 3135556.42315



Effective Period - Median: 0.22154



IPython Console

Files

Help

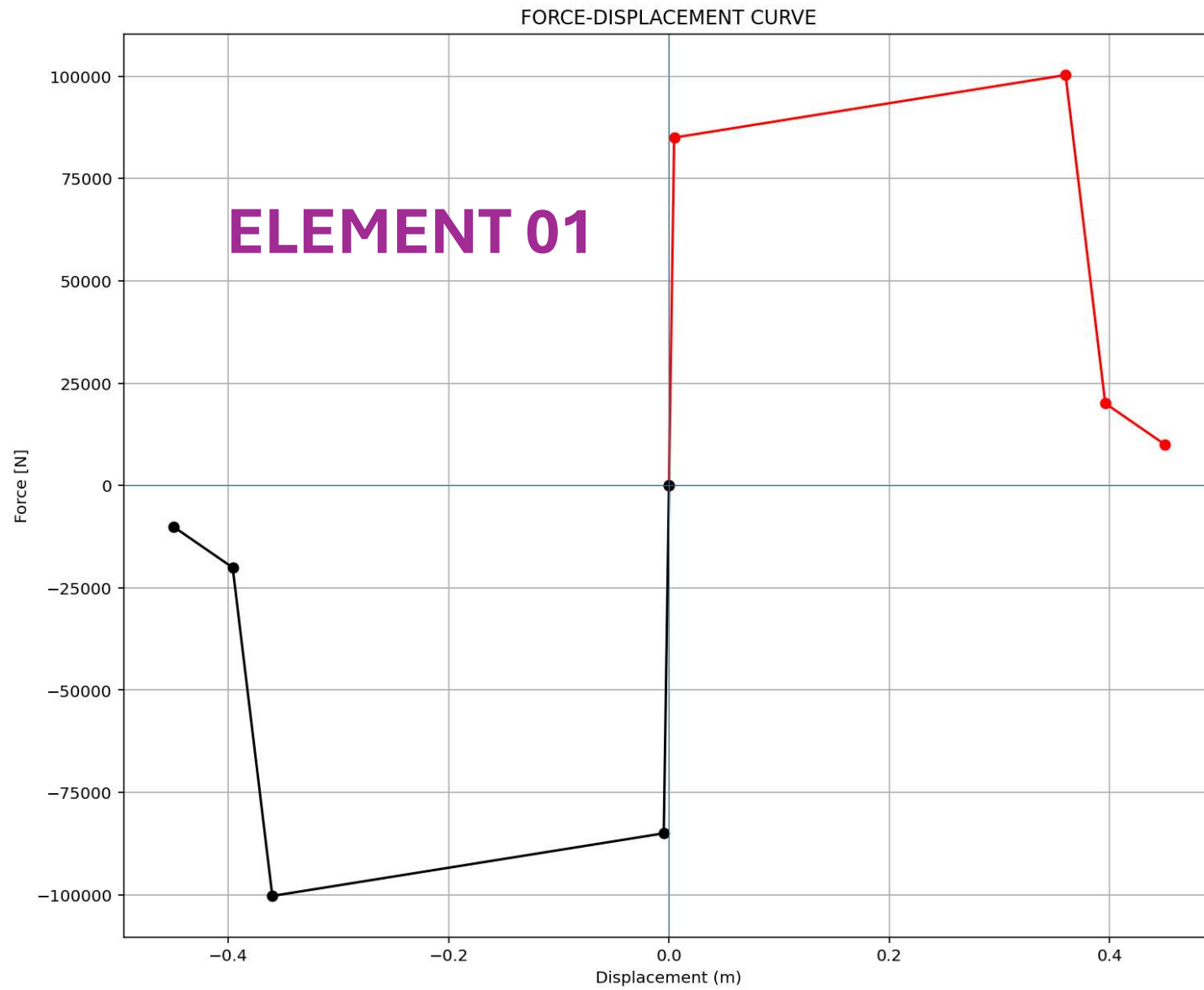
Variable Explorer

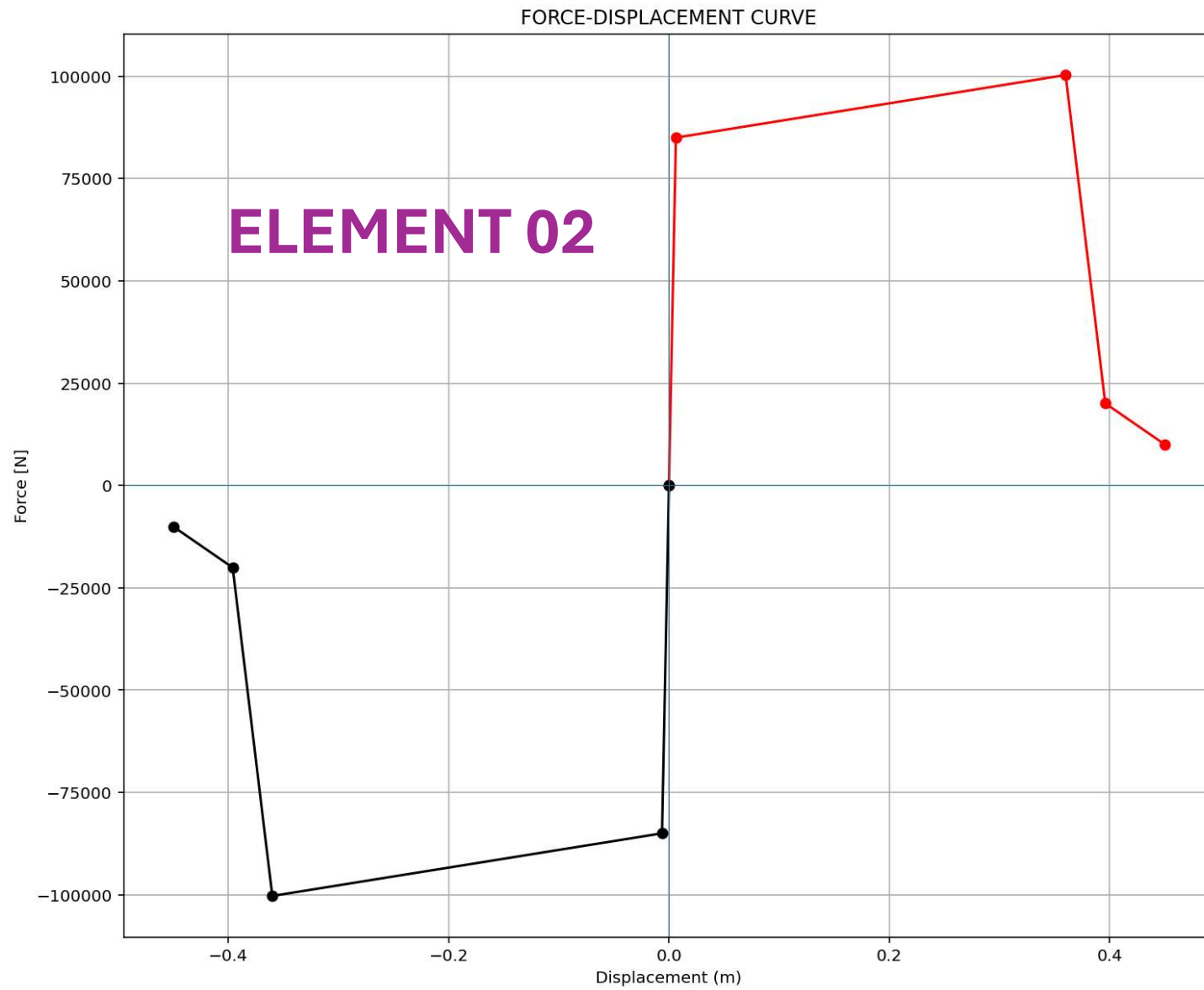
Debugger

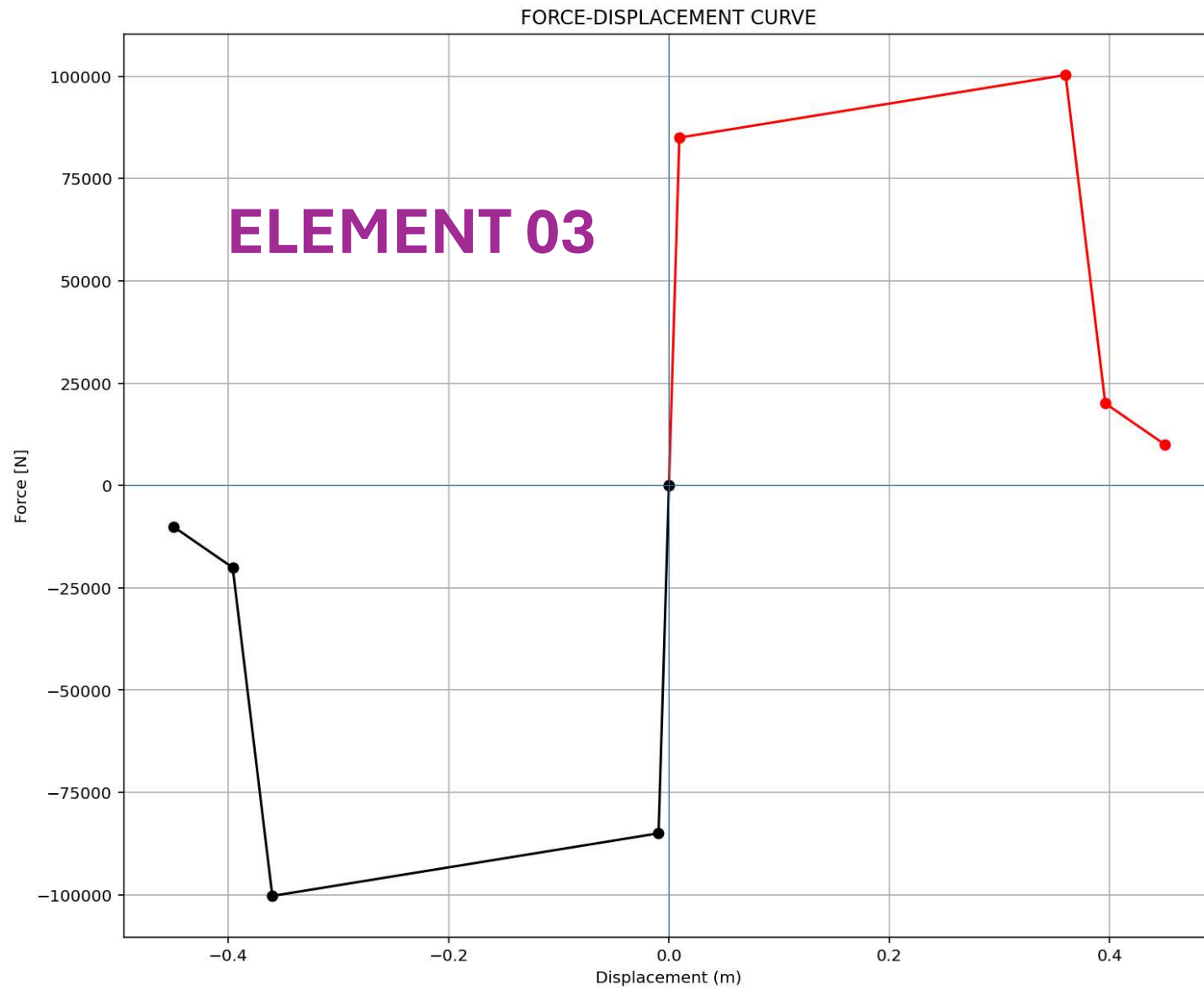
Plots

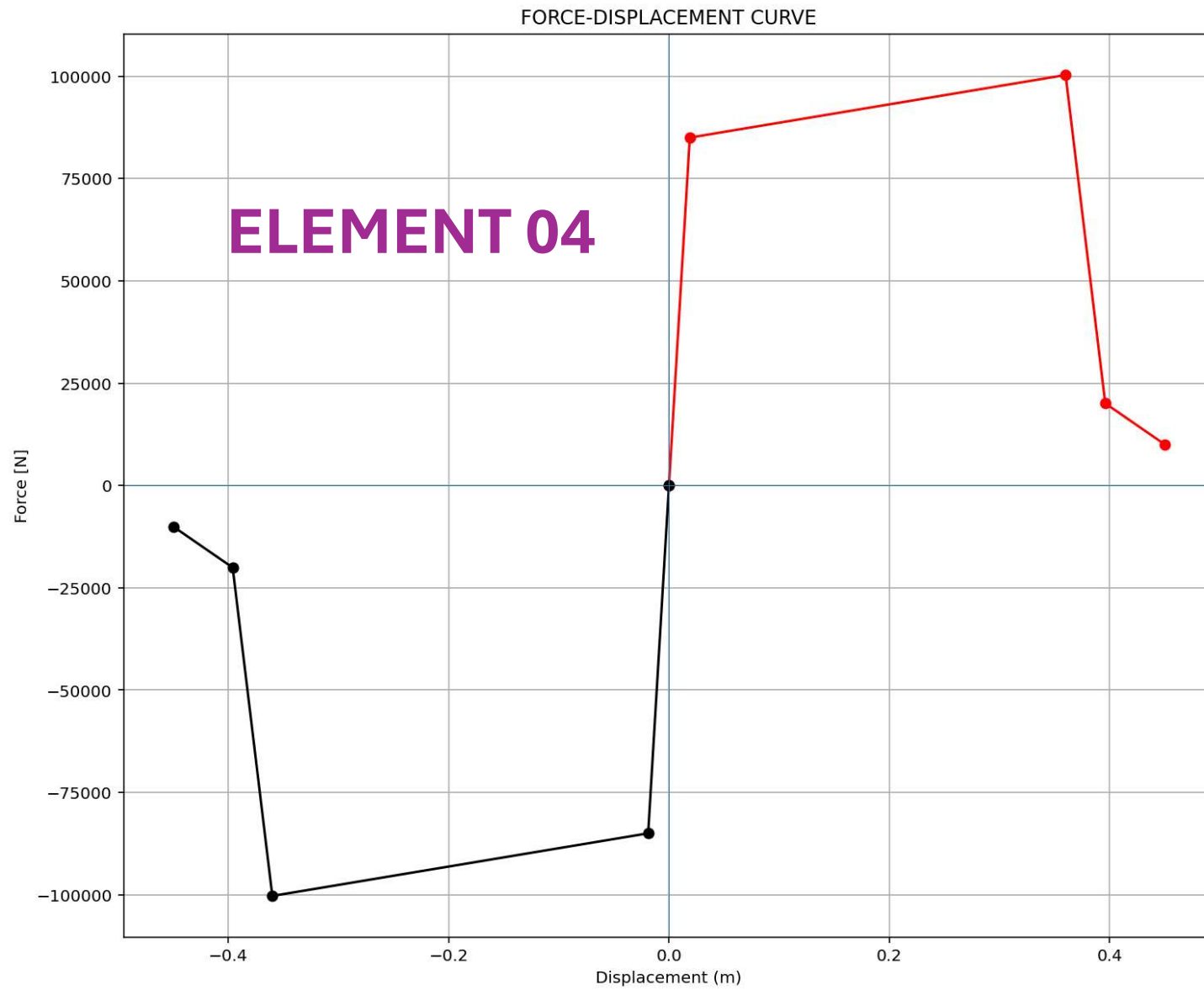
History

Inline Conda: anaconda3 (Python 3.12.7) ✓ LSP: Python Line 6, Col 108 UTF-8 CRLF RW Mem 48%







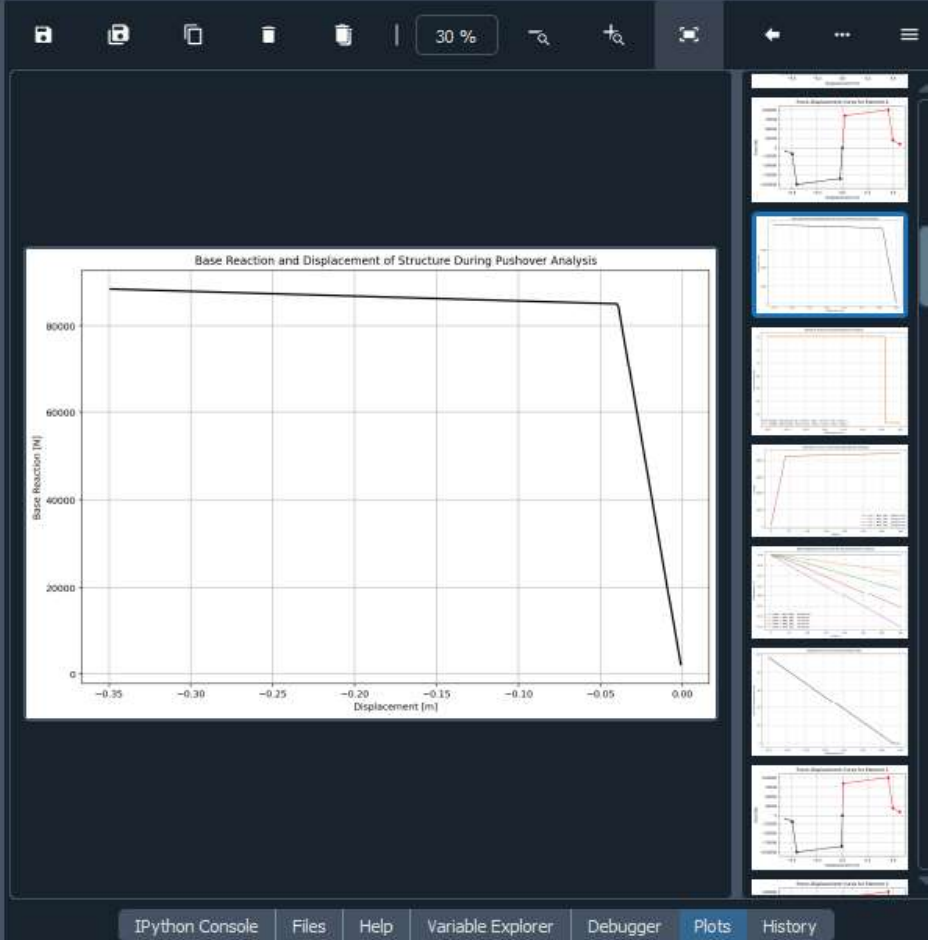


STATIC ANALYSIS

```

1 #####
2 # >> IN THE NAME OF ALLAH, THE MOST GRACIOUS, THE MOST MERCIFUL <<
3 # COMPREHENSIVE NONLINEAR SEISMIC ASSESSMENT OF A MULTI-DEGREE-FREEDOM STRUCTURE : AN OPEN
4 # FOR STATIC PUSHOVER, CYCLIC DEGRADATION, STATIC TIME-HISTORY AND DYNAMIC TIME-HISTORY
5 #
6 # P(t) = P0 sin(wt)
7 # P(t) = P0 exp(-0.05wt) sin(wt)
8 #
9 # ASSESSMENT OF DUCTILITY DAMAGE INDICES FOR STRUCTURAL ELEMENTS AND SYSTEMS AND E
10 # OF ENERGY DISSIPATION CAPACITY INDEX
11 #
12 # THIS PROGRAM WRITTEN BY SALAR DELAVAR GHASHGHAEI (QASHQAI)
13 # EMAIL: salar.d.ghashghaei@gmail.com
14 #####
15 """
16 Nonlinear Seismic Performance Assessment of a MDOF:
17 An OpenSeesPy Framework for Material and Geometric Nonlinearity Under Static, Cyclic, and
18
19 This OpenSeesPy script performs rigorous nonlinear static and dynamic analysis of a MDOF
20 for performance-based earthquake engineering. The 2D model incorporates both material non-
21 (elastic-perfectly plastic or hysteretic steel with strain hardening)
22 and geometric nonlinearity (corotational truss formulation) to capture P-delta effects
23 and large displacements-essential for collapse assessment.
24
25 Eight analysis protocols are implemented:
26 (1) [PERIOD] : Structural Period
27 (2) [STATIC] : Gravity load analysis establishing dead load state
28 (3) [PUSHOVER] : Displacement-controlled pushover generating full capacity curves
29 and plastic mechanism identification
30 (4) [CYCLIC_DISPLACEMENT] : Symmetric cyclic displacement protocol capturing hysteresis,
31 pinching behavior, and energy dissipation degradation
32 (5) [STATIC_EXTERNAL_TIME-DEPENDENT_LOADING] : Static Analysis of External time-dependent
33 (6) [DYNAMIC_EXTERNAL_TIME-DEPENDENT_LOADING] : Dynamic Analysis of External time-dependen
34 (7) [FREE-VIBRATION] : Free-vibration with initial conditions extracting damping ratios

```

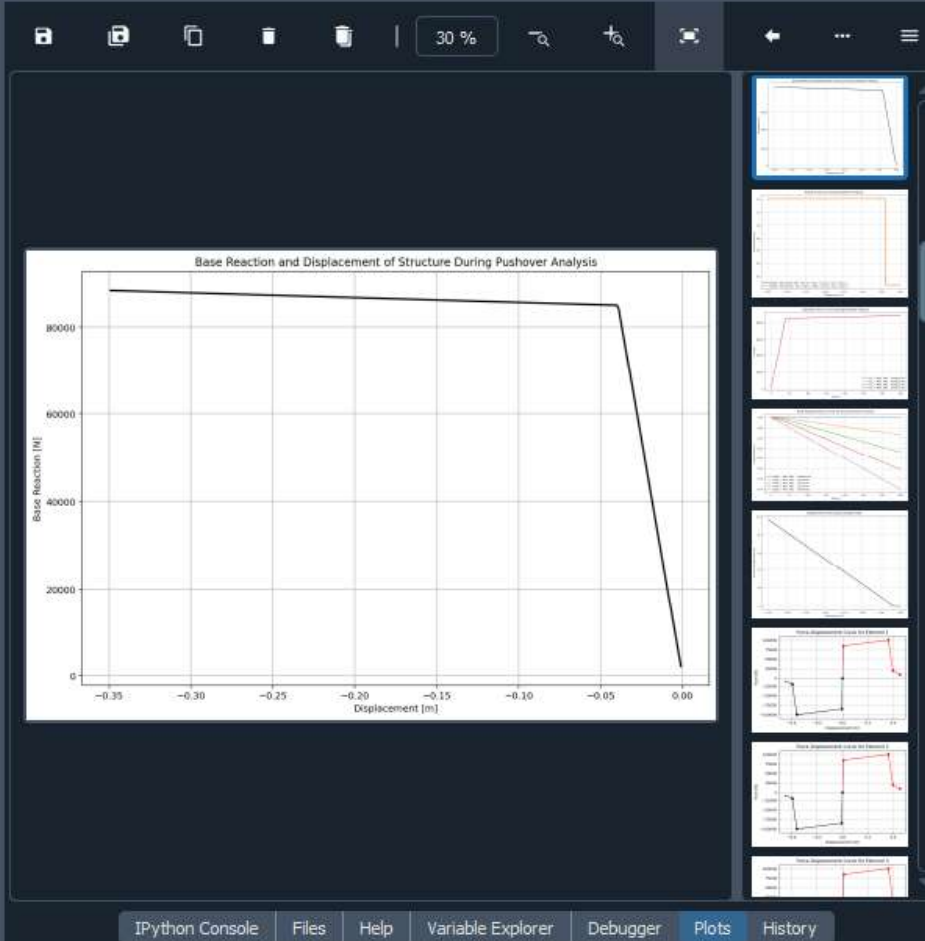


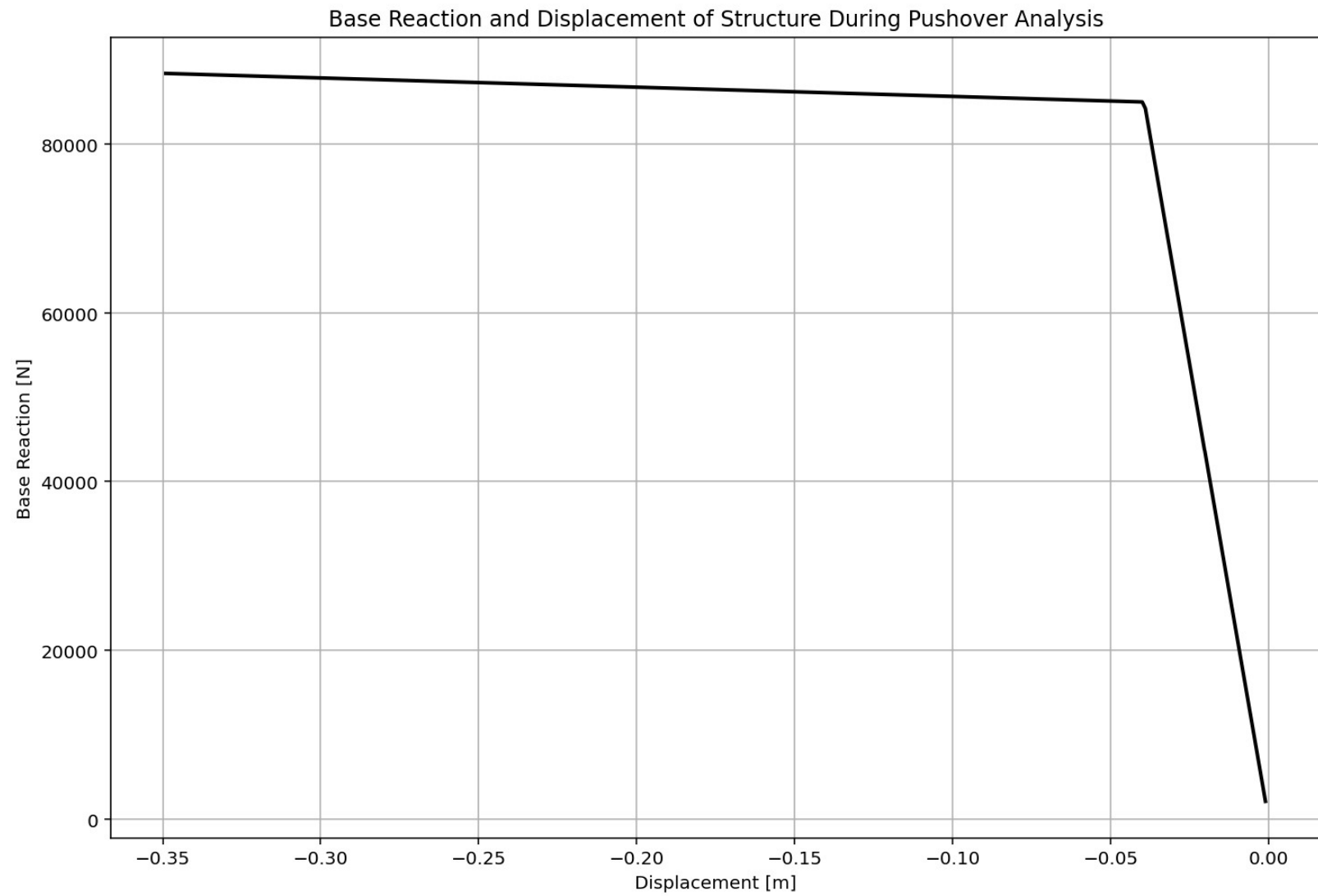
PUSHOVER ANALYSIS

```

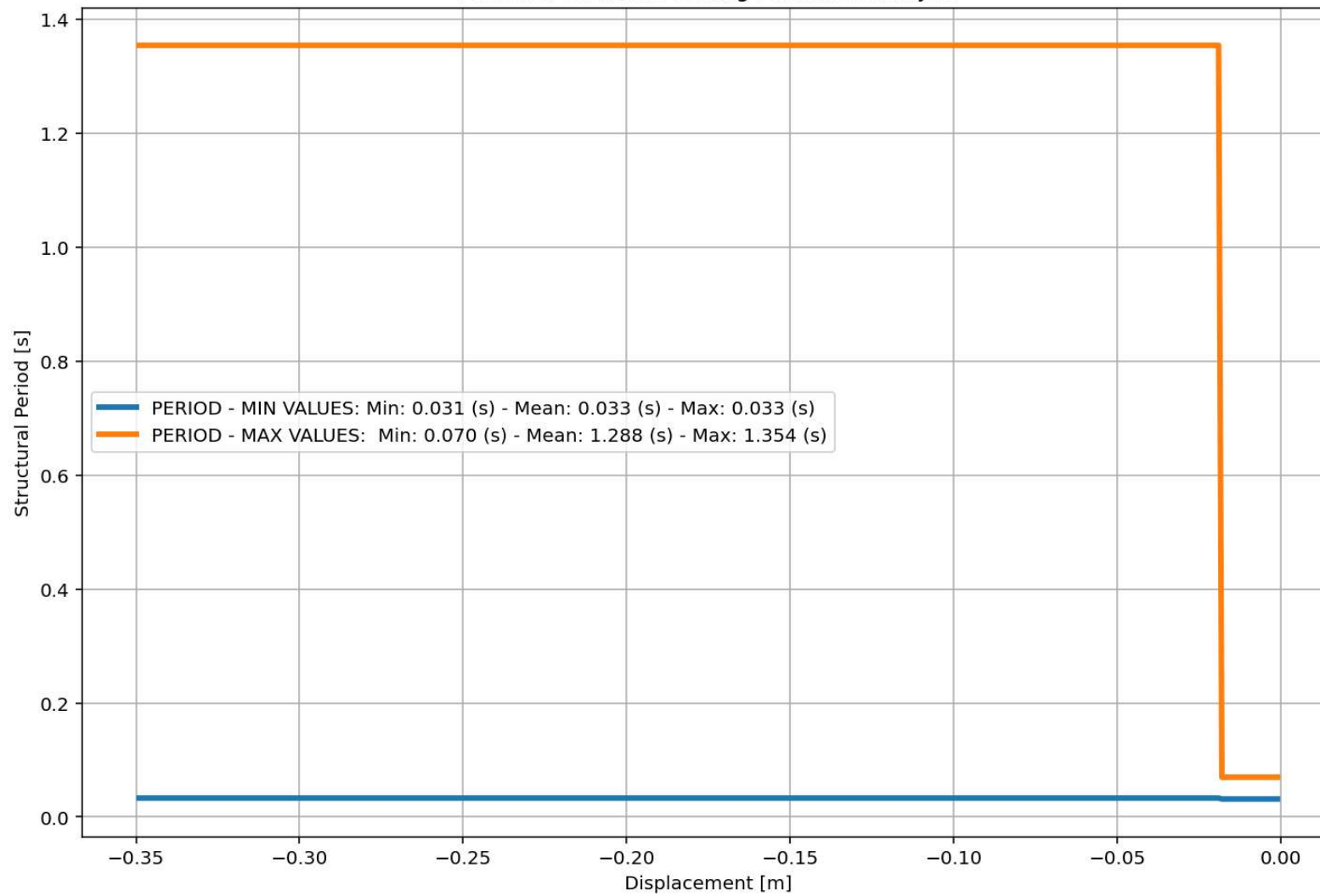
241
242
243 if ANAL_TYPE == 'PUSHOVER': # STATIC TIME-HISTORY ANALYSIS
244     IDctrlDOF = 1 # 1: Horizontal Displacement - 2: Vertical Displacement
245     DINC = -0.001 # [m] Incremental Vertical Displacement
246     DMAX = -0.35 # [m] Max. Displacement
247     ops.integrator('DisplacementControl', center_node, IDctrlDOF, DINC)
248     ops.analysis('Static')
249     Nsteps = int(np.abs(DMAX/ DINC))
250     STEP = 0.0
251     for step in range(Nsteps):
252         OK = ops.analyze(1)
253         S02.ANALYSIS(OK, 1, MAX_TOLERANCE, MAX_ITERATIONS)
254         ops.reactions()
255         reaction.append(ops.nodeReaction(1, 1)) # BASE REACTIO
256         disp.append(ops.nodeDisp(center_node, 1)) # DISPLACEMENT
257         # IN EACH STEP, STRUCTURE PERIOD GOING TO BE CALCULATED
258         #PERIODmin, PERIODmax = S06.RAYLEIGH DAMPING(1, 0.5*DR, DR, 0, 1)
259         PERIODmin, PERIODmax = S05.EIGENVALUE_ANALYSIS(1, PLOT=True)
260         PERIOD_MIN.append(PERIODmin)
261         PERIOD_MAX.append(PERIODmax)
262         DI.append(100*(np.abs(disp[-1])-DY)/(DSU-DY)) # DAMAGE INDEX
263         if DI[-1] <= 0.0: # DAMAGE INDEX -> 0.0 <= DI <=100
264             DI[-1] = 0.0
265         if DI[-1] >= 100:
266             DI[-1] = 100.0
267         # Store forces and displacements
268         for ele_id in ele_force.keys():
269             ele_force[ele_id].append(ops.eleResponse(ele_id, 'force')[0]) # [N]
270         # Store displacements
271         for node_id in node_displacements.keys():
272             node_displacements[node_id].append(ops.nodeDisp(node_id, 1))
273         STEP += 1
274         #print(STEP, disp[-1], reaction[-1])
275         print(f"Step: {STEP}, Displacement: {disp[-1]:.4f} mm, Reaction: {reaction[-1]}

```

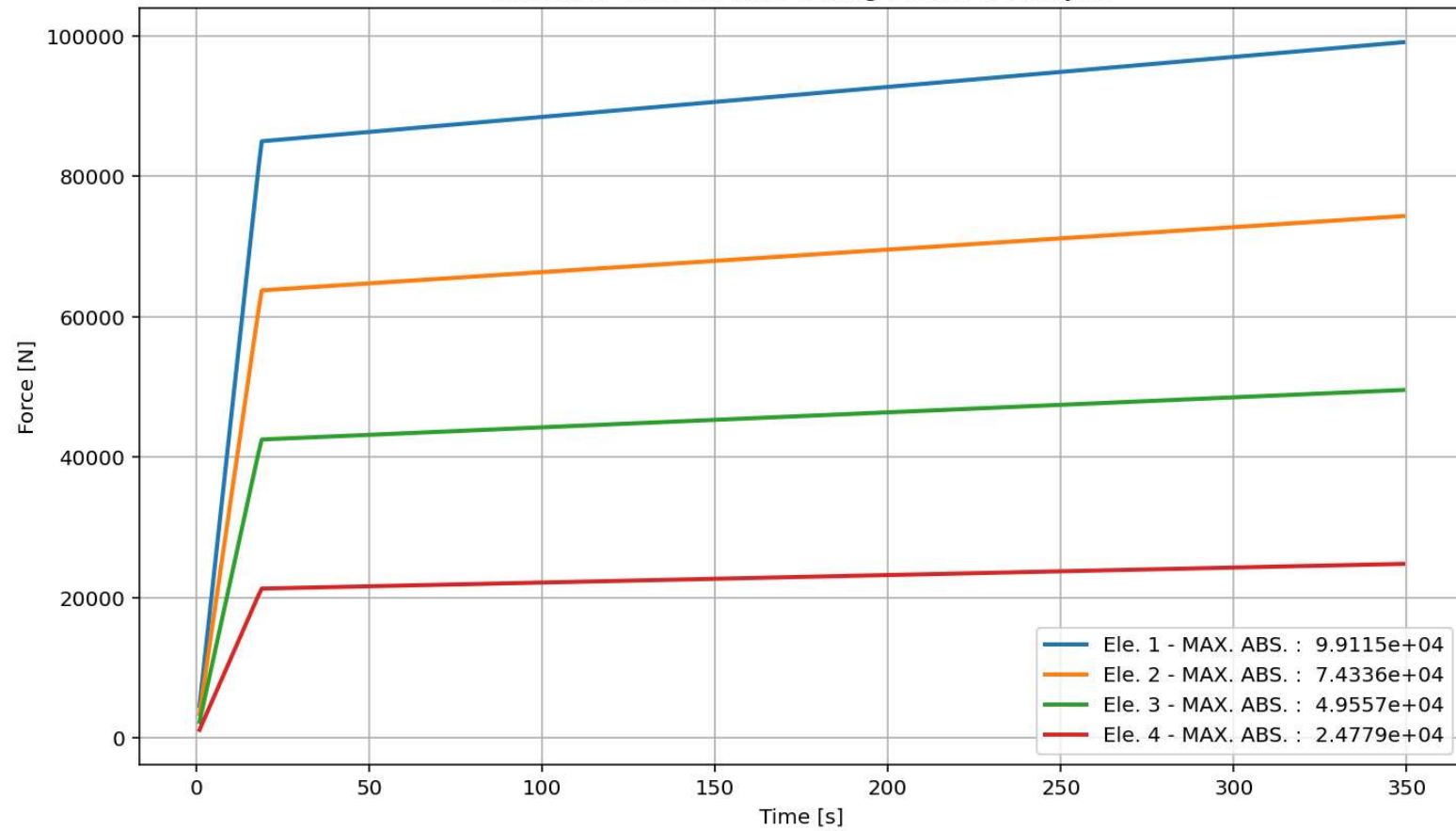




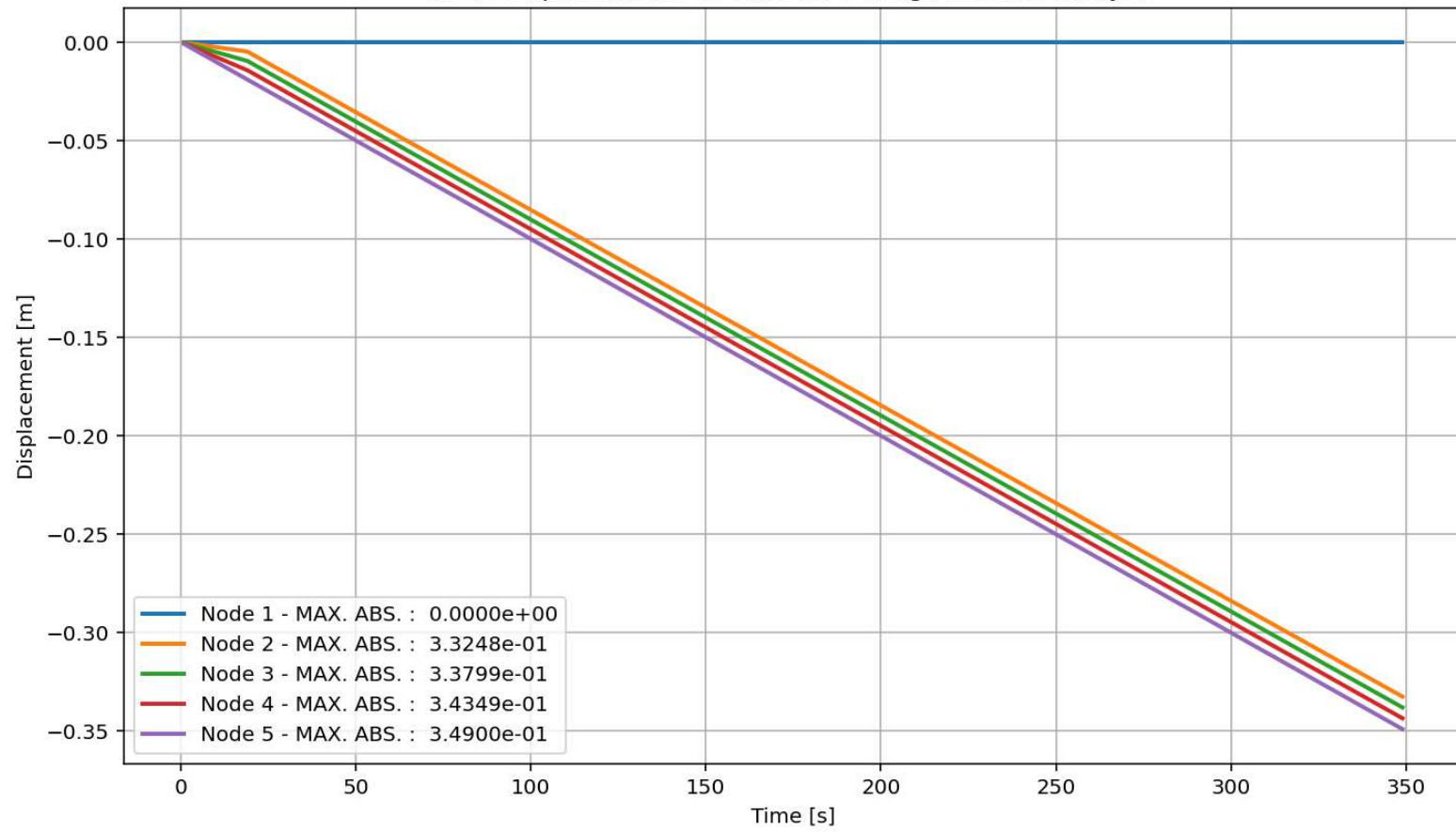
Period of Structure During Pushover Analysis

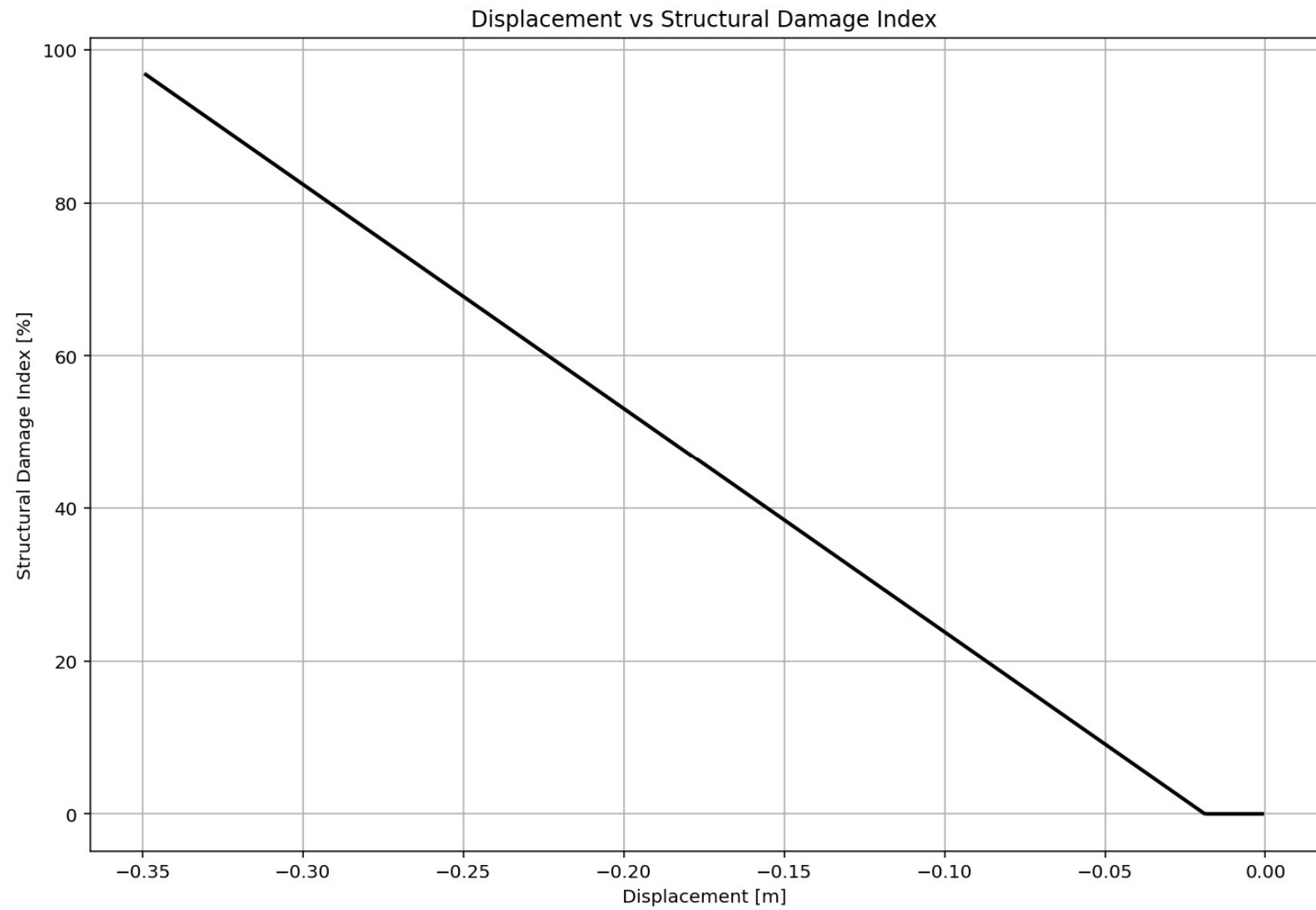


elements force vs Time During Pushover Analysis



Node Displacements vs Time for During Pushover Analysis





Spyder (Python 3.12)

File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\Dell\Desktop\OPENSEES_FILES\+EIGHT_ANALY..._ANA\COMPUTE_EFFECTIVE_PROPERTIES_FUN_PUSHOVER.py

COMPUTE_EFFECTIVE_..._FREE_VIBRATION.py COMPUTE_EFFECTIVE_...ES_FUN_PUSHOVER.py

```
1  """ EQUIVALENT SDOF SYSTEM DERIVATION VIA DISPLACEMENT-BASED SEISMIC DESIGN PROCEDURE WITH P
2  # Change MDOF to SDOF System with Displacement Based Design Concept
3  # THIS PROGRAM WRITTEN BY SALAR DELAVAR GHASHGHAEI (QASHQAI)
4  """
5  This script implements a displacement-based pushover transformation,
6  converting a multi-degree-of-freedom (MDOF) system into an equivalent
7  single-degree-of-freedom (SDOF) system for seismic assessment.
8  It calculates effective modal properties-displacement, mass, and
9  stiffness-by weighting element forces and nodal displacements according
10 to a presumed deformed shape.
11 The derived effective period provides a simplified dynamic characteristic
12 for performance-based engineering. The visualizations effectively track the
13 evolution of these equivalent parameters throughout the nonlinear analysis steps.
14 """
15 #-----
16 displacement_X_1 = np.array(list(node_displacements.values())[1]) # DOF 02
17 displacement_X_2 = np.array(list(node_displacements.values())[2]) # DOF 03
18 displacement_X_3 = np.array(list(node_displacements.values())[3]) # DOF 04
19 displacement_X_4 = np.array(list(node_displacements.values())[4]) # DOF 05
20
21 ele_force_01 = np.array(list(ele_force.values())[0]) # ELEMENT 01
22 ele_force_02 = np.array(list(ele_force.values())[1]) # ELEMENT 02
23 ele_force_03 = np.array(list(ele_force.values())[2]) # ELEMENT 03
24 ele_force_04 = np.array(list(ele_force.values())[3]) # ELEMENT 04
25
26 STIFF_X_1 = np.abs(ele_force_01 / displacement_X_1)
27 STIFF_X_2 = np.abs(ele_force_02 / displacement_X_2)
28 STIFF_X_3 = np.abs(ele_force_03 / displacement_X_3)
29 STIFF_X_4 = np.abs(ele_force_04 / displacement_X_4)
30
31 MX2 = (MASS[0] * np.square(displacement_X_1) +
32        MASS[1] * np.square(displacement_X_2) +
33        MASS[2] * np.square(displacement_X_3) +
34        MASS[3] * np.square(displacement_X_4))
```

Effective Displacement - Median: 0.16549

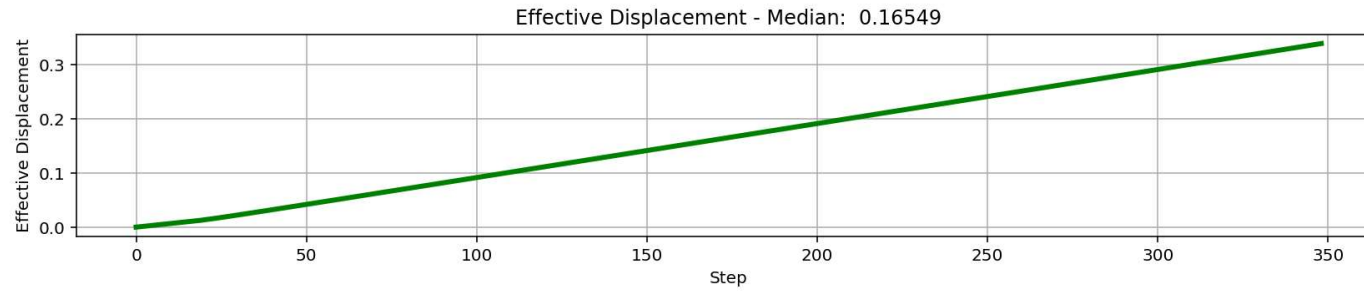
Effective Mass - Median: 4994.83302

Effective Stiffness - Median: 1384867.11610

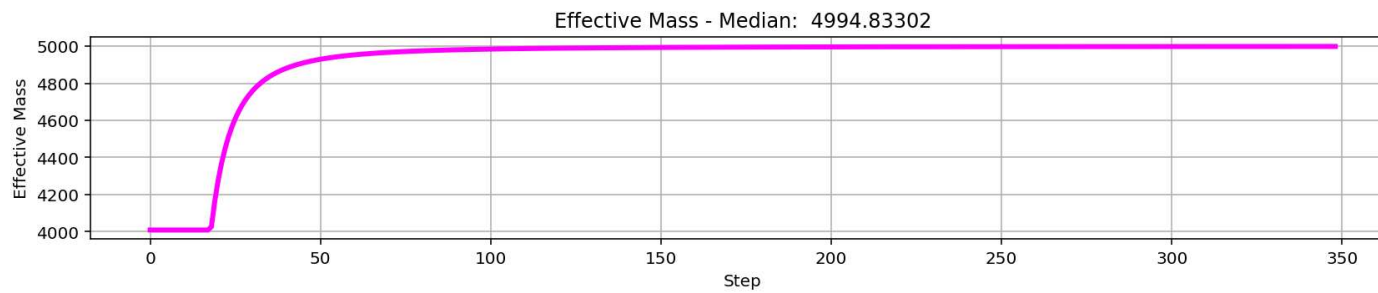
Effective Period - Median: 0.37734

IPython Console Files Help Variable Explorer Debugger Plots History

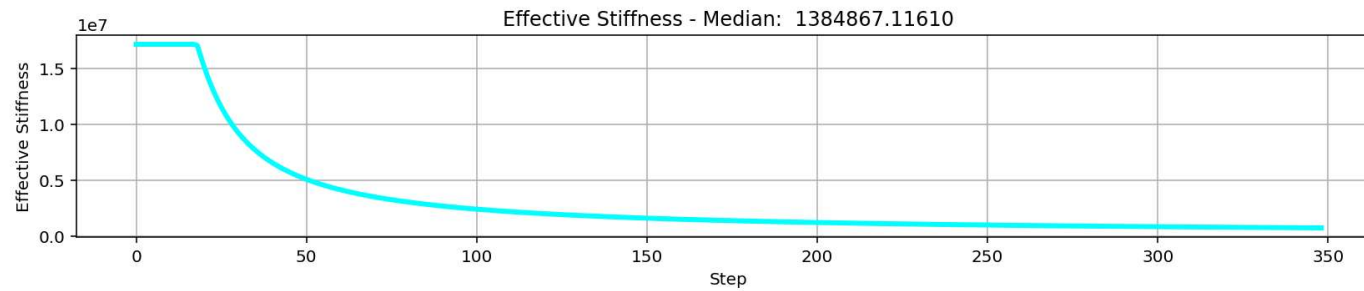
Inline Conda: anaconda3 (Python 3.12.7) LSP: Python Line 1, Col 1 UTF-8 CRLF RW Mem 41%



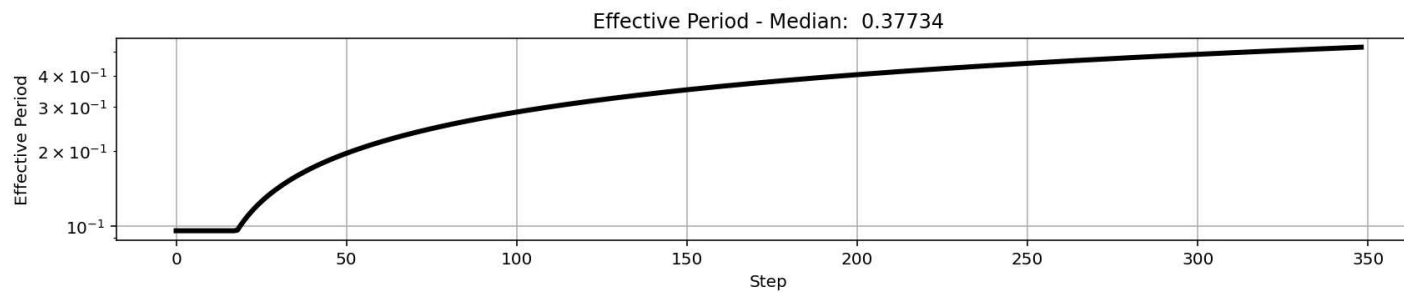
$$\Delta_d = \frac{\sum_{i=1}^n m_i \Delta_i^2}{\sum_{i=1}^n m_i \Delta_i}$$



$$m_{eff} = \frac{\sum_{i=1}^n m_i \Delta_i}{\Delta_d}$$



$$k_{eff} = \frac{\sum_{i=1}^n k_i \Delta_i}{\Delta_d}$$



$$T_{eff} = \frac{2\pi}{\sqrt{\frac{k_{eff}}{m_{eff}}}}$$

CYCLIC DISPLACEMENT ANALYSIS

Spyder (Python 3.12)

File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\ DELL\Desktop\OPENSEES_FILES\+EIGHT_ANALY...N_CAPACITY_INDEX\22_MDOF_8_ANA\P(t)_MDOF_8_ANA.py

P(t)_MDOF_8_ANA.py X

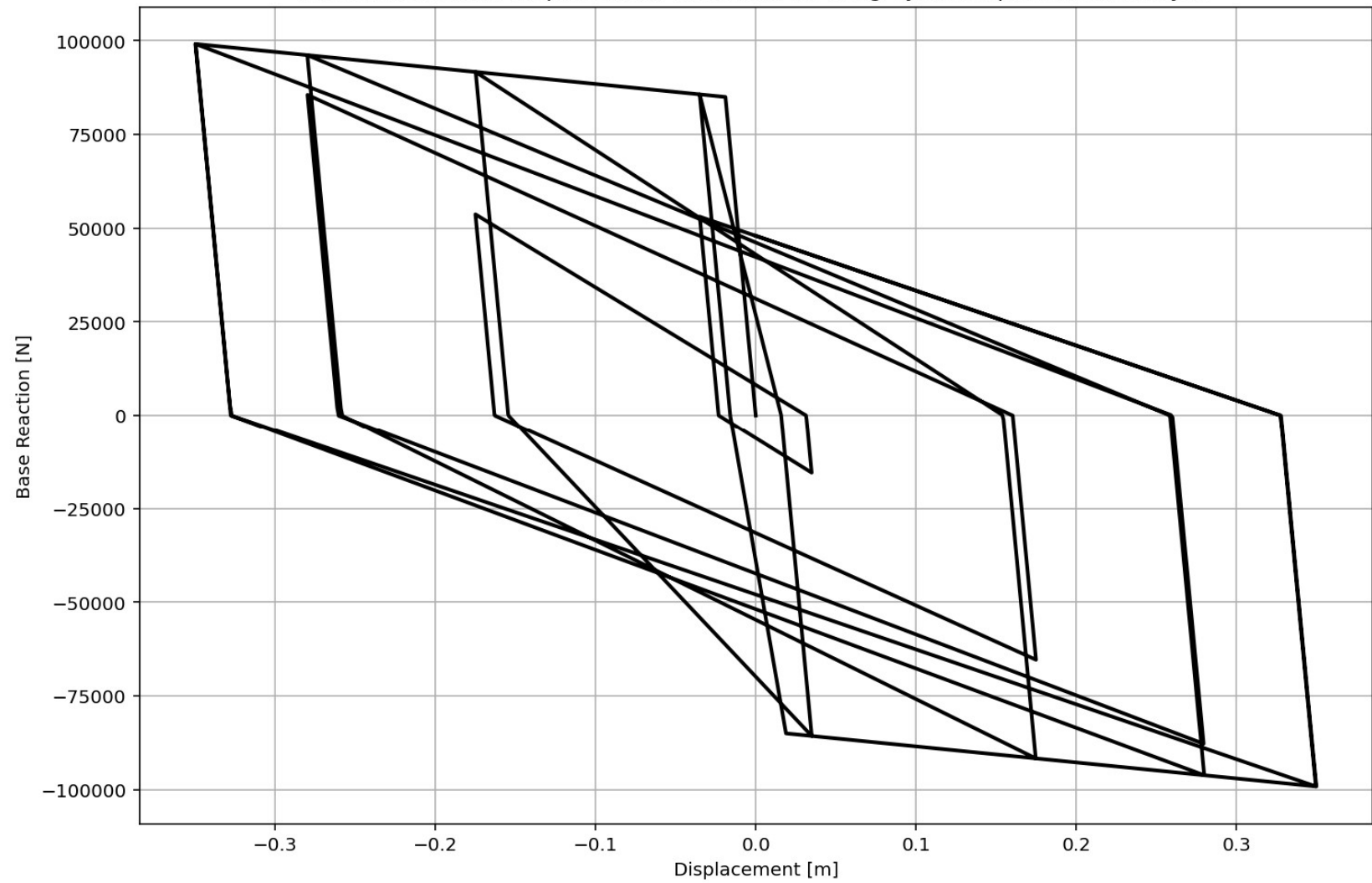
```
284 if ANAL_TYPE == 'CYCLIC_DISPLACEMENT': # STATIC TIME-HISTORY ANALYSIS
285     IDctrlDOF = 1 # 1: Horizontal Displacement - 2: Vertical Displacement
286     DMAX = -0.35 # [m] Max. Displacement
287     n_points = 10000
288     # 4. CYCLIC DISPLACEMENT PROTOCOL
289     # Key strain points (same logic as your protocol)
290     key_disp = np.array([
291         0.0,
292         0.1*DMAX, -0.1*DMAX,
293         0.5*DMAX, -0.5*DMAX,
294         0.8*DMAX, -0.8*DMAX,
295         DMAX, -DMAX,
296         0.1*DMAX, -0.1*DMAX,
297         0.5*DMAX, -0.5*DMAX,
298         0.8*DMAX, -0.8*DMAX,
299         DMAX, -DMAX,
300         0.0
301     ])
302
303     # Generate 1000-point displacement protocol
304     disp_protocol = np.interp(
305         np.linspace(0, len(key_disp) - 1, n_points),
306         np.arange(len(key_disp)),
307         key_disp
308     )
309
310     ops.analysis('Static')
311     STEP = 0.0
312     for target_disp in disp_protocol:
313         current_disp = ops.nodeDisp(center_node, 1) # DISPLACEMENT APPLIED IN MIDDLE N...
314         dU = target_disp - current_disp
315         ops.integrator('DisplacementControl', center_node, IDctrlDOF, dU)
316         OK = ops.analyze(1)
317         S02.ANALYSIS(OK, 1, MAX_TOLERANCE, MAX_ITERATIONS)
```

Base Reaction and Displacement of Structure During Cyclic-Displacement Analysis

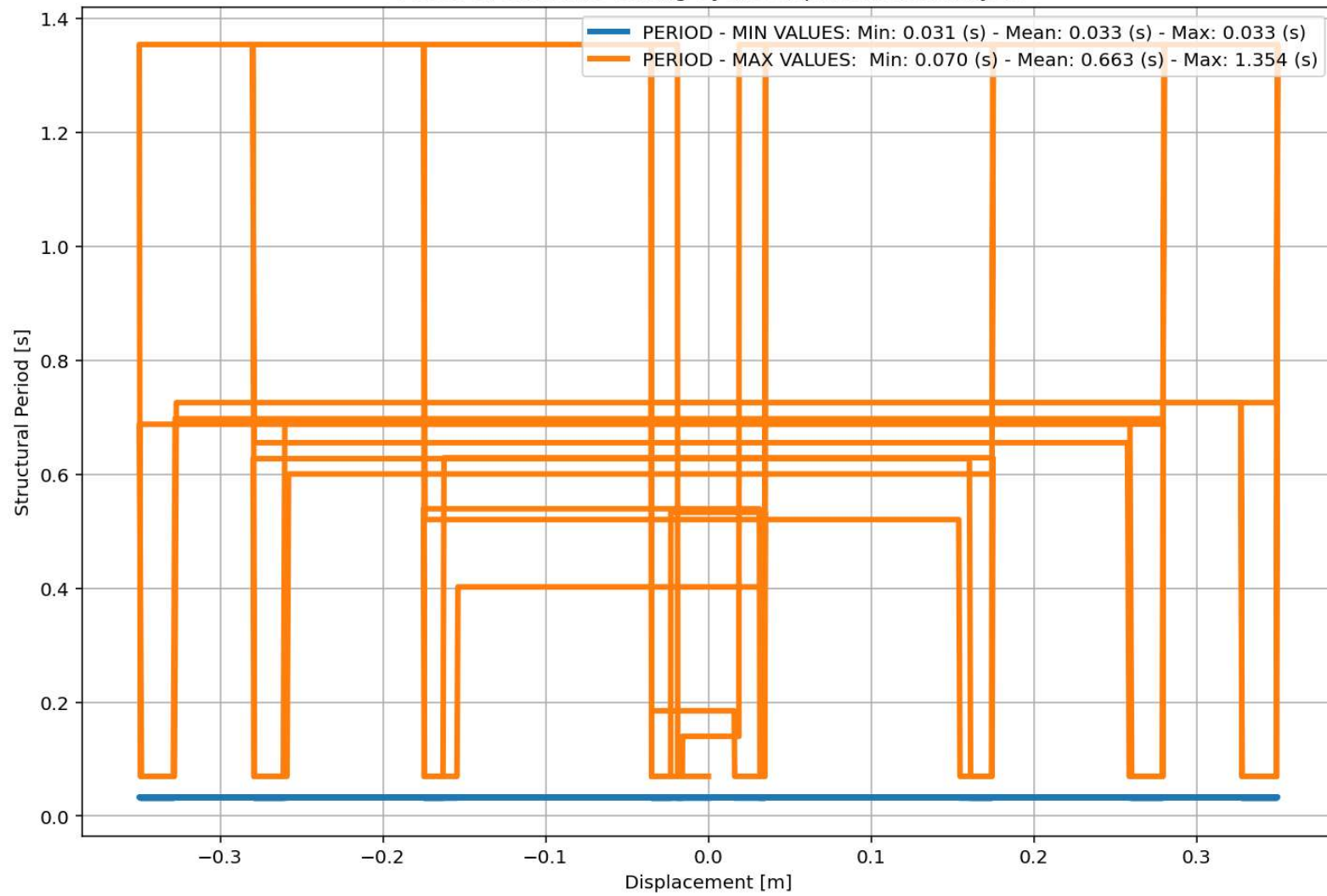
IPython Console Files Help Variable Explorer Debugger Plots History

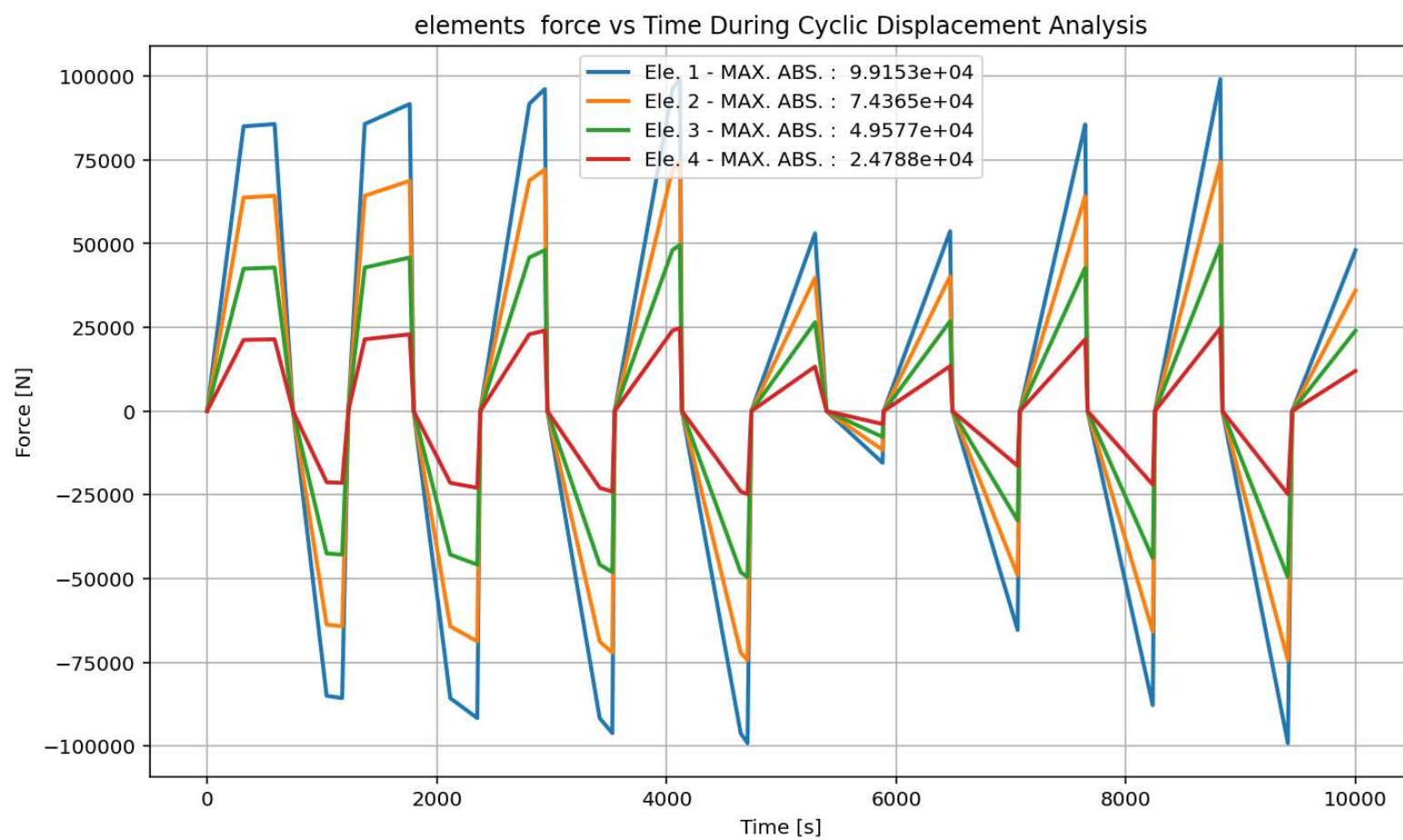
Inline Conda: anaconda3 (Python 3.12.7) ✓ LSP: Python Line 1307, Col 23 UTF-8 CRLF RW Mem 45%

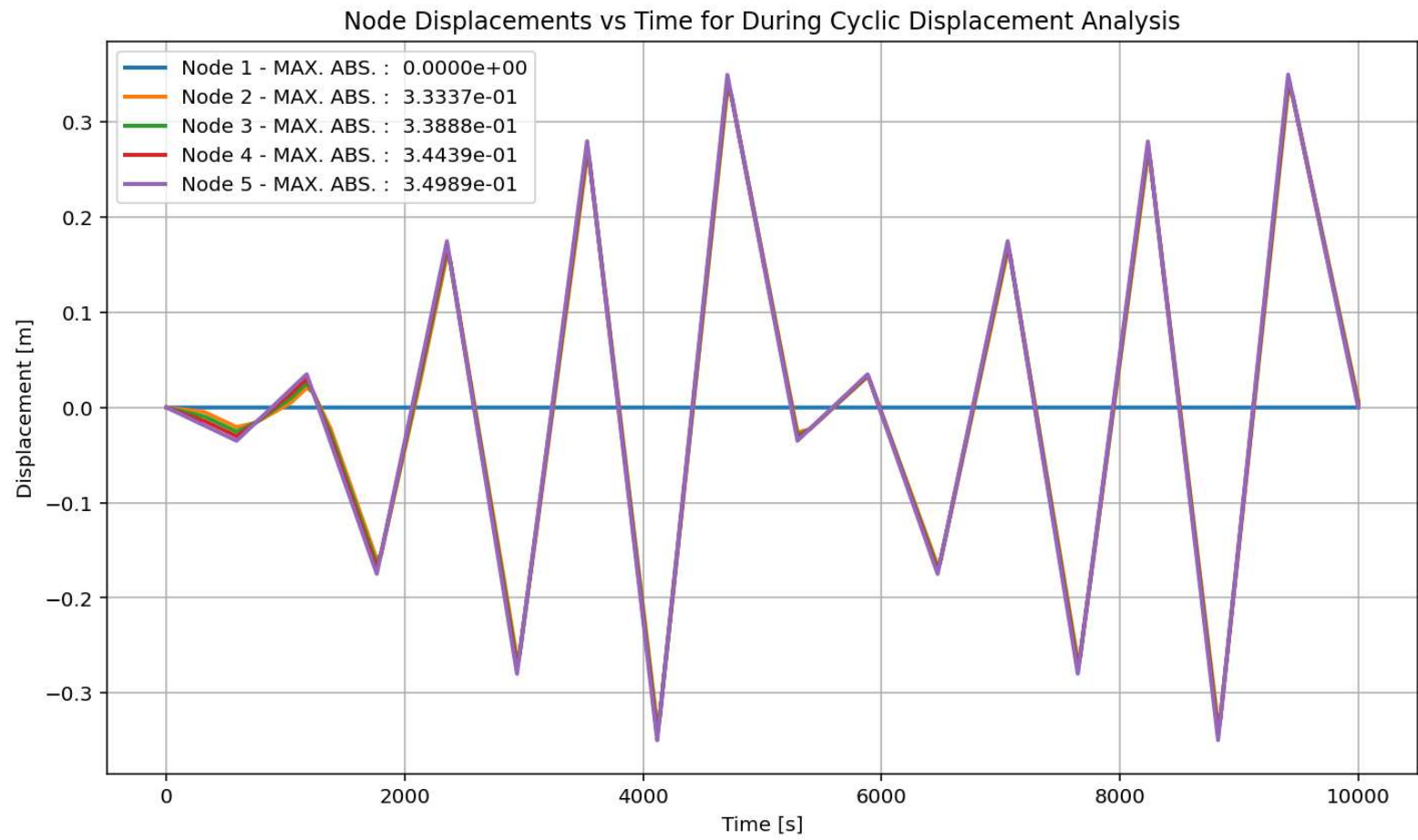
Base Reaction and Displacement of Structure During Cyclic-Displacement Analysis

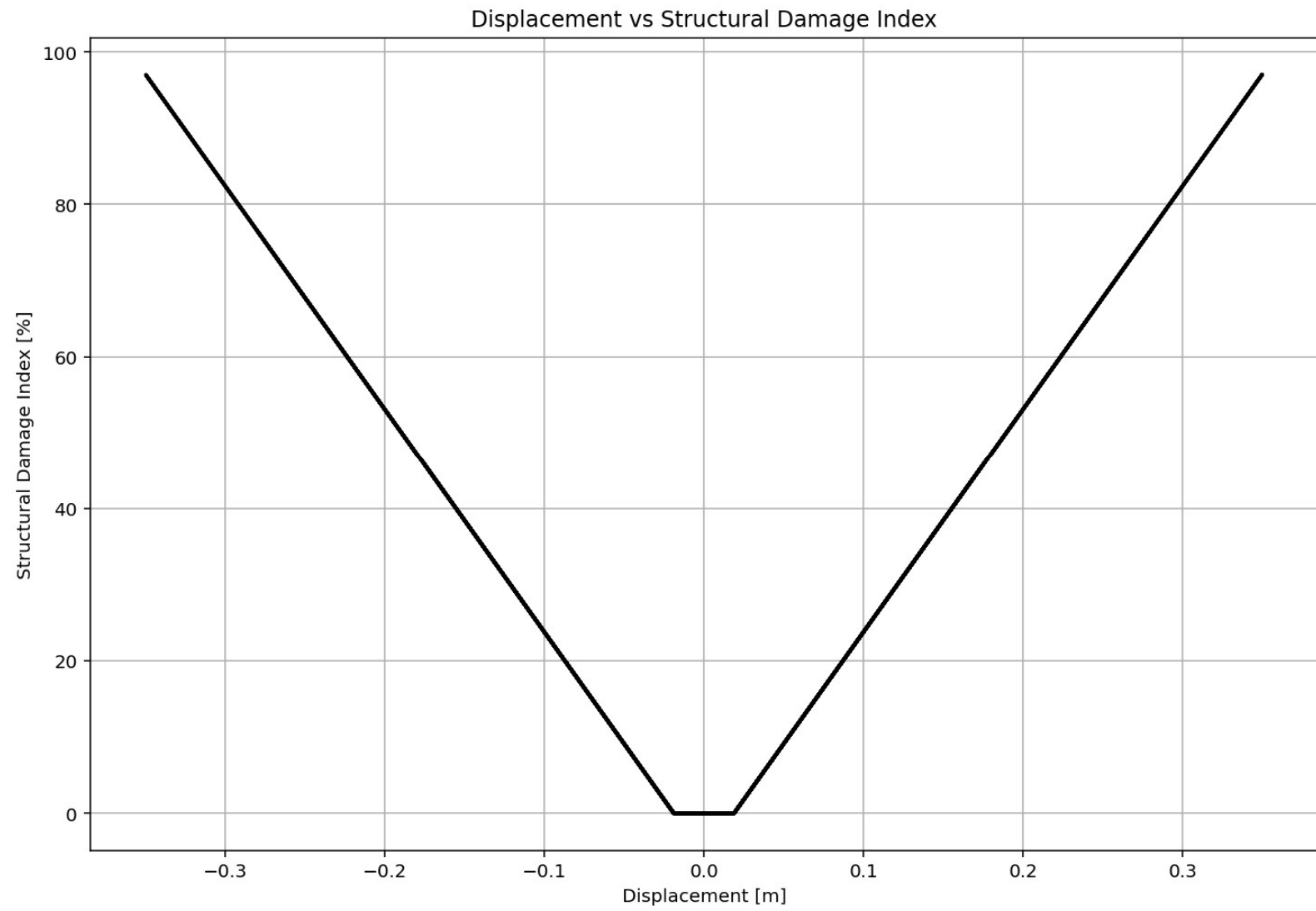


Period of Structure During Cyclic Displacement Analysis

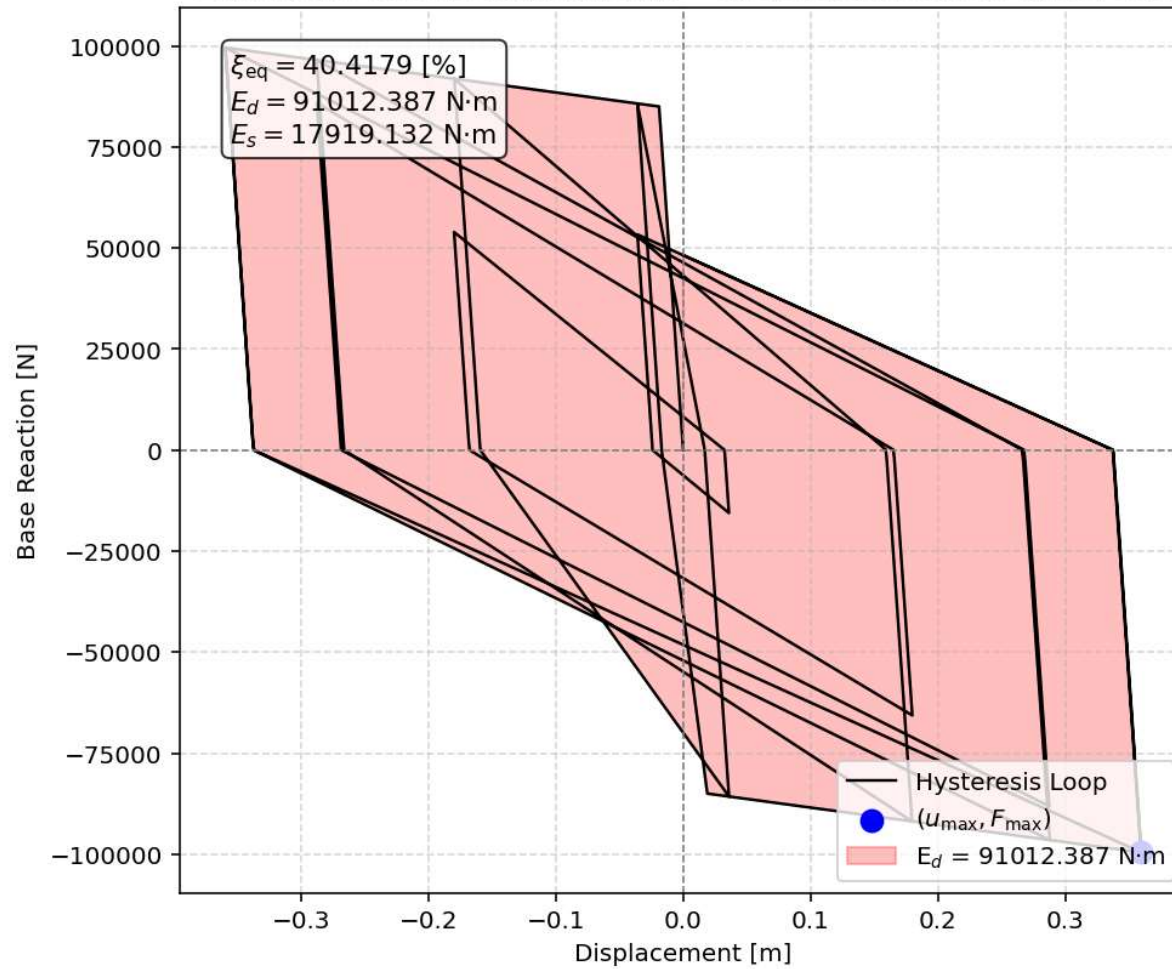








Equivalent viscous damping ratio - Cyclic Displacement Hysteresis



STATIC TIME-HISTORY WITH EXTERNAL TIME-DEPENDENT LOADING ANALYSIS

Spyder (Python 3.12)

File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\Dell\Desktop\OPENSEES_FILES\HEIGHT_ANALY...N_CAPACITY_INDEX\22_MDOF_8_ANA\P(t)_MDOF_8_ANA.py

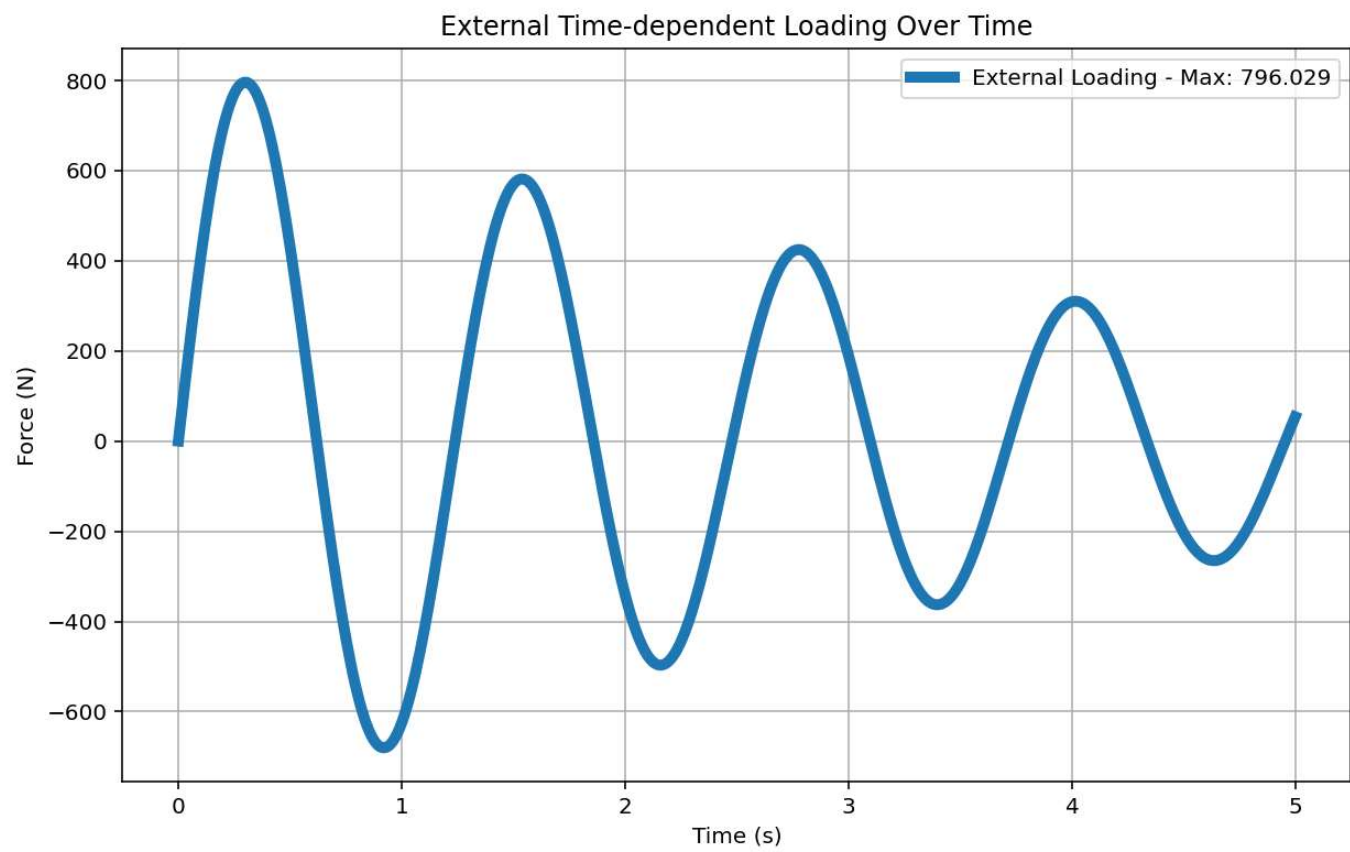
P(t)_MDOF_8_ANA.py

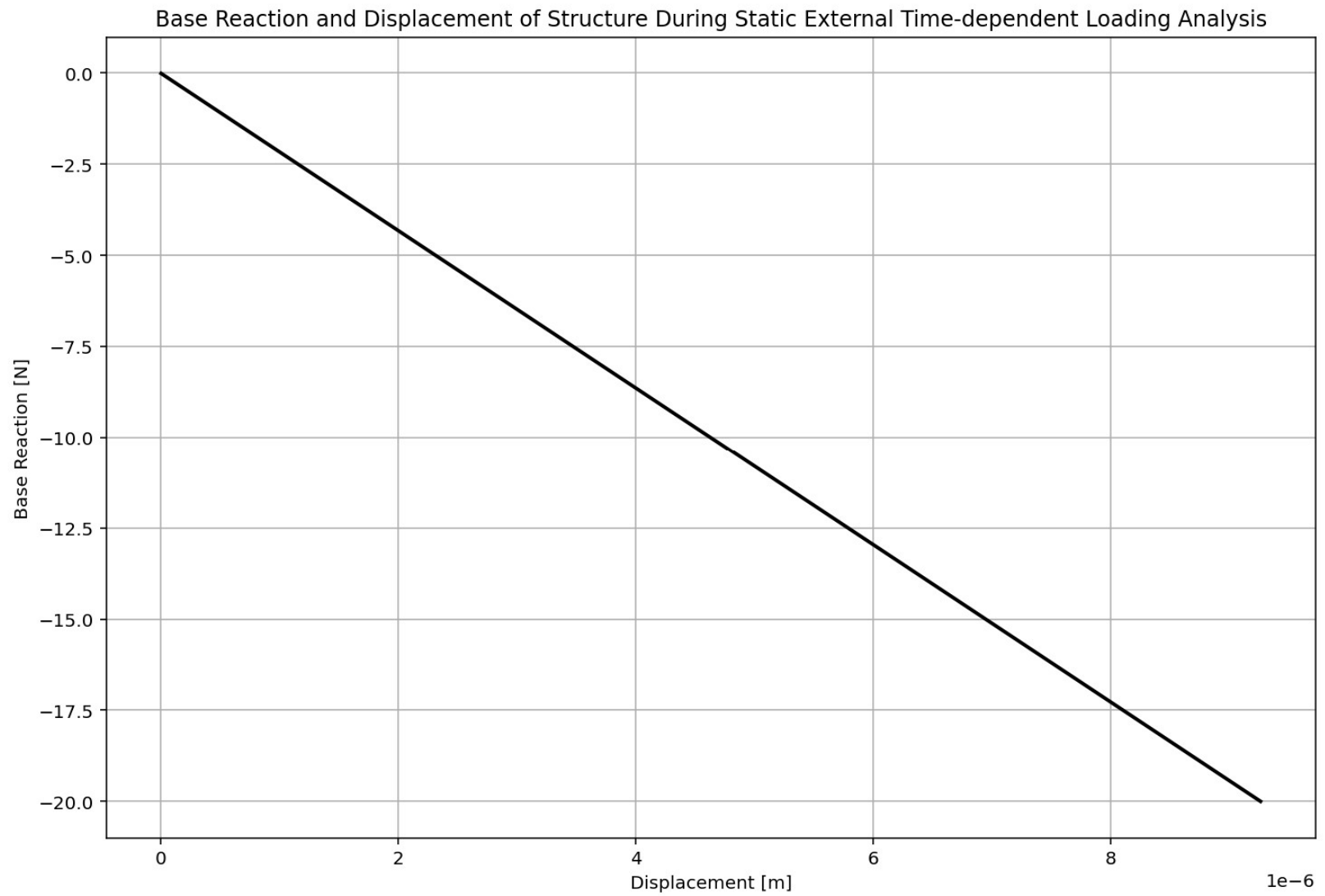
```
349 if ANAL_TYPE == 'STATIC EXTERNAL TIME-DEPENDENT LOADING': # STATIC TIME-HISTORY ANALYSIS
350     %% DEFINE EXTERNAL TIME-DEPENDENT LOADING PROPERTIES
351     # IN HERE ANALYSIS TIME AND DURATION ARE LOAD STEPS
352     duration = 20.0 # [s] Analysis duration
353     dt = 0.01 # [s] Time step
354     DT = dt # [s] Time step
355     DT_time = 5.0 # [s] Total external Load Analysis Durations [*****]
356     force_amplitude = 5960.0 # [N] Amplitude Force
357     omega_DT = 5.0715 # [rad/s] Natural angular frequency
358     time_steps = int(duration/dt)
359
360     # Check function
361     def CHECK_FUN(DT_time ,duration):
362         if DT_time > duration:
363             print('\n\nAnalysis Duration Must be greater than External Load Duration\n\n')
364             exit()
365         return -1
366
367     CHECK_FUN(DT_time ,duration)
368     def EXTERNAL_TIME_DEPENDENT(force_amplitude, omega_DT, DT, DT_time): # P(t) = P0 sin(omega*t)
369         import numpy as np
370         import matplotlib.pyplot as plt
371         # External Load Durations
372         num_steps = int(DT_time / DT)
373         load_time = np.linspace(0, DT_time, num_steps)
374         target_frequency = 1.0 * omega_DT # Target excitation frequency
375         DT_load = force_amplitude * np.sin(target_frequency * load_time)
376         # Plot External Time-dependent Loading
377         plt.figure(figsize=(10, 6))
378         plt.plot(load_time, DT_load, label=f'External Loading - Max: {np.max(DT_load)}')
379         plt.title('External Time-dependent Loading Over Time')
380         plt.xlabel('Time (s)')
381         plt.ylabel('Force (N)')
382         plt.grid(True)
```

External Time-dependent Loading Over Time

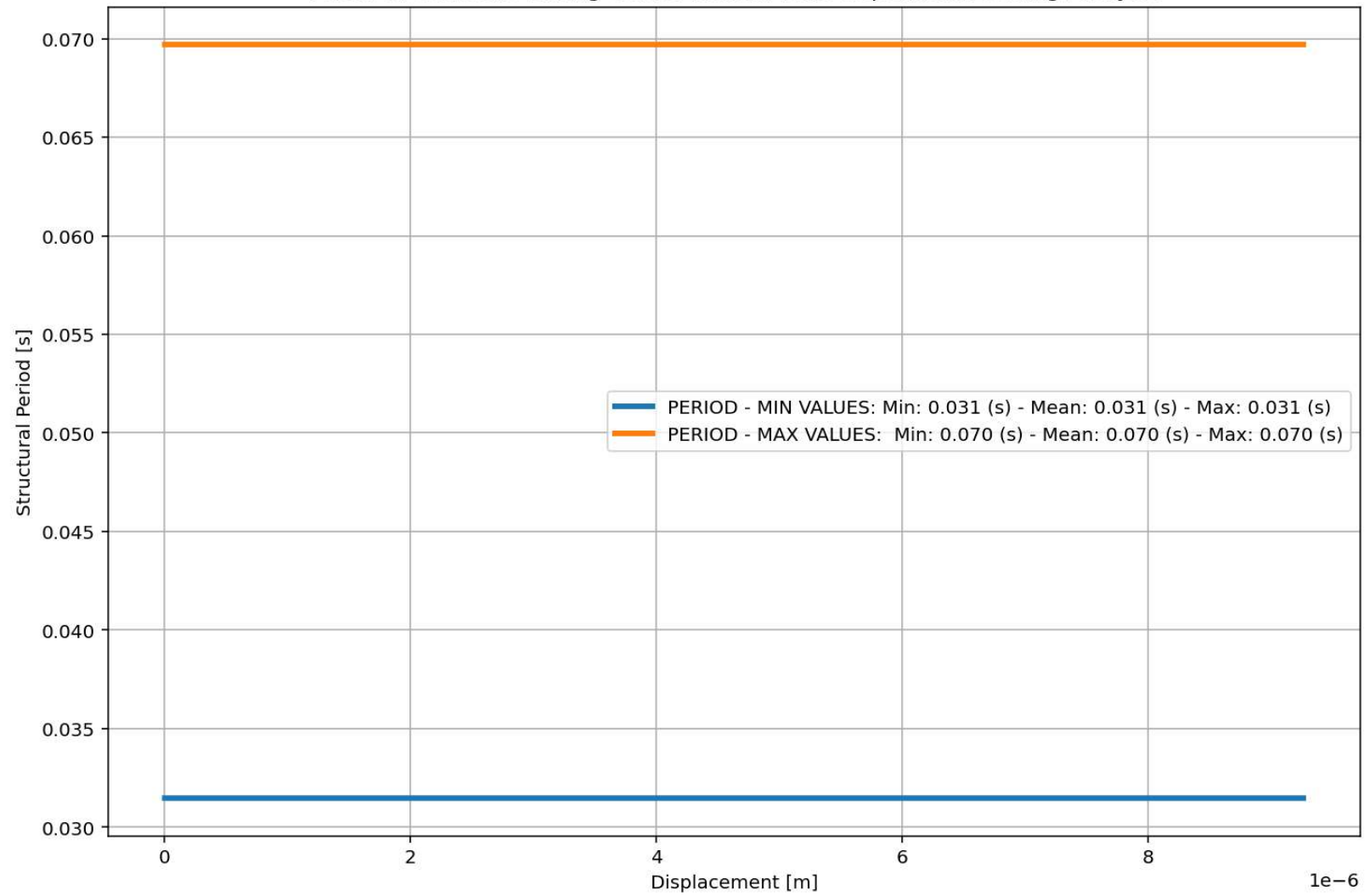
IPython Console Files Help Variable Explorer Debugger Plots History

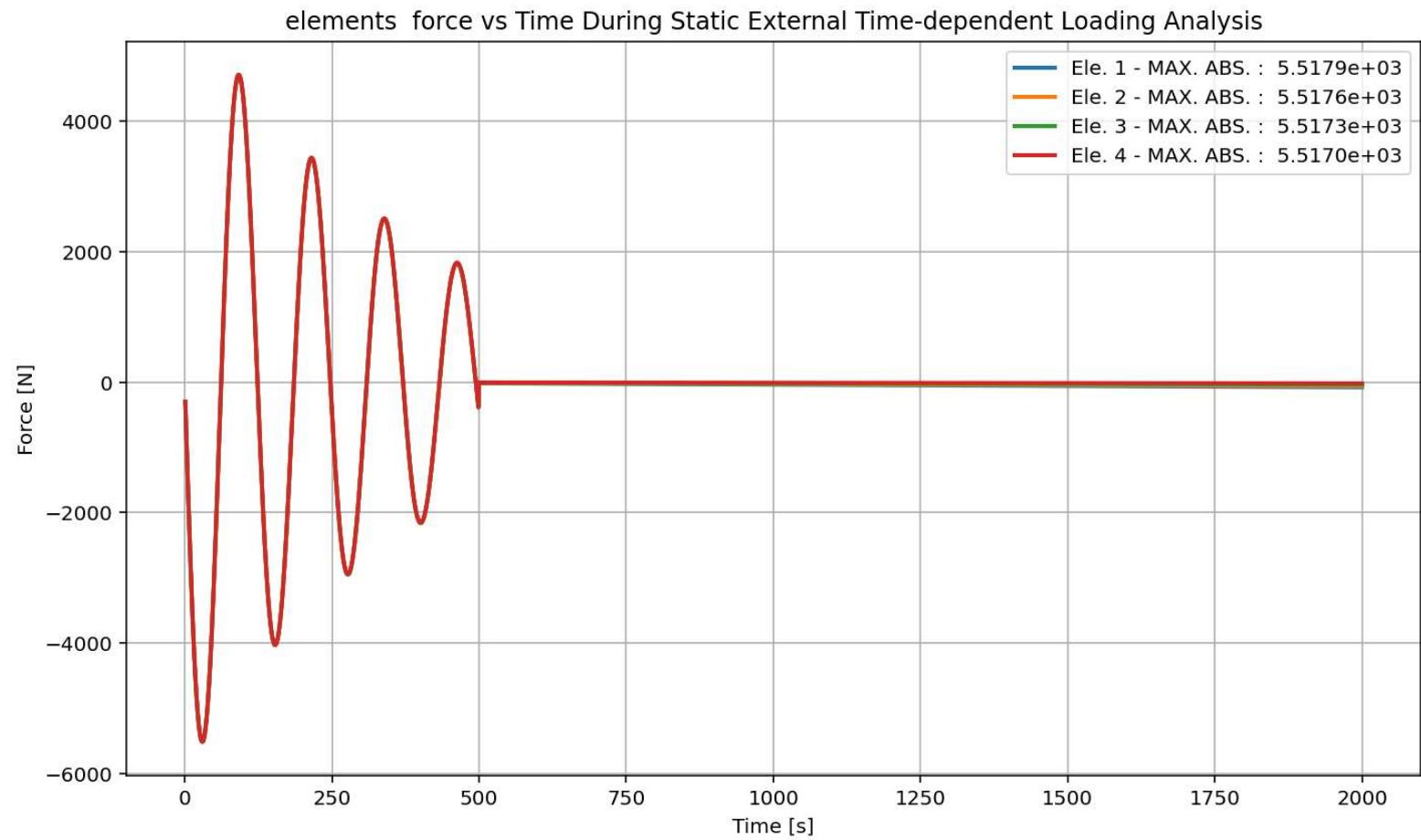
Inline Conda: anaconda3 (Python 3.12.7) LSP: Python Line 1307, Col 23 UTF-8 CRLF RW Mem 45%



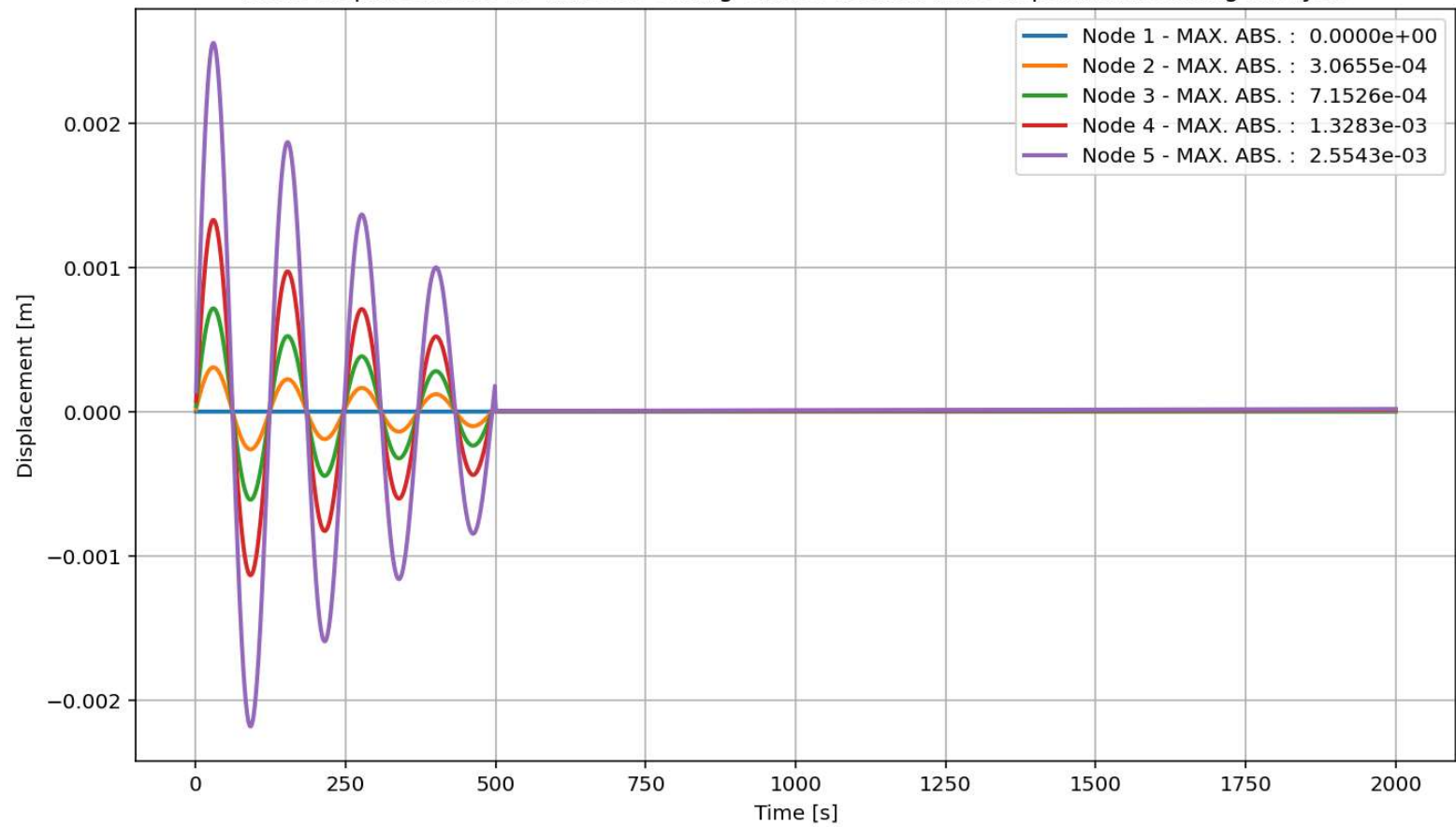


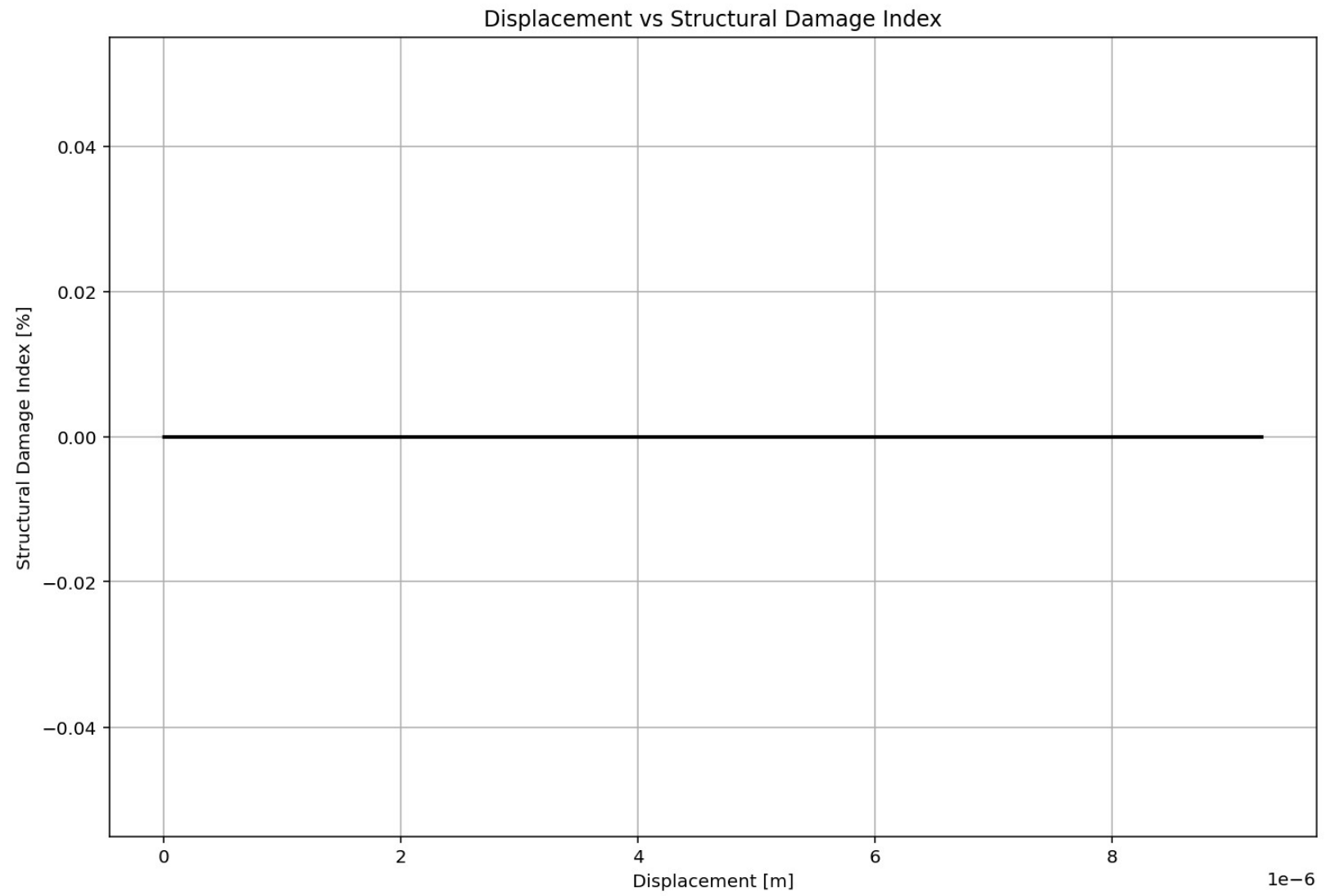
Period of Structure During Static External Time-dependent Loading Analysis





Node Displacements vs Time for During Static External Time-dependent Loading Analysis





DYNAMIC TIME-HISTORY WITH EXTERNAL TIME-DEPENDENT LOADING ANALYSIS

Spyder (Python 3.12)

File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\De\l\Desktop\OPENSEES_FILES\+EIGHT_ANALY...N_CAPACITY_INDEX\22_MDOF_8_ANA\P(t)_MDOF_8_ANA.py

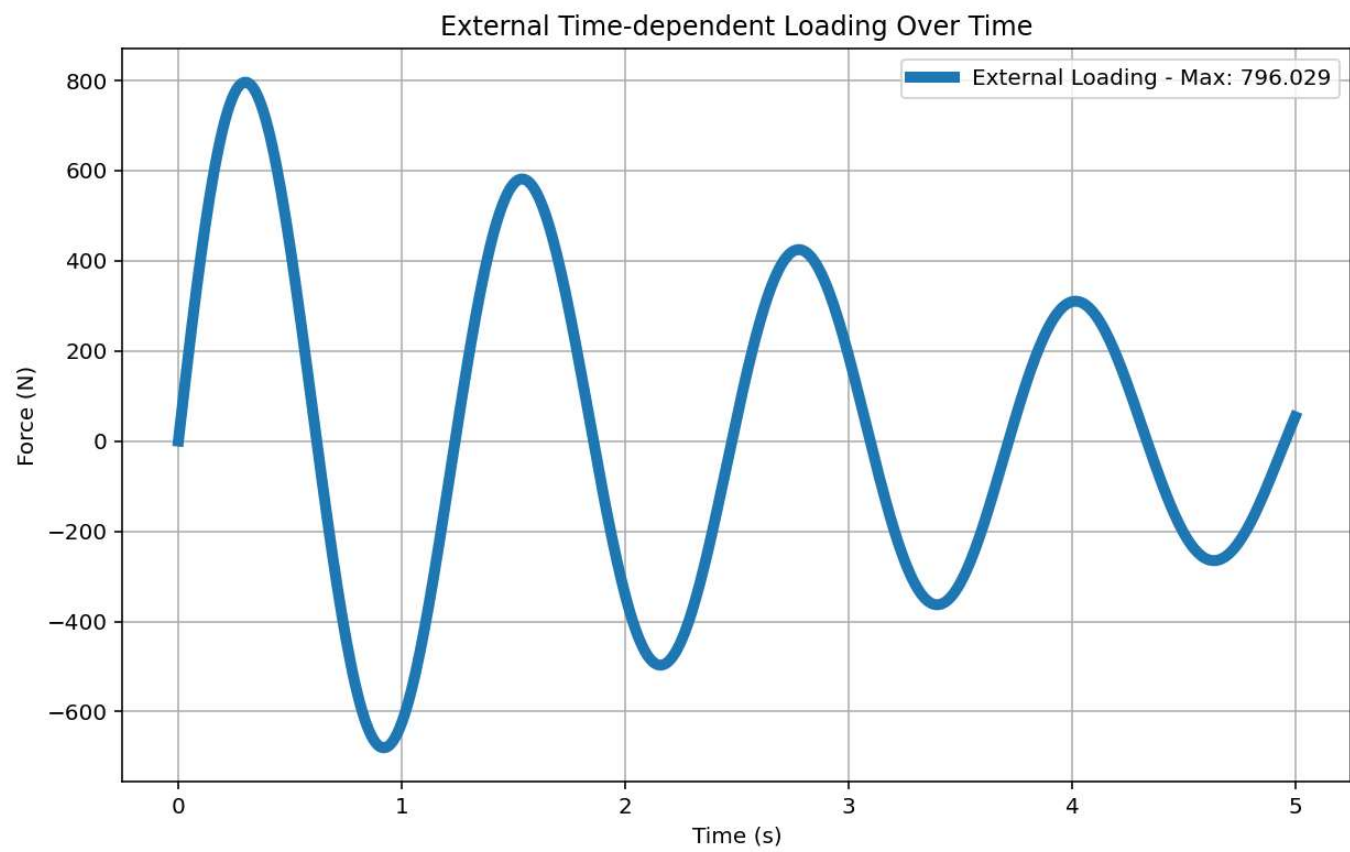
P(t)_MDOF_8_ANA.py X

```
462 if ANAL_TYPE == 'DYNAMIC EXTERNAL TIME-DEPENDENT LOADING': # DYNAMIC TIME-HISTORY ANAL
463     %% DEFINE EXTERNAL TIME-DEPENDENT LOADING PROPERTIES
464     duration = 20.0          # [s] Analysis duration
465     dt = 0.01               # [s] Time step
466     DT = dt                 # [s] Time step
467     DT_time = 5.0           # [s] Total external Load Analysis Durations [*****]
468     force_amplitude = 5960.0 # [N] Amplitude Force
469     omega_DT = 5.0715        # [rad/s] Natural angular frequency
470     DR = 0.03               # Damping Ratio
471
472     # Check function
473     def CHECK_FUN(DT_time ,duration):
474         if DT_time > duration:
475             print('\n\nAnalysis Duration Must be greater than External Load Duration\n')
476             exit()
477         return -1
478
479     CHECK_FUN(DT_time ,duration)
480     def EXTERNAL_TIME_DEPENDENT(force_amplitude, omega_DT, DT, DT_time): # P(t) = P0 sin(omega_DT * t)
481         import numpy as np
482         import matplotlib.pyplot as plt
483         # External Load Durations
484         num_steps = int(DT_time / DT)
485         load_time = np.linspace(0, DT_time, num_steps)
486         target_frequency = 1.0 * omega_DT # Target excitation frequency
487         DT_load = force_amplitude * np.sin(target_frequency * load_time)
488         # Plot External Time-dependent Loading
489         plt.figure(figsize=(10, 6))
490         plt.plot(load_time, DT_load, label=f'External Loading - Max: {np.max(DT_Load):.1f}')
491         plt.title('External Time-dependent Loading Over Time')
492         plt.xlabel('Time (s)')
493         plt.ylabel('Force (N)')
494         plt.grid(True)
495         plt.legend()
```

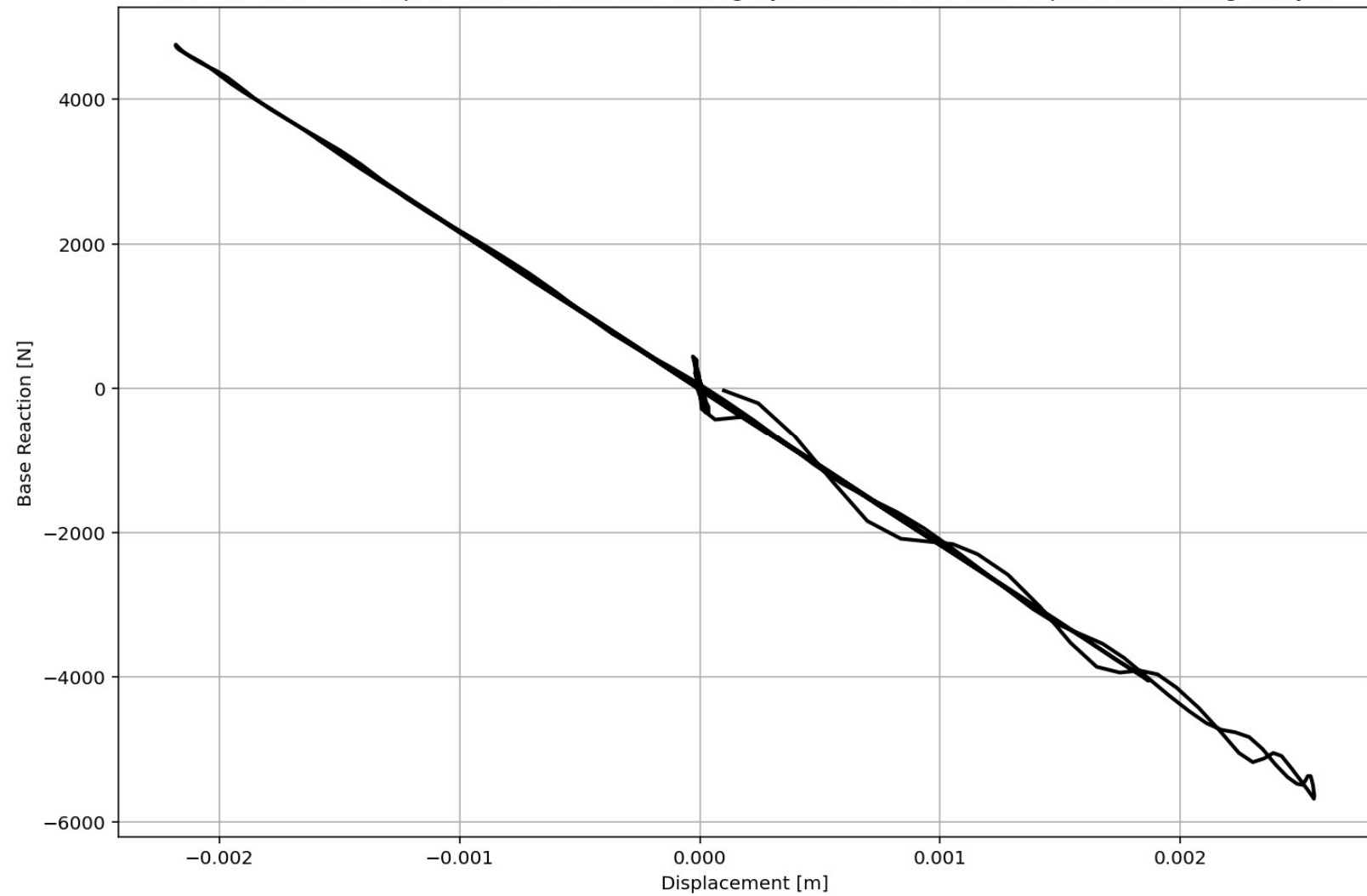
External Time-dependent Loading Over Time

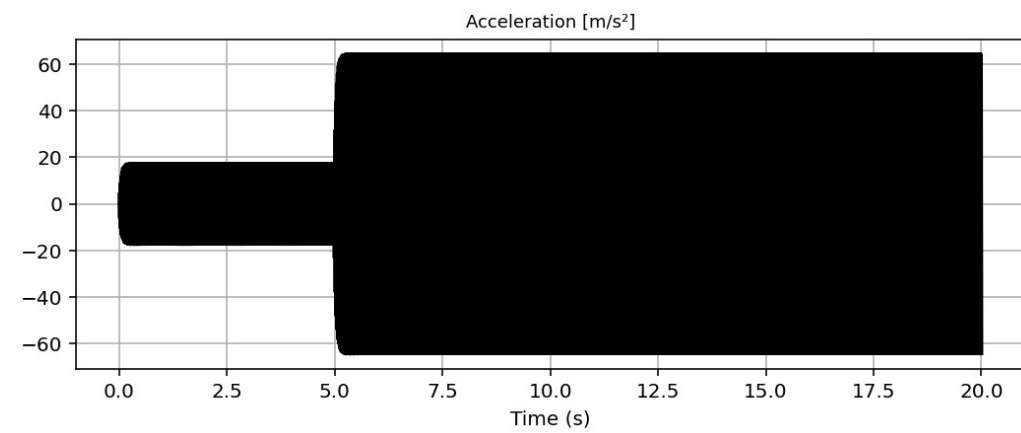
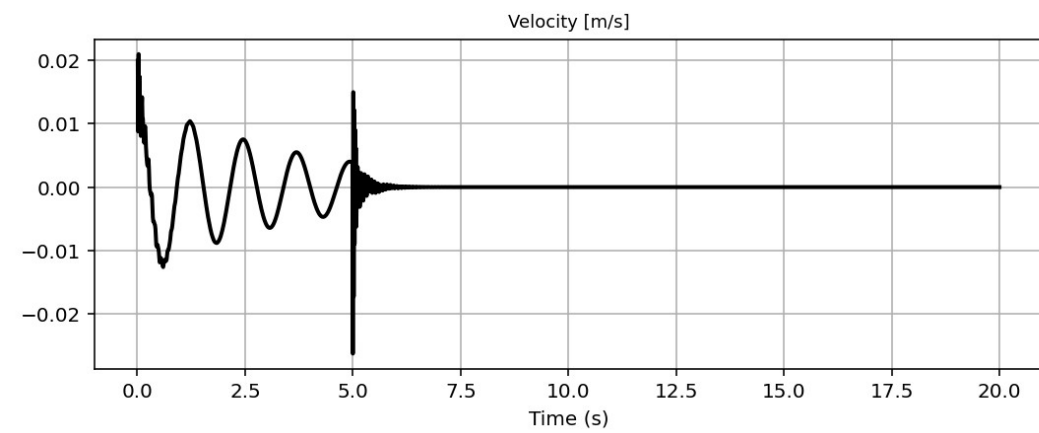
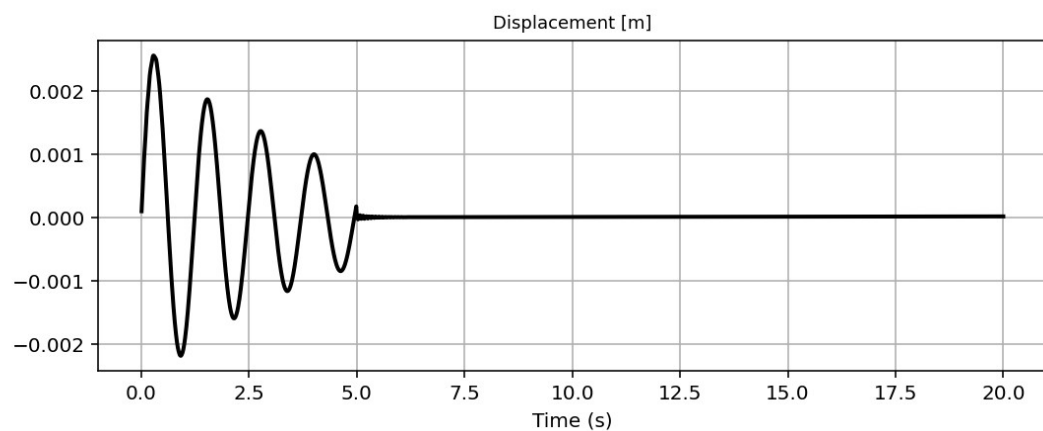
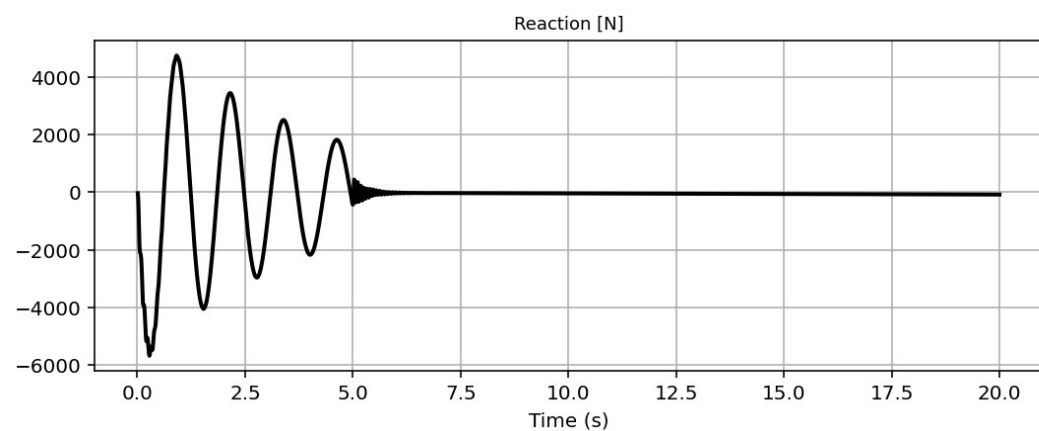
IPython Console Files Help Variable Explorer Debugger Plots History

Inline Conda: anaconda3 (Python 3.12.7) ✓ LSP: Python Line 1307, Col 23 UTF-8 CRLF RW Mem 46%

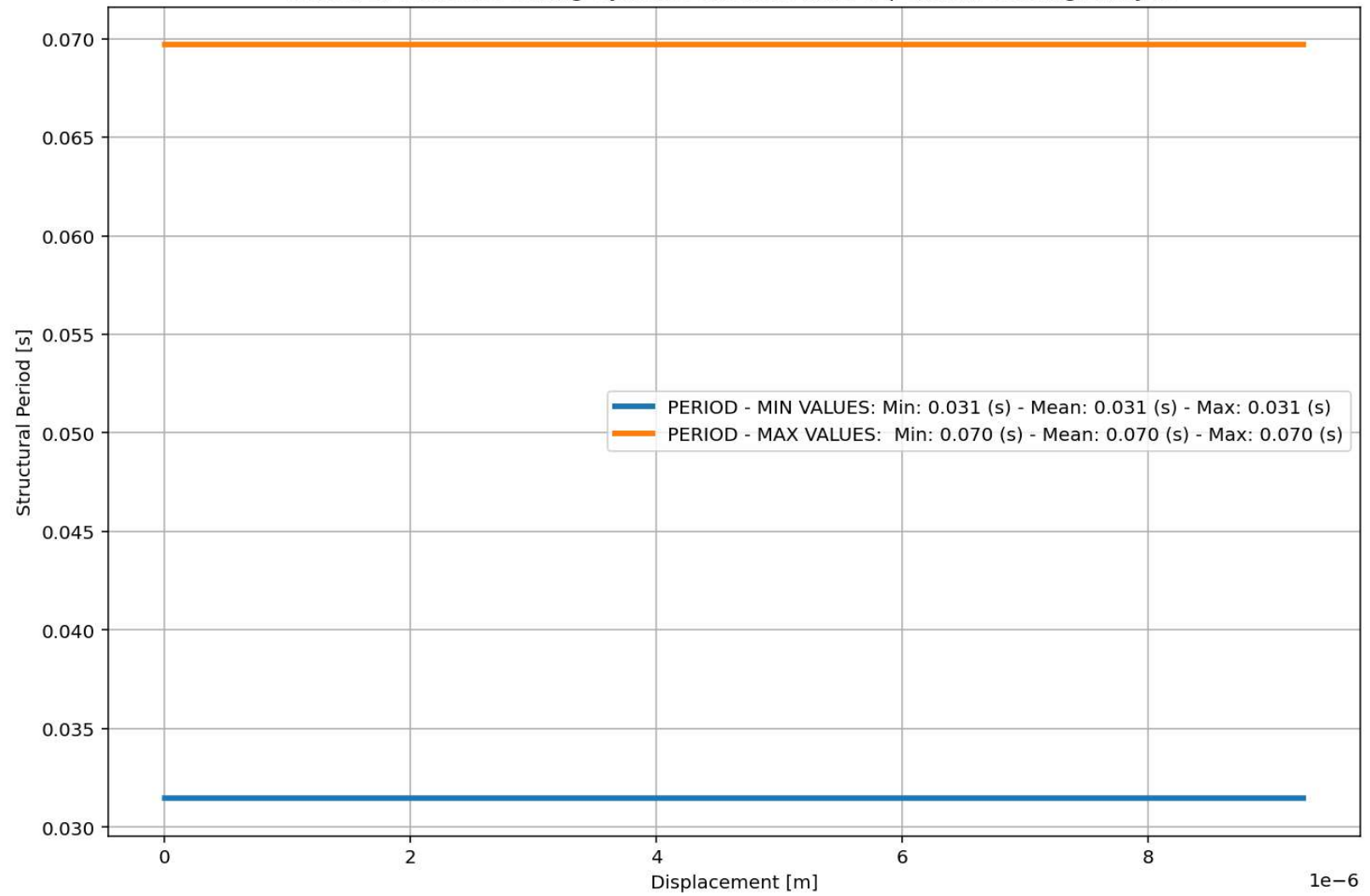


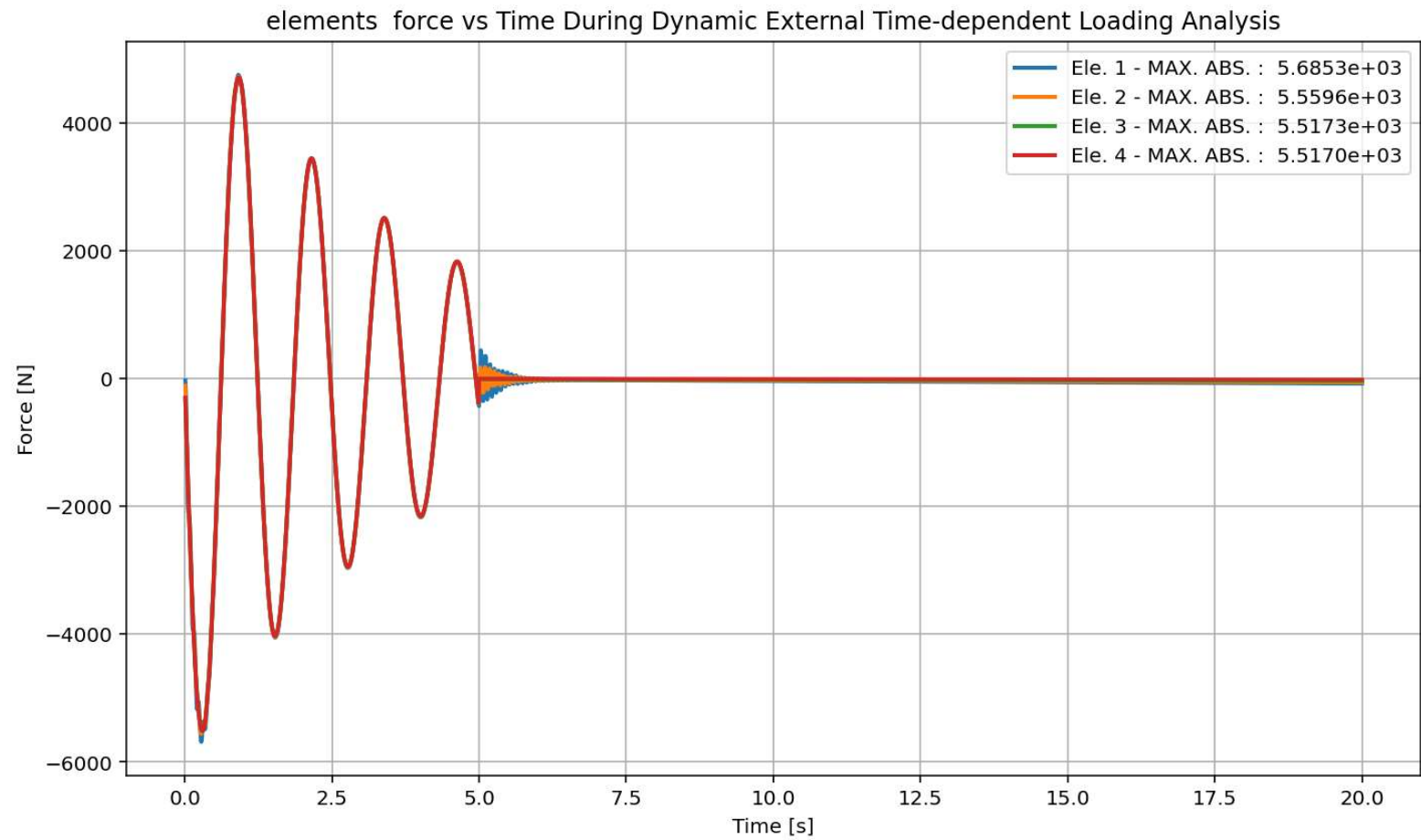
Base Reaction and Displacement of Structure During Dynamic External Time-dependent Loading Analysis

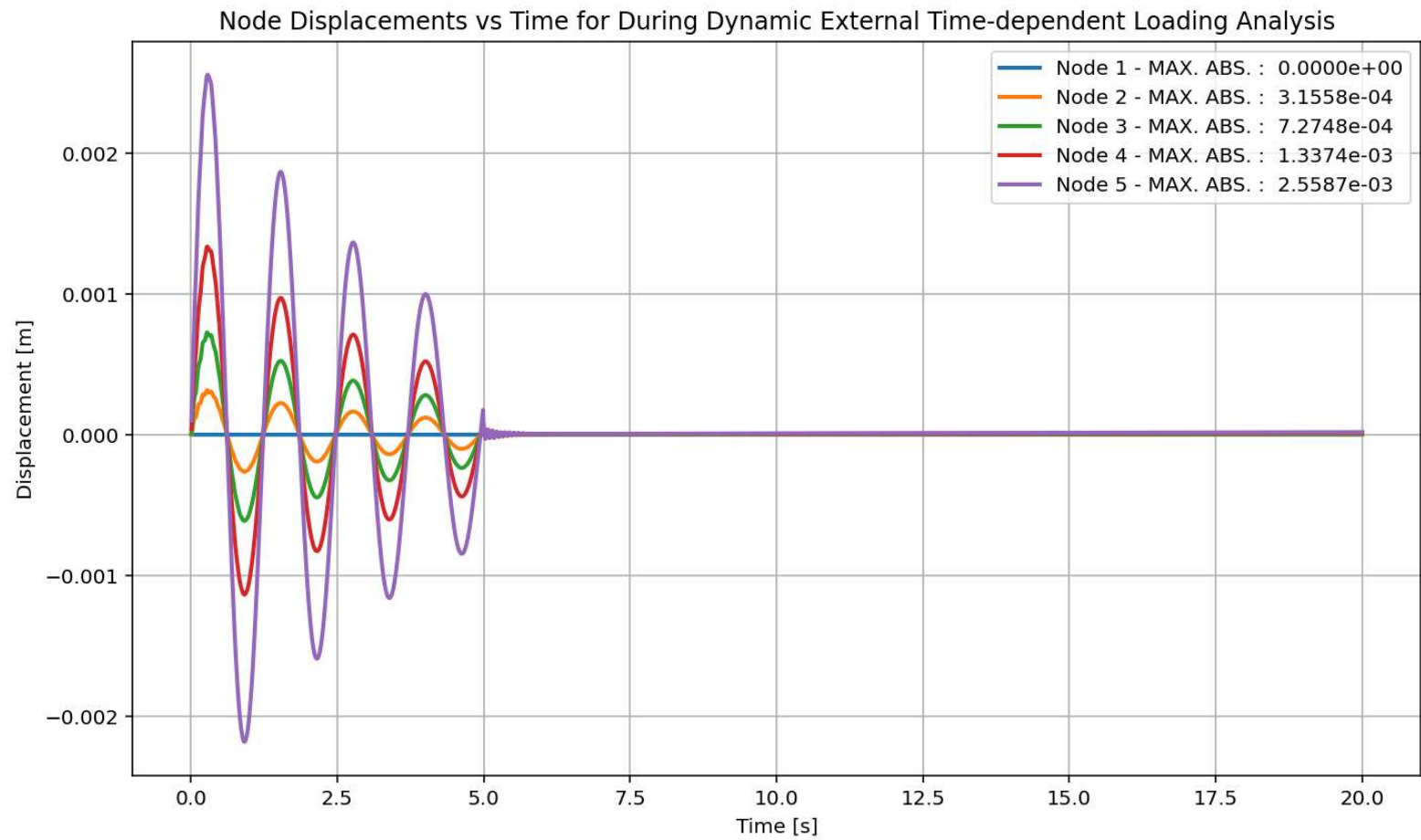


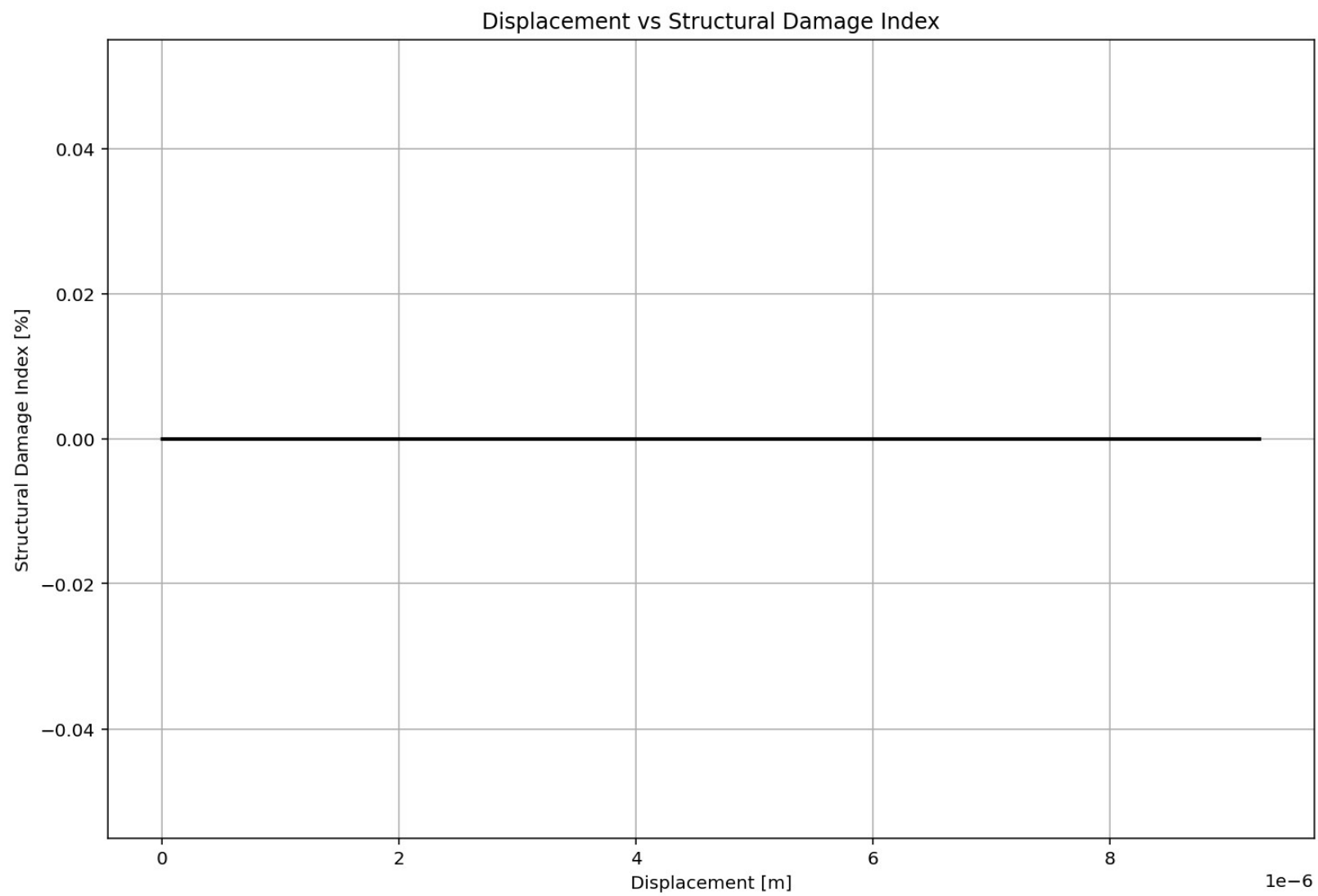


Period of Structure During Dynamic External Time-dependent Loading Analysis









FREE-VIBRATION ANALYSIS

Spyder (Python 3.12)

File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\Dell\Desktop\OPENSEES_FILES\+EIGHT_ANALY...N_CAPACITY_INDEX\22_MDOF_8_ANA\P(t)_MDOF_8_ANA.py

P(t)_MDOF_8_ANA.py X COMPUTE_EFFECTIVE_..._FREE_VIBRATION.py X

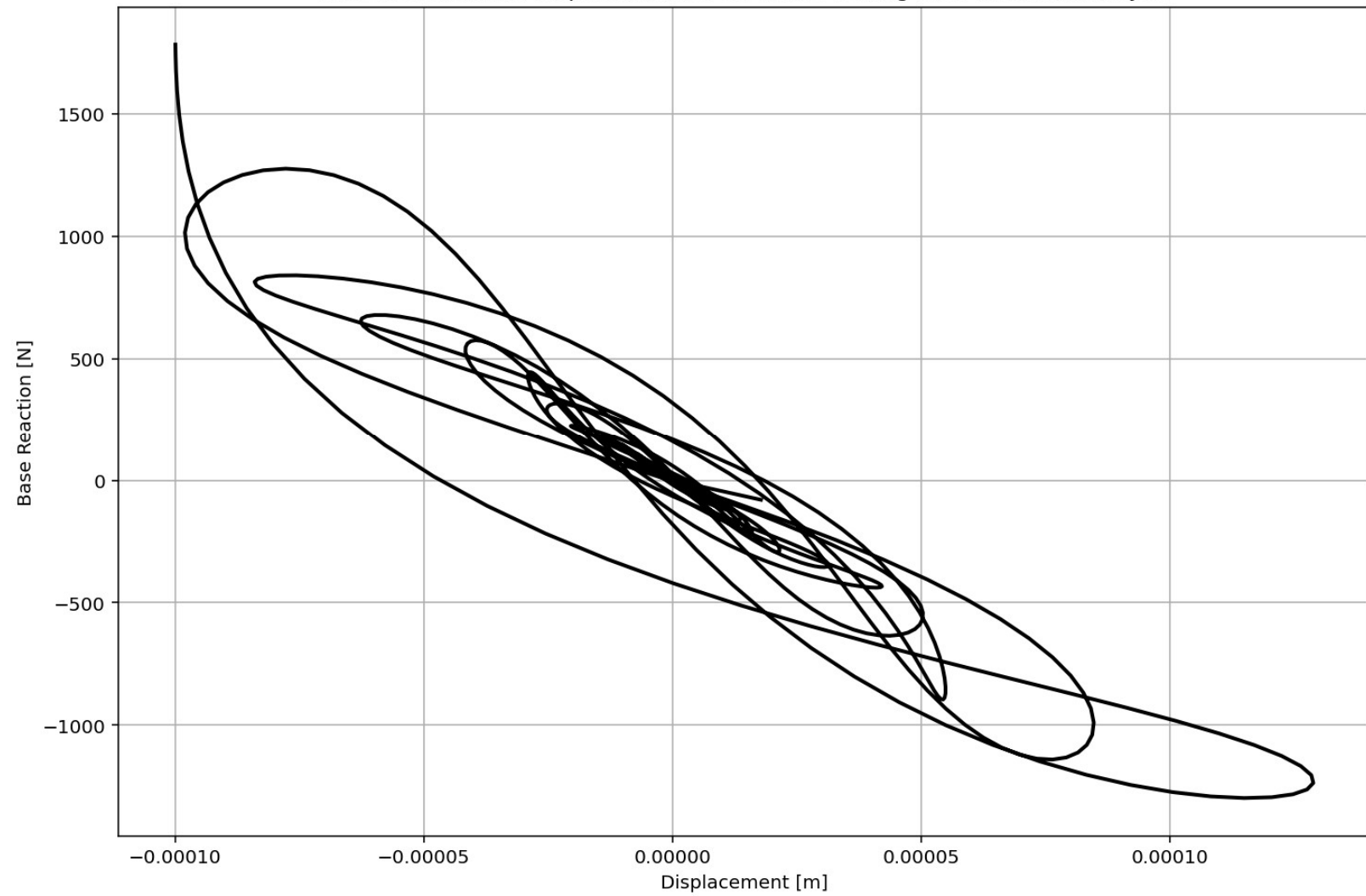
```
618 ##### DEFINE PARAMETERS FOR FREE-VIBRATION ANALYSIS
619 u0 = -0.00010      # [m] Initial displacement
620 v0 = 0.000015     # [m/s] Initial velocity
621 a0 = 0.000065     # [m/s^2] Initial acceleration
622 IU = True         # Free Vibration with Initial Displacement
623 IV = True         # Free Vibration with Initial Velocity
624 IA = True         # Free Vibration with Initial Acceleration
625 duration = 20.0   # [s] Analysis duration
626 dt = 0.001       # [s] Time step
627 DR = 0.03        # Damping Ratio
628
629 disp_02, disp_03, disp_04, disp_05 = [], [], [], []
630 velo_02, velo_03, velo_04, velo_05 = [], [], [], []
631 accel_02, accel_03, accel_04, accel_05 = [], [], [], []
632
633 if IU == True:
634     # Define initial displacement
635     ops.setNodeDisp(2, 1, u0, '-commit')
636     ops.setNodeDisp(3, 1, u0, '-commit')
637     ops.setNodeDisp(4, 1, u0, '-commit')
638     ops.setNodeDisp(center_node, 1, u0, '-commit')
639     # INFO LINK: https://openseespydoc.readthedocs.io/en/latest/src/setNodeDisp.htm
640 if IV == True:
641     # Define initial velocity
642     ops.setNodeVel(2, 1, v0, '-commit')
643     ops.setNodeVel(3, 1, v0, '-commit')
644     ops.setNodeVel(4, 1, v0, '-commit')
645     ops.setNodeVel(center_node, 1, v0, '-commit')
646     # INFO LINK: https://openseespydoc.readthedocs.io/en/stable/src/setNodeVel.htm
647 if IA == True:
648     # Define initial acceleration
649     ops.setNodeAccel(2, 1, a0, '-commit')
650     ops.setNodeAccel(3, 1, a0, '-commit')
651     ops.setNodeAccel(4, 1, a0, '-commit')
```

Base Reaction and Displacement of Structure During Free-vibration Analysis

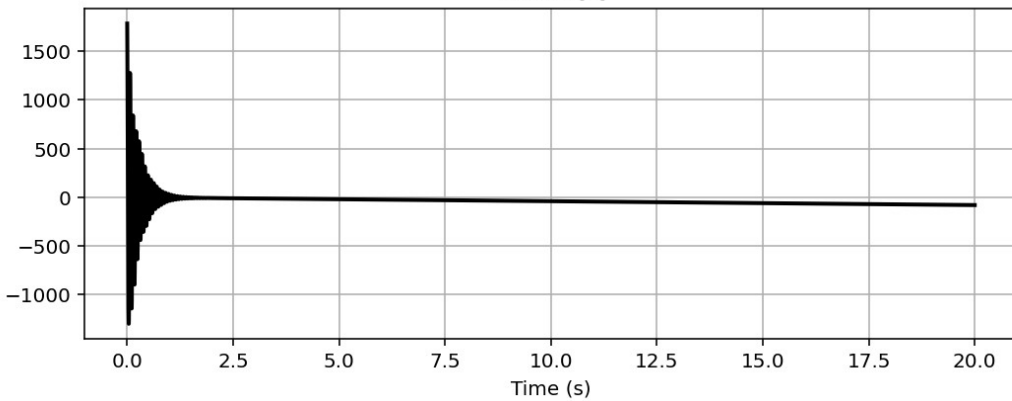
IPython Console Files Help Variable Explorer Debugger Plots History

Inline Conda: anaconda3 (Python 3.12.7) LSP: Python Line 619, Col 70 UTF-8 CRLF RW Mem 45%

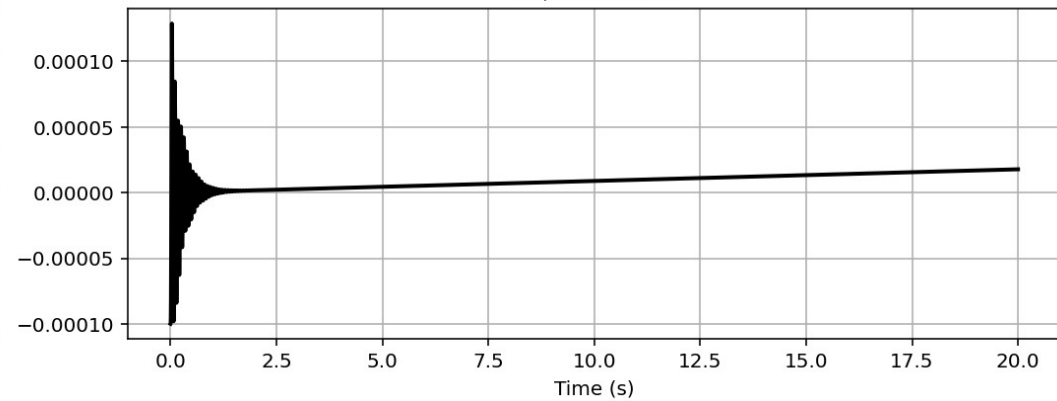
Base Reaction and Displacment of Structure During Free-vibration Analysis



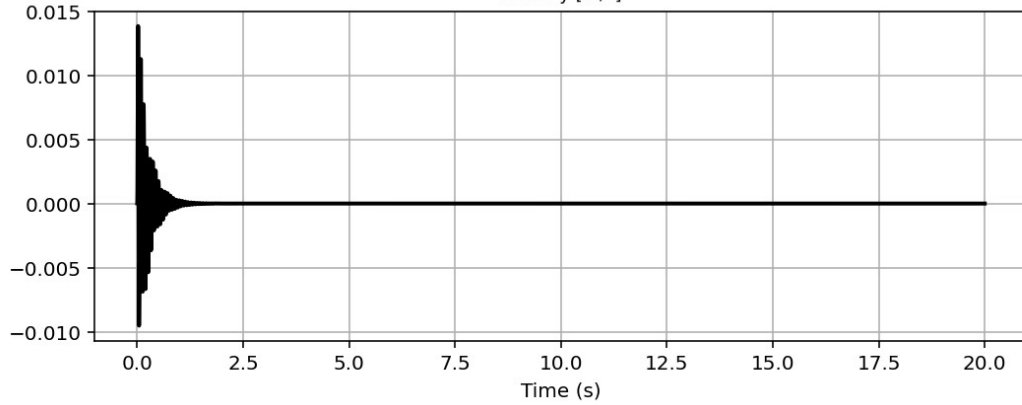
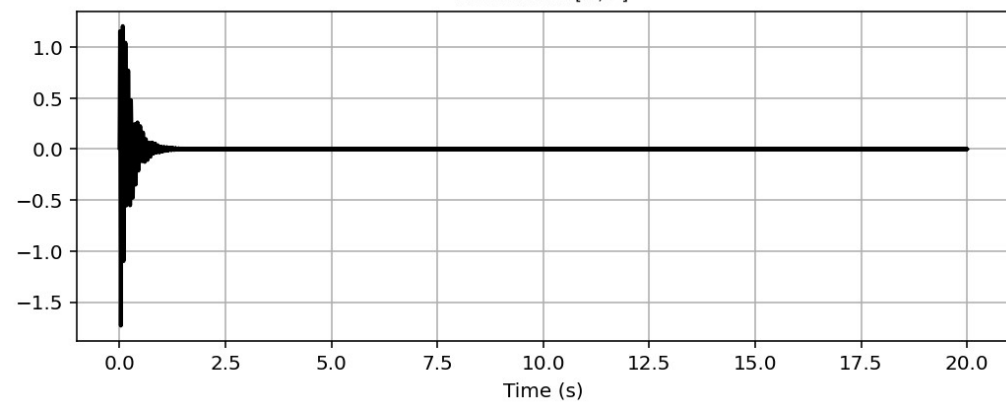
Reaction [N]



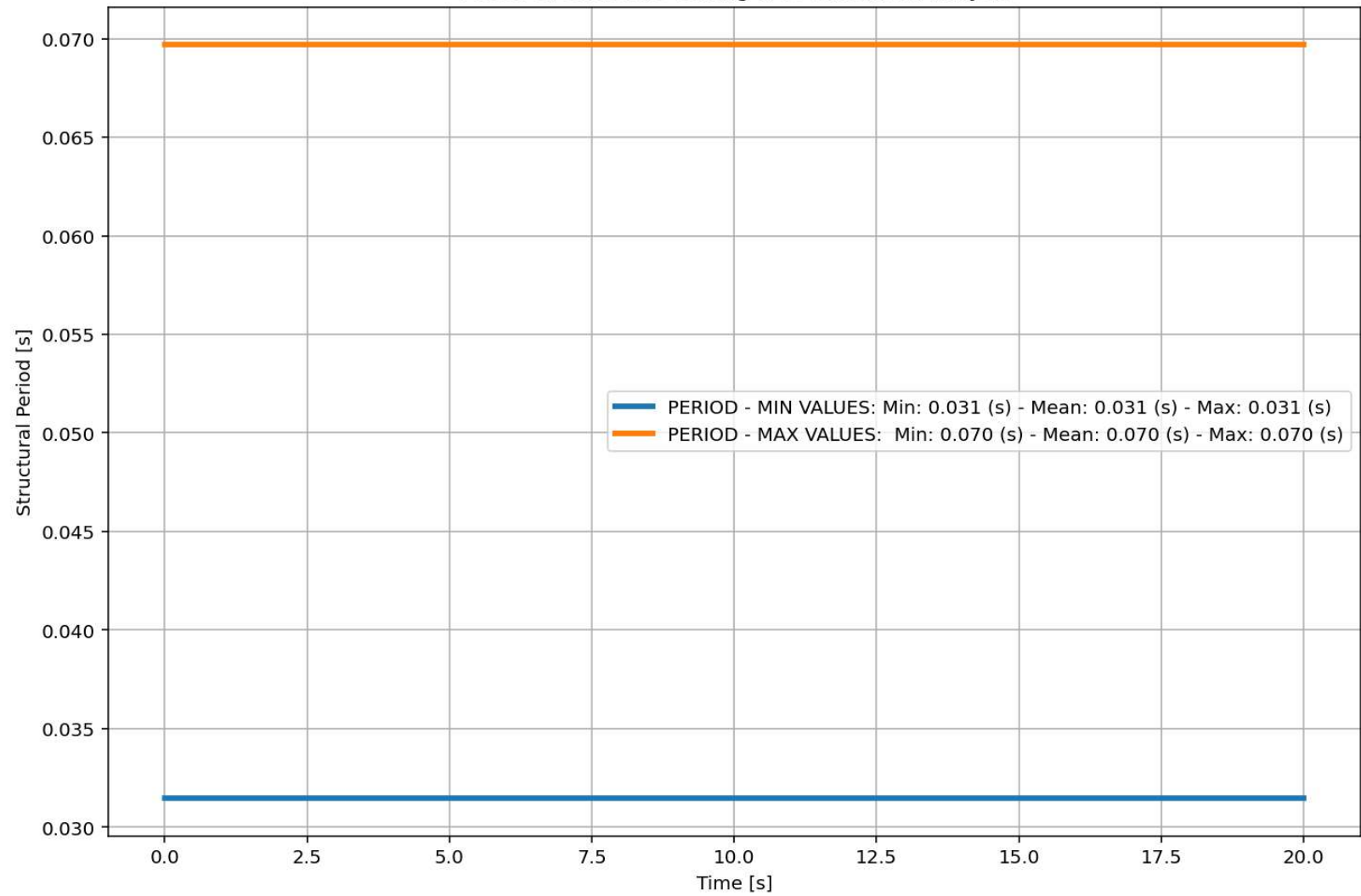
Displacement [m]

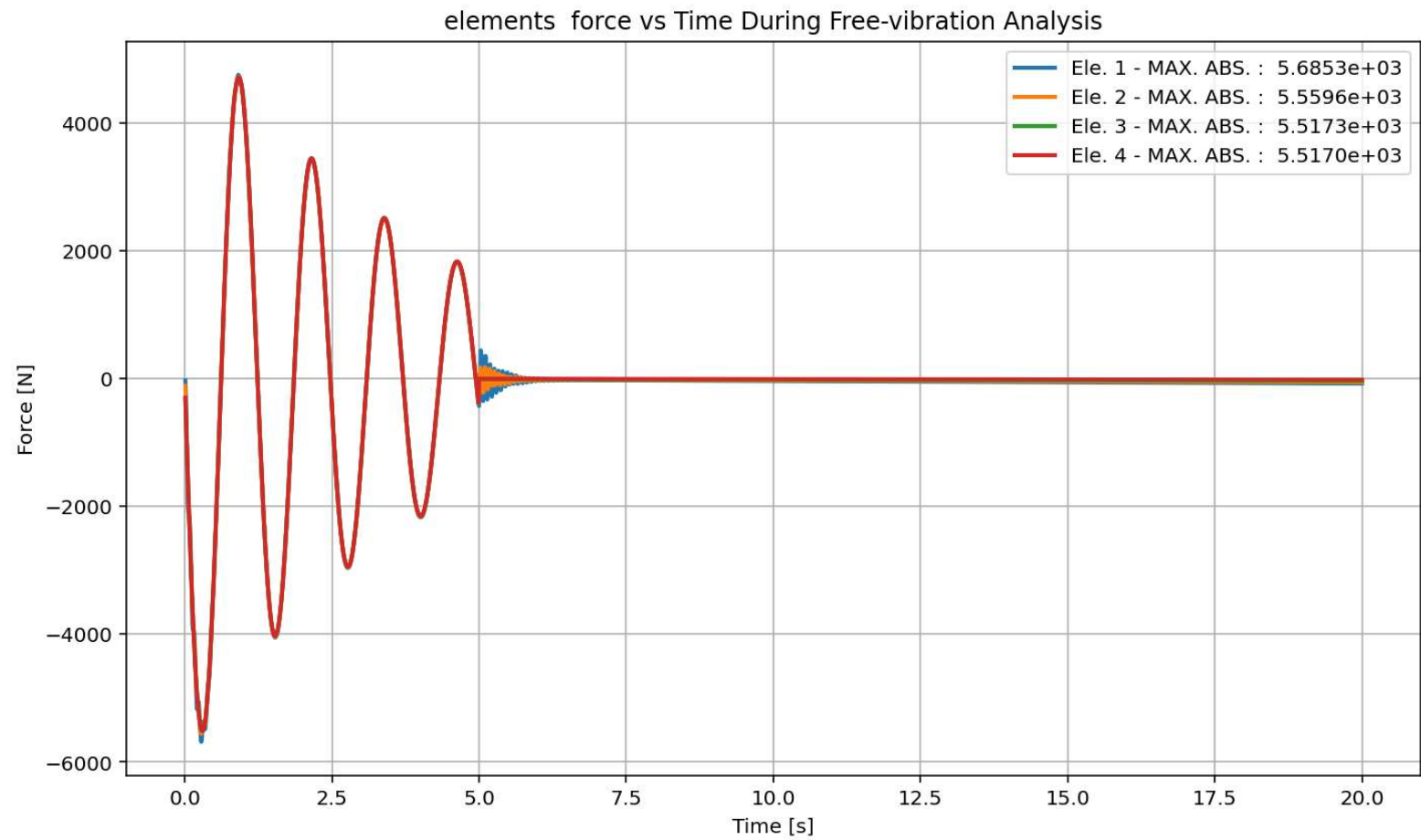


Velocity [m/s]

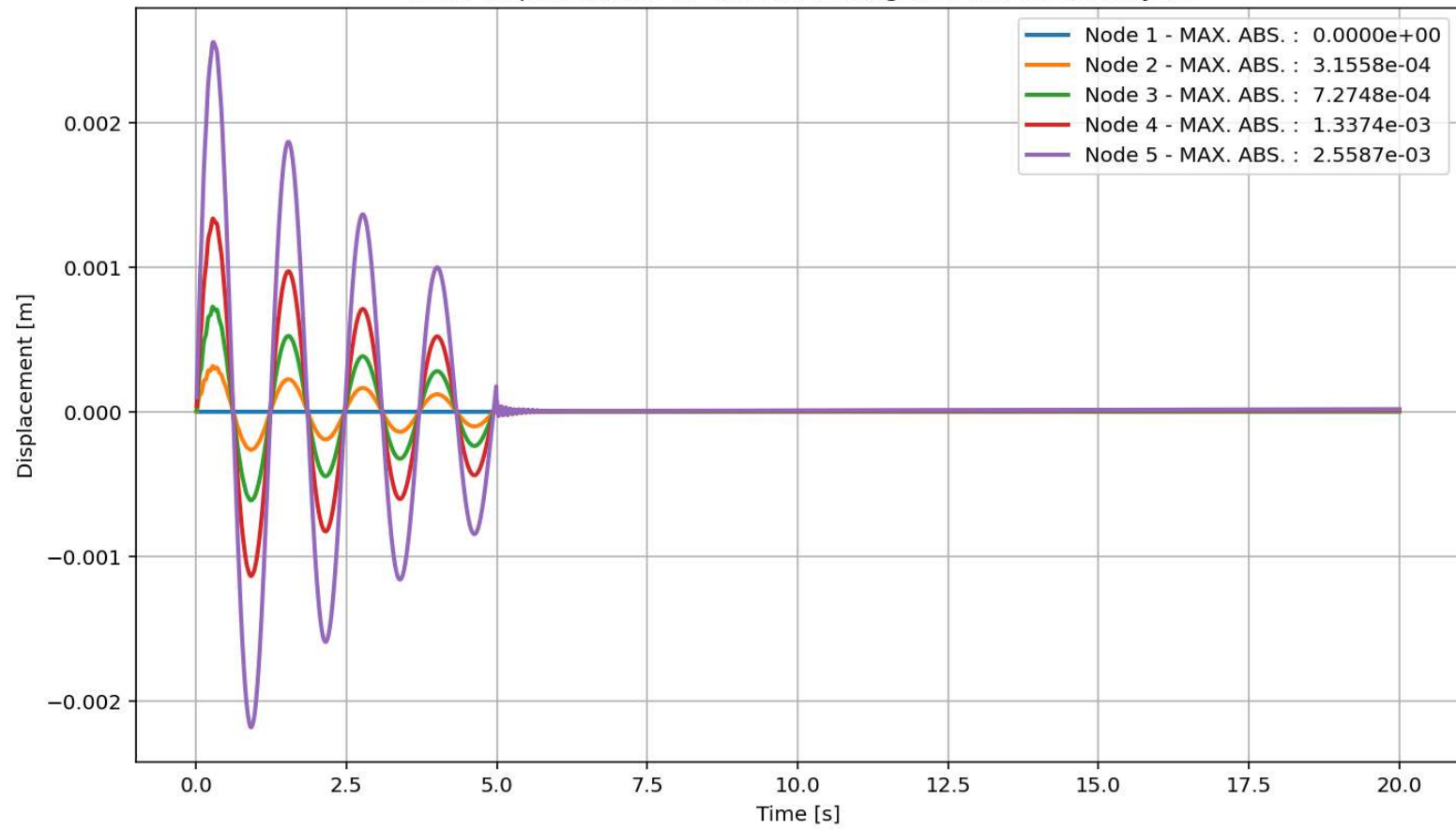
Acceleration [m/s²]

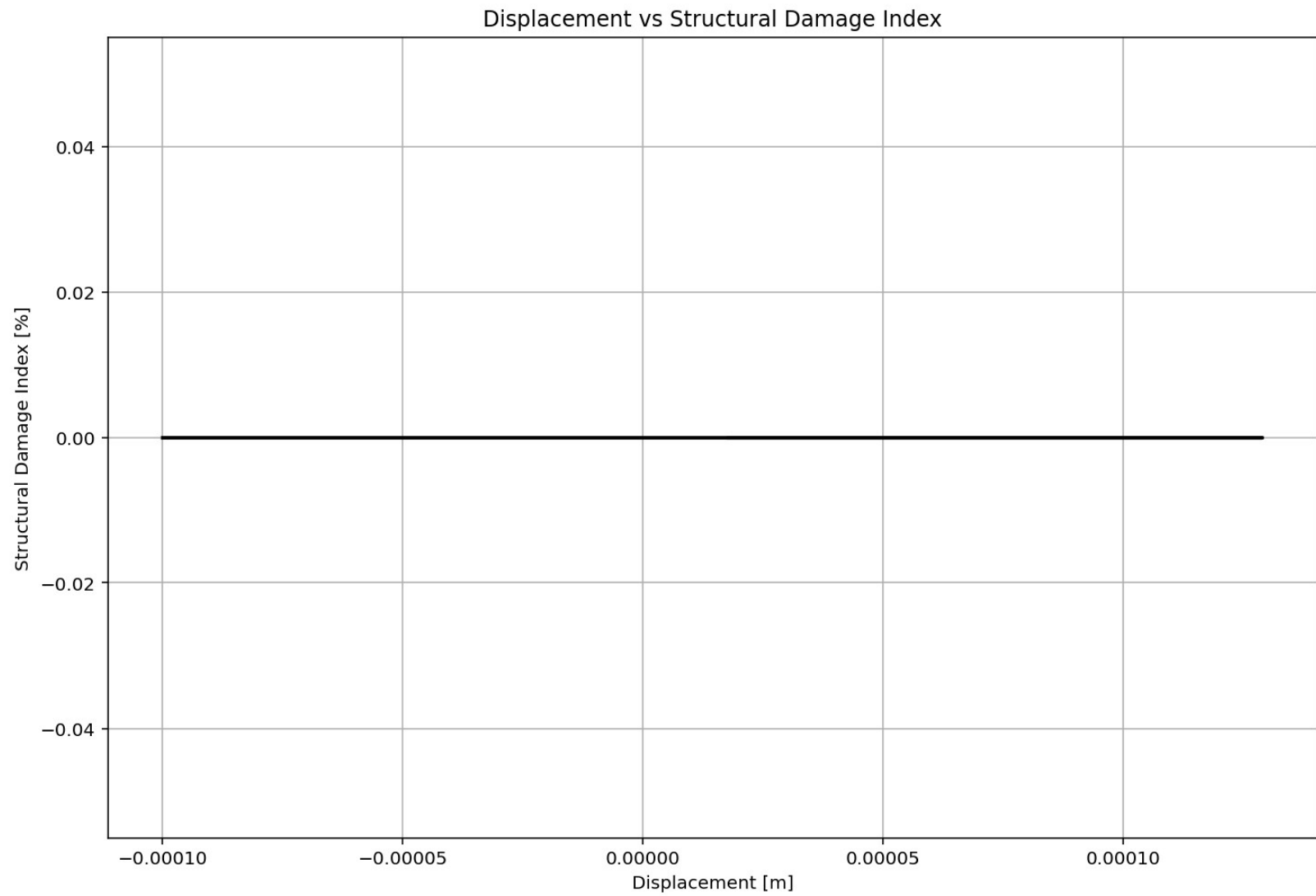
Period of Structure During Free-vibration Analysis





Node Displacements vs Time for During Free-vibration Analysis





Spyder (Python 3.12)

File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\Dell\Desktop\OPENSEES_FILES\+EIGHT_ANALY...OMPUTE_EFFECTIVE_PROPERTIES_FUN_FREE_VIBRATION.py

P(t)_MDOF_8_ANA.py COMPUTE_EFFECTIVE_..._FREE_VIBRATION.py

```
1  """ EQUIVALENT SDOF SYSTEM DERIVATION VIA DISPLACEMENT-BASED SEISMIC DESIGN PROCEDURE WITH
2  # Change MDOF to SDOF System with Displacement Based Design Concept
3  # THIS PROGRAM WRITTEN BY SALAR DELAVAR GHASHGHAEE (QASHQAI)
4  """
5  This script implements a displacement-based pushover transformation,
6  converting a multi-degree-of-freedom (MDOF) system into an equivalent
7  single-degree-of-freedom (SDOF) system for seismic assessment.
8  It calculates effective modal properties-displacement, mass, and
9  stiffness-by weighting element forces and nodal displacements according
10 to a presumed deformed shape.
11 The derived effective period provides a simplified dynamic characteristic
12 for performance-based engineering. The visualizations effectively track the
13 evolution of these equivalent parameters throughout the nonlinear analysis steps.
14 """
15
16 #-----
17 import numpy as np
18
19 displacement_X_1 = np.array(list(node_displacements.values())[1]) # DOF 02
20 displacement_X_2 = np.array(list(node_displacements.values())[2]) # DOF 03
21 displacement_X_3 = np.array(list(node_displacements.values())[3]) # DOF 04
22 displacement_X_4 = np.array(list(node_displacements.values())[4]) # DOF 05
23
24 ele_force_01 = np.array(list(ele_force.values())[0]) # ELEMENT 01
25 ele_force_02 = np.array(list(ele_force.values())[1]) # ELEMENT 02
26 ele_force_03 = np.array(list(ele_force.values())[2]) # ELEMENT 03
27 ele_force_04 = np.array(list(ele_force.values())[3]) # ELEMENT 04
28
29 STIFF_X_1 = np.abs(ele_force_01 / displacement_X_1)
30 STIFF_X_2 = np.abs(ele_force_02 / displacement_X_2)
31 STIFF_X_3 = np.abs(ele_force_03 / displacement_X_3)
32 STIFF_X_4 = np.abs(ele_force_04 / displacement_X_4)
33
34 MX2 = (MASS[0] * np.square(displacement_X_1) +
        MASS[1] * np.square(displacement_X_2) +
```

Effective Displacement - Median: 0.00902

Effective Mass - Median: 40.85 (5408)

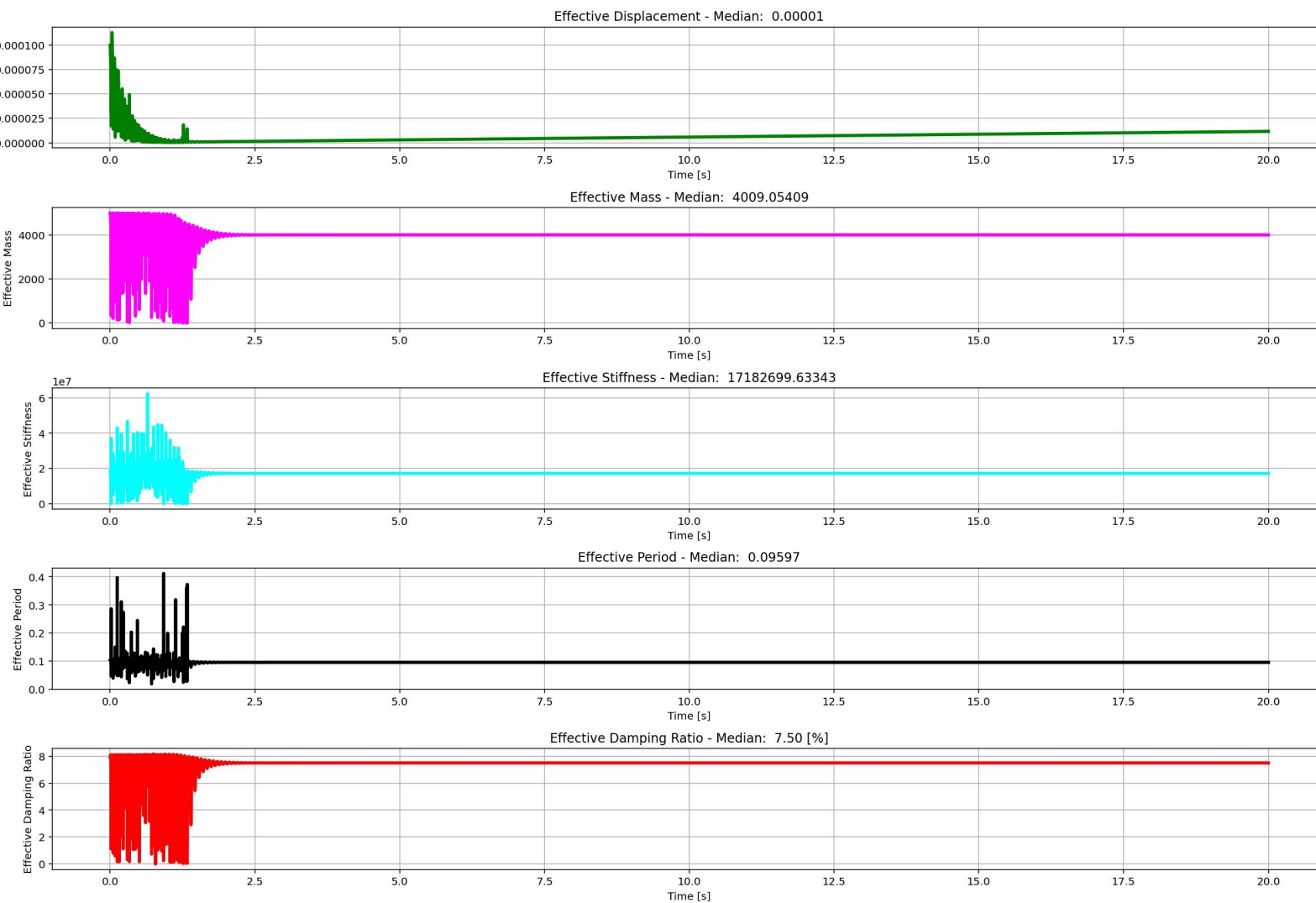
Effective Stiffness - Median: 3718299.63361

Effective Period - Median: 0.00997

Effective Damping Ratio - Median: 1.54 (%)

IPython Console Files Help Variable Explorer Debugger Plots History

Inline Conda: anaconda3 (Python 3.12.7) ✓ LSP: Python Line 7, Col 55 UTF-8 CRLF RW Mem 45%



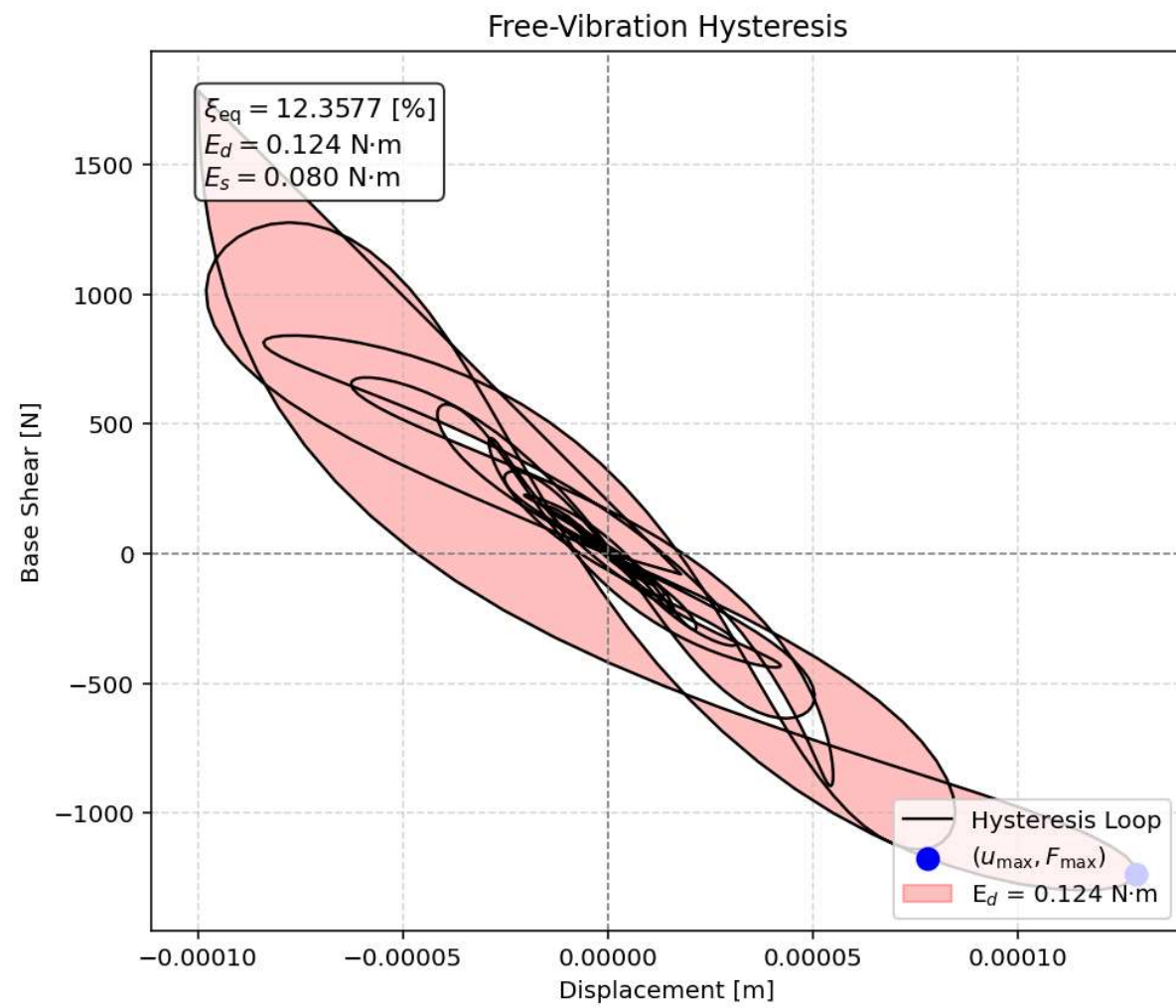
$$\Delta_d = \frac{\sum_{i=1}^n m_i \Delta_i^2}{\sum_{i=1}^n m_i \Delta_i}$$

$$m_{eff} = \frac{\sum_{i=1}^n m_i \Delta_i}{\Delta_d}$$

$$k_{eff} = \frac{\sum_{i=1}^n k_i \Delta_i}{\Delta_d}$$

$$T_{eff} = \frac{2\pi}{\sqrt{\frac{k_{eff}}{m_{eff}}}}$$

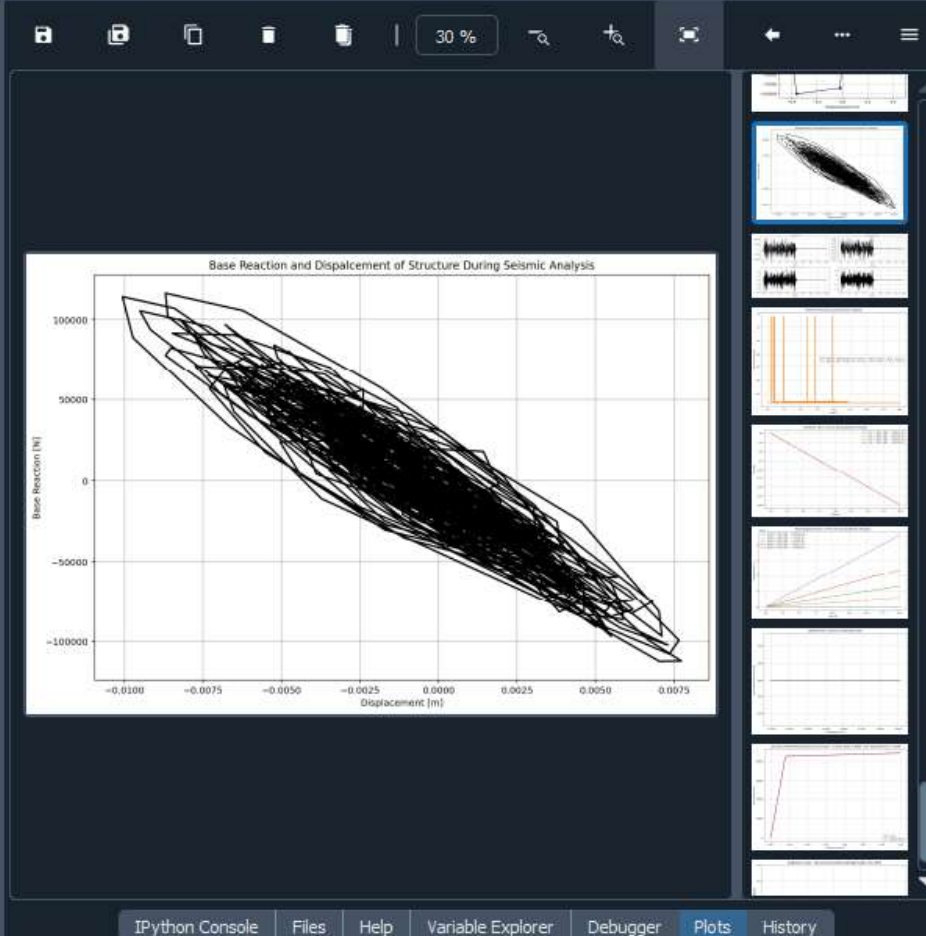
$$\xi_{eff} = \frac{\sum_{i=1}^n \xi_i \Delta_i}{\Delta_d}$$

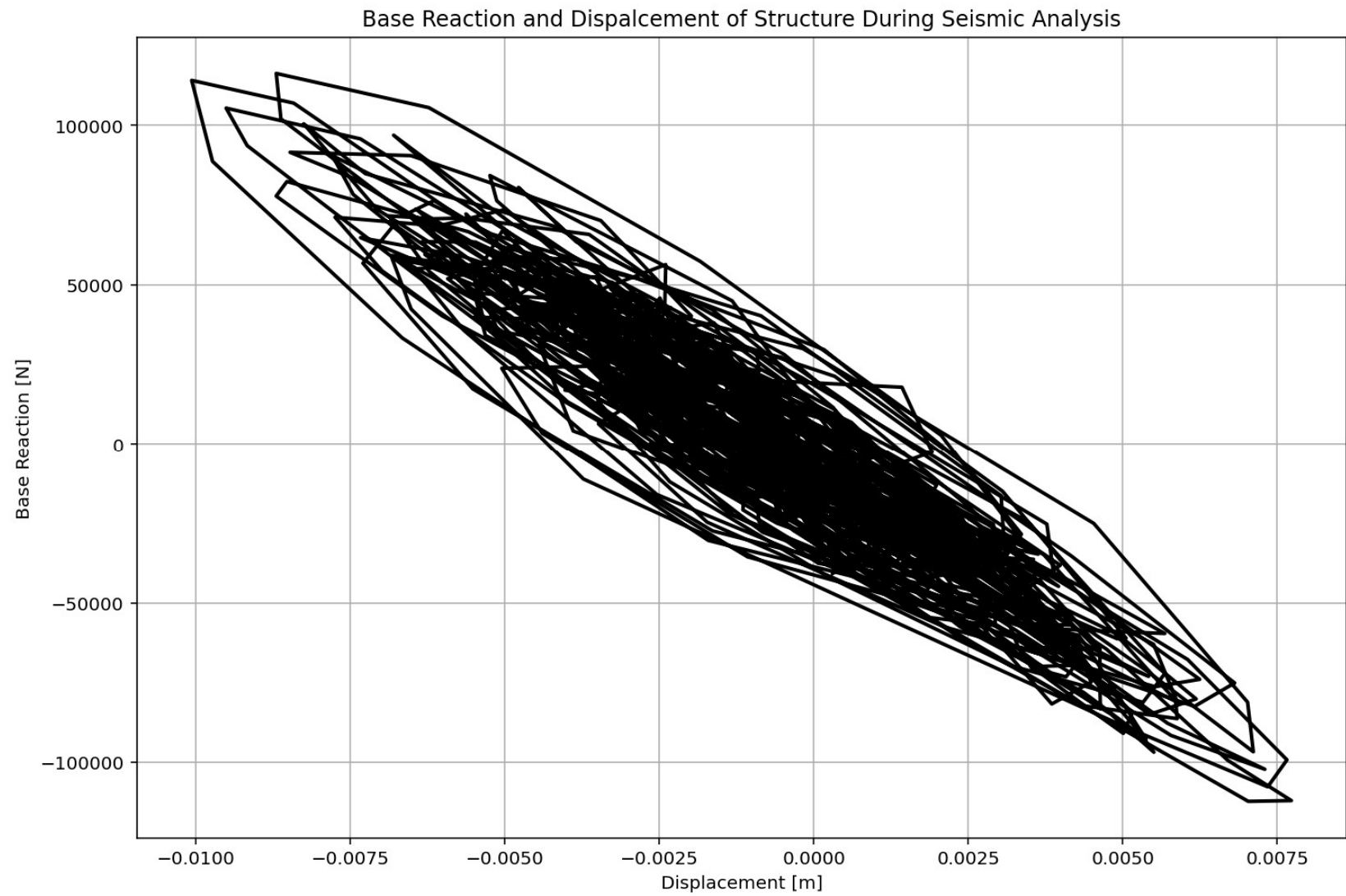


SEISMIC ANALYSIS

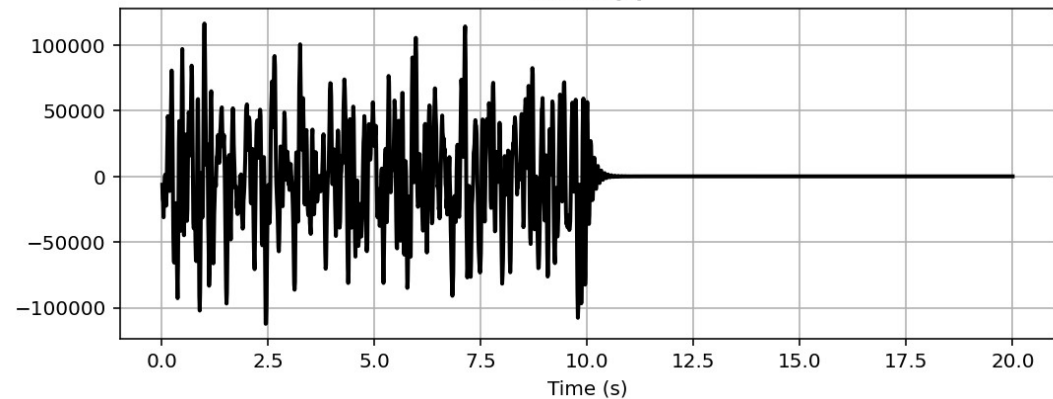
```

672 if ANAL TYPE == 'SEISMIC': # DYNAMIC TIME-HISTORY ANALYSIS
673     %% DEFINE PARAMETERS FOR SEISMIC ANALYSIS
674     duration = 20.0           # [s] Analysis duration
675     dt = 0.01                # [s] Time step
676     GMfact = 9810            # [mm/s²] standard acceleration of gravity or st
677     SSF_X = 0.10             # Seismic Acceleration Scale Factor in X Direct
678     SSF_Y = 0.10             # Seismic Acceleration Scale Factor in Y Direct
679     iv0_X = 0.0005           # [mm/s] Initial velocity applied to the node
680     iv0_Y = 0.0005           # [mm/s] Initial velocity applied to the node
681     st_iv0 = 0.0             # [s] Initial velocity applied starting time
682     SEI = 'X'                # Seismic Direction
683     DR = 0.03                # Damping ratio
684
685     # Define time series for input motion (Acceleration time history)
686     if SEI == 'X':
687         SEISMIC_TAG_01 = 100
688         ops.timeSeries('Path', SEISMIC_TAG_01, '-dt', dt, '-filePath', 'Ground_Acceler
689         # Define load patterns
690         # pattern UniformExcitation $patternTag $dof -accel $stsTag <-vel0 $vel0> <-fac
691         ops.pattern('UniformExcitation', SEISMIC_TAG_01, 1, '-accel', SEISMIC_TAG_01,
692     if SEI == 'Y':
693         SEISMIC_TAG_02 = 200
694         ops.timeSeries('Path', SEISMIC_TAG_02, '-dt', dt, '-filePath', 'Ground_Acceler
695         ops.pattern('UniformExcitation', SEISMIC_TAG_02, 2, '-accel', SEISMIC_TAG_02,
696     if SEI == 'XY':
697         SEISMIC_TAG_01 = 100
698         ops.timeSeries('Path', SEISMIC_TAG_01, '-dt', dt, '-filePath', 'Ground_Acceler
699         # Define load patterns
700         # pattern UniformExcitation $patternTag $dof -accel $stsTag <-vel0 $vel0> <-fac
701         ops.pattern('UniformExcitation', SEISMIC_TAG_01, 1, '-accel', SEISMIC_TAG_01,
702         SEISMIC_TAG_02 = 200
703         ops.timeSeries('Path', SEISMIC_TAG_02, '-dt', dt, '-filePath', 'Ground_Acceler
704         ops.pattern('UniformExcitation', SEISMIC_TAG_02, 2, '-accel', SEISMIC_TAG_02,
705         print('Seismic Defined Done.')
    
```

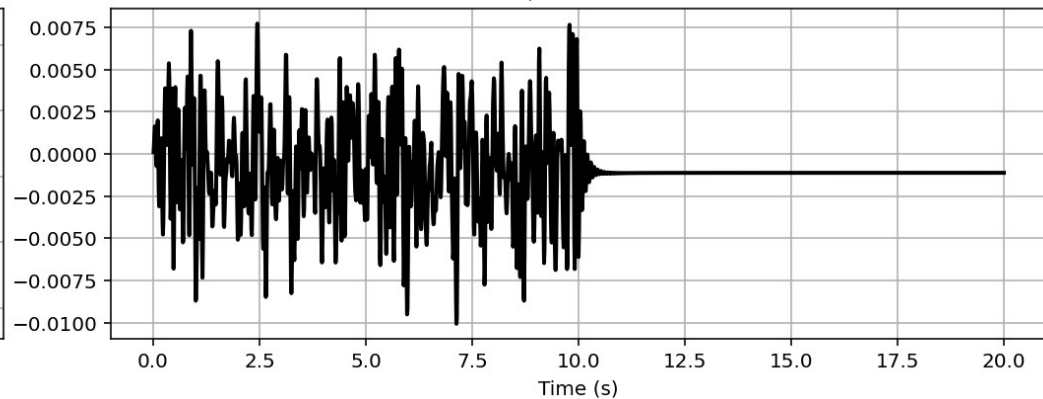




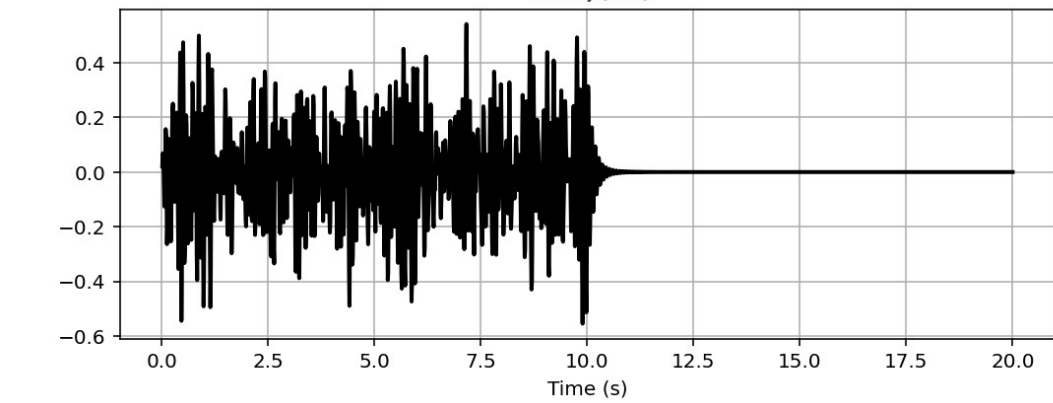
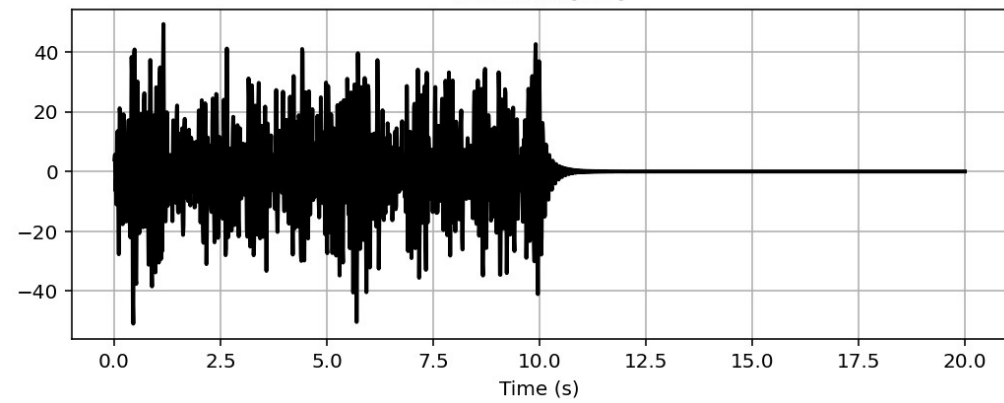
Reaction [N]



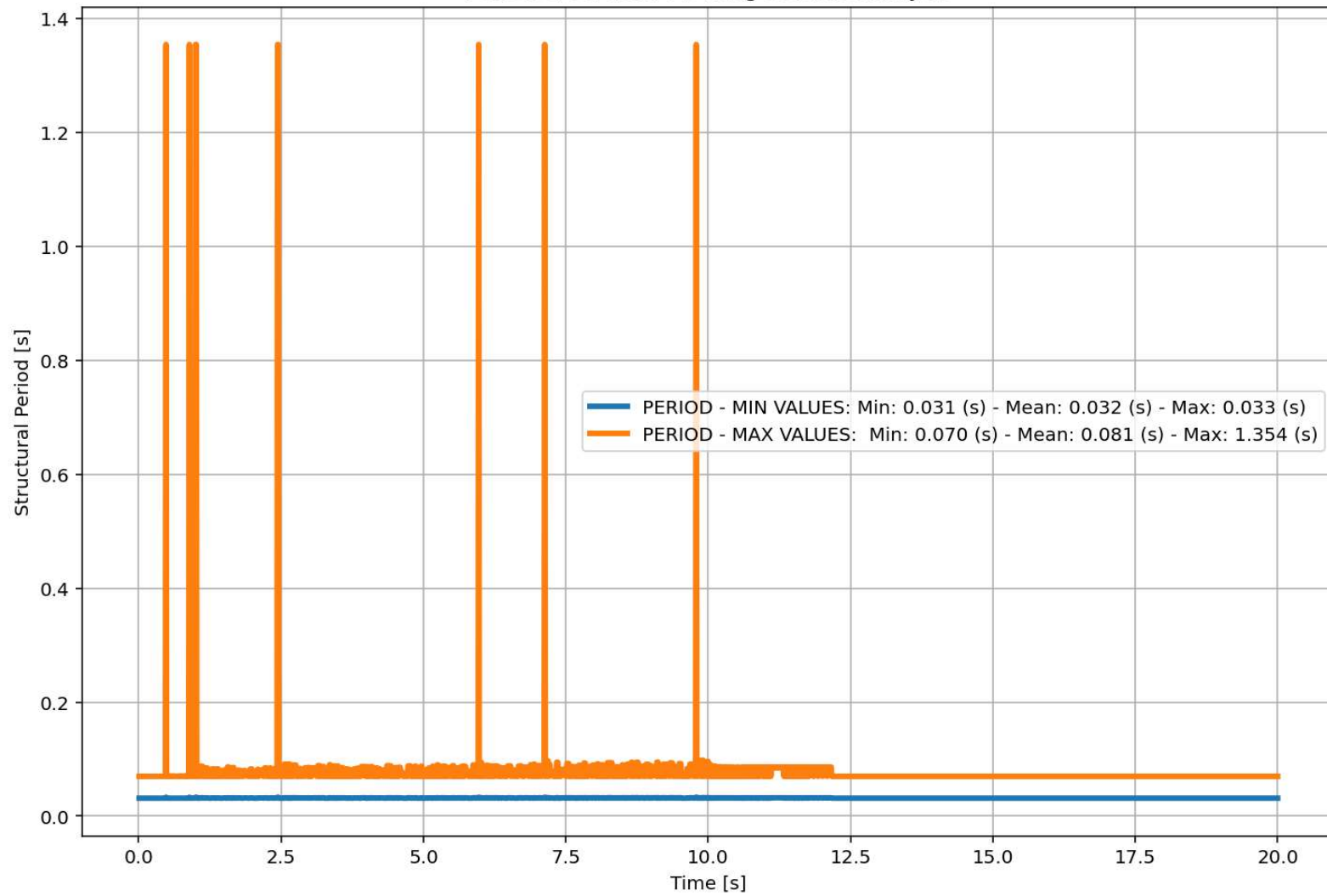
Displacement [m]

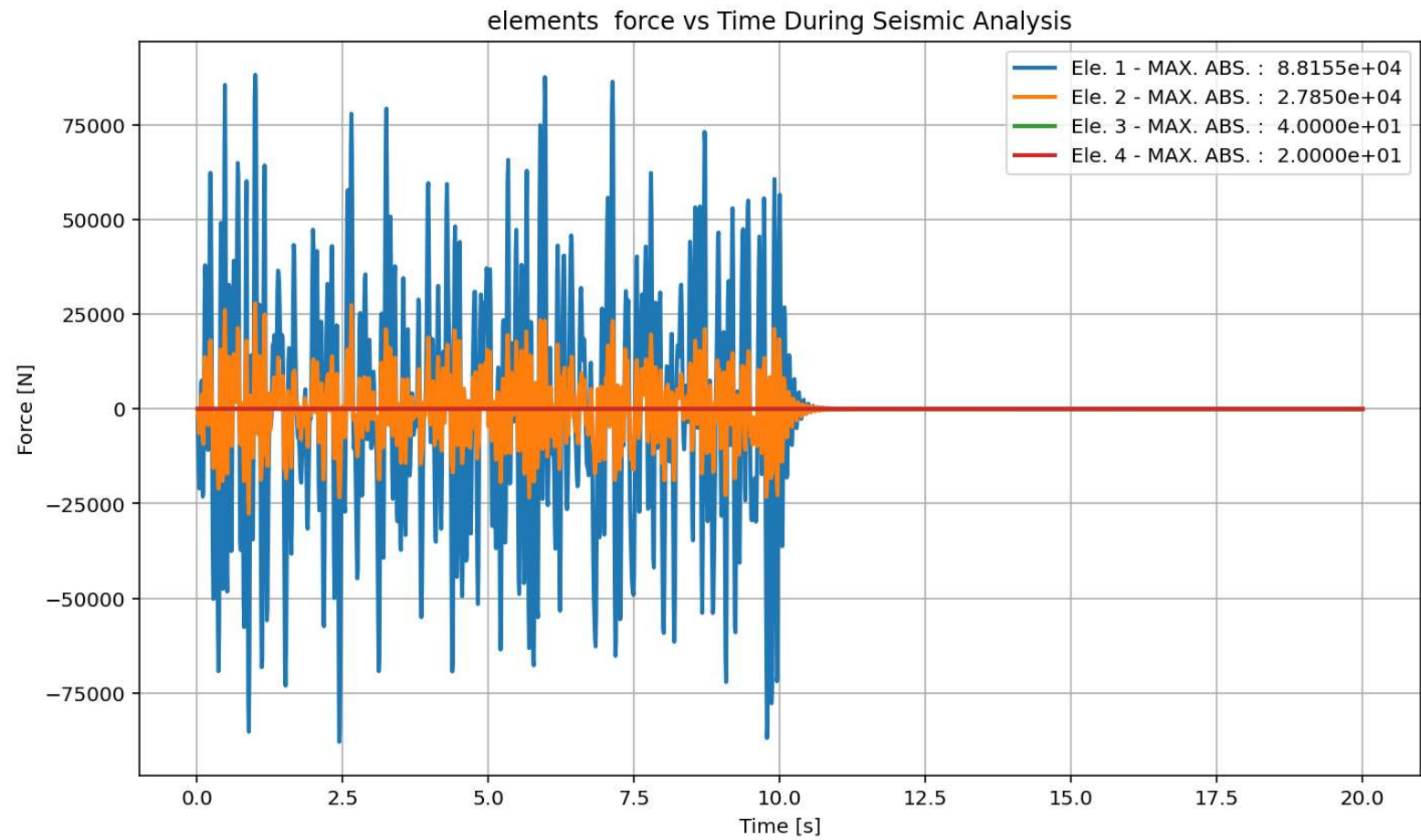


Velocity [m/s]

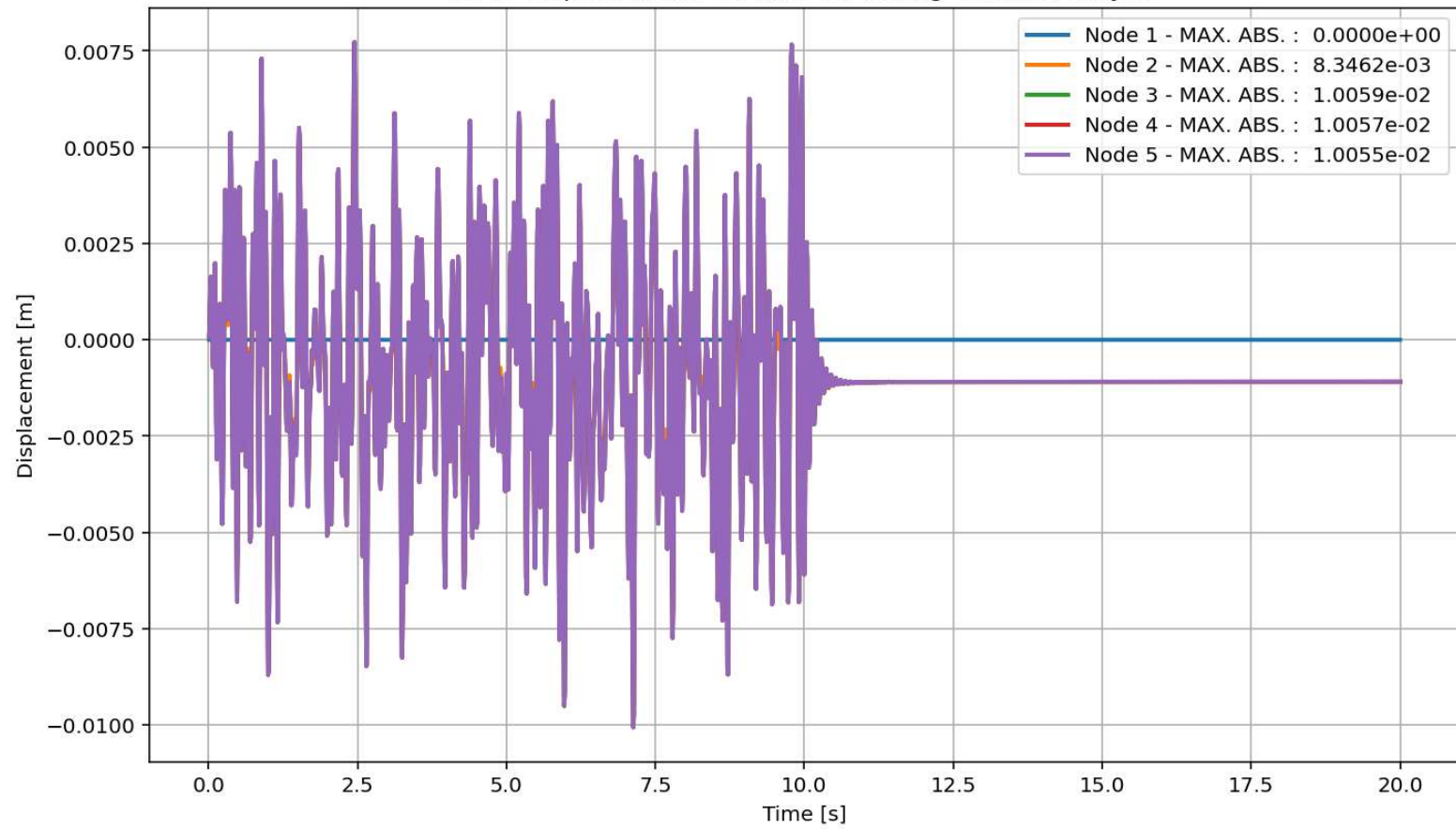
Acceleration [m/s²]

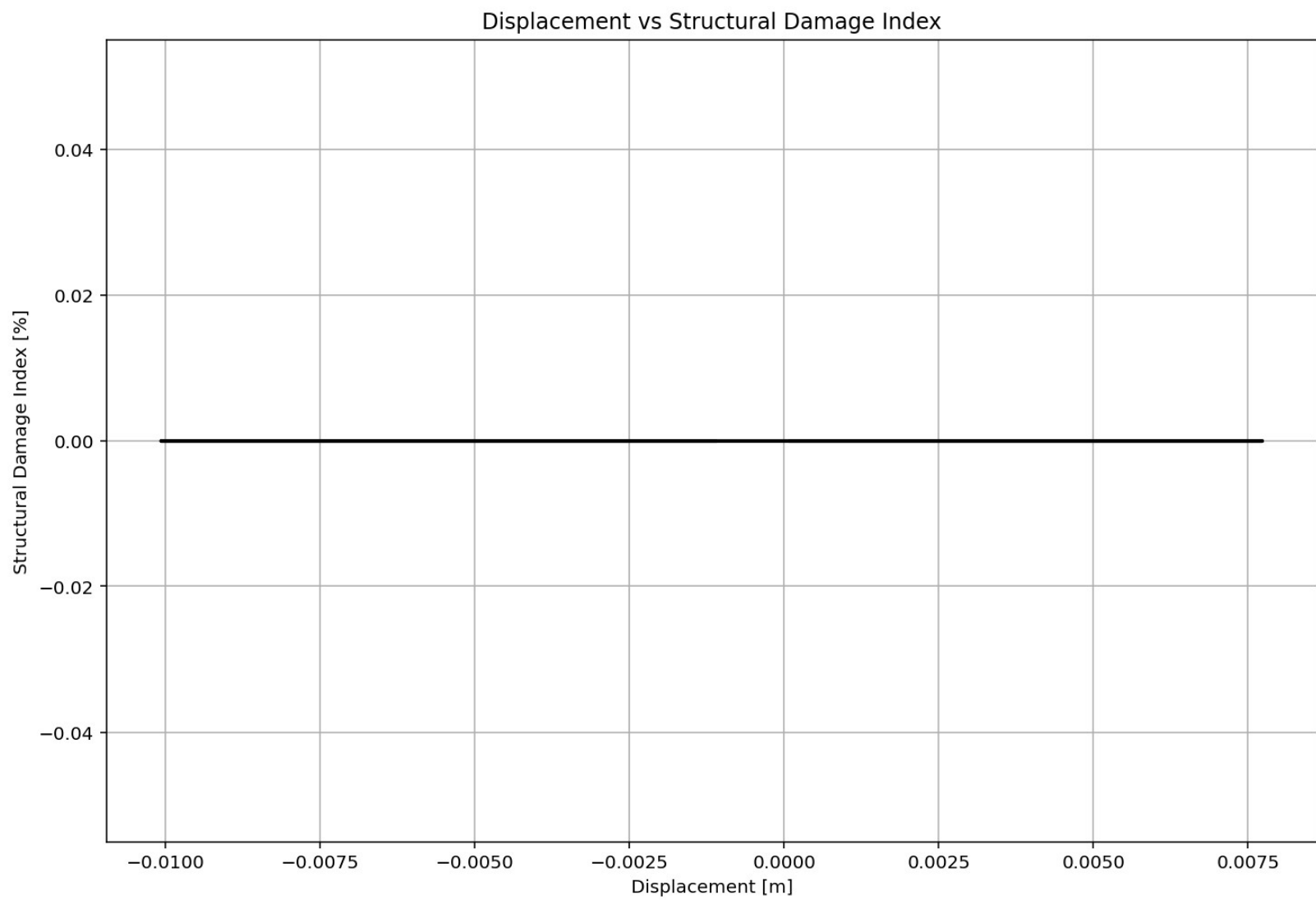
Period of Structure During Seismic Analysis

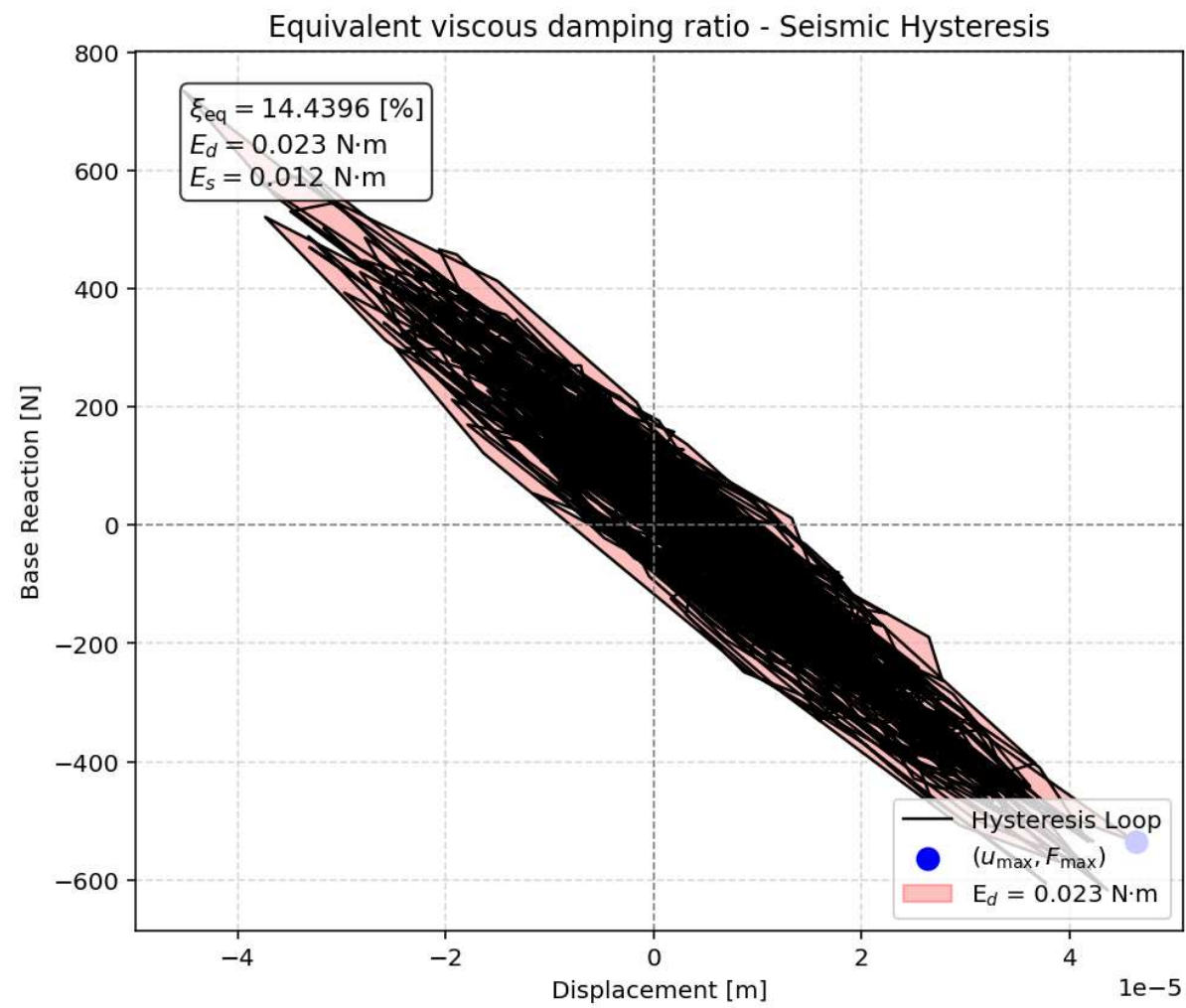


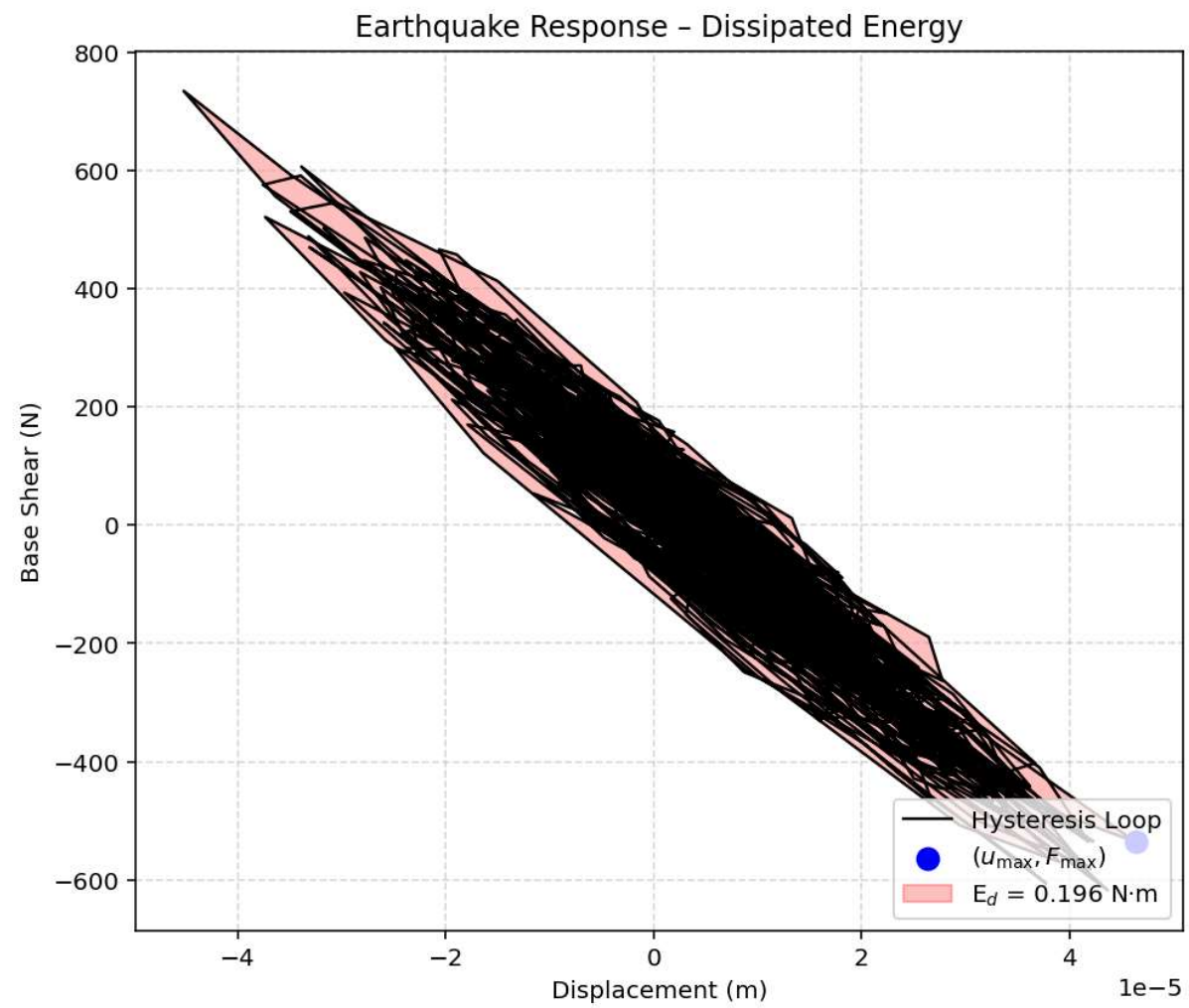


Node Displacements vs Time for During Seismic Analysis

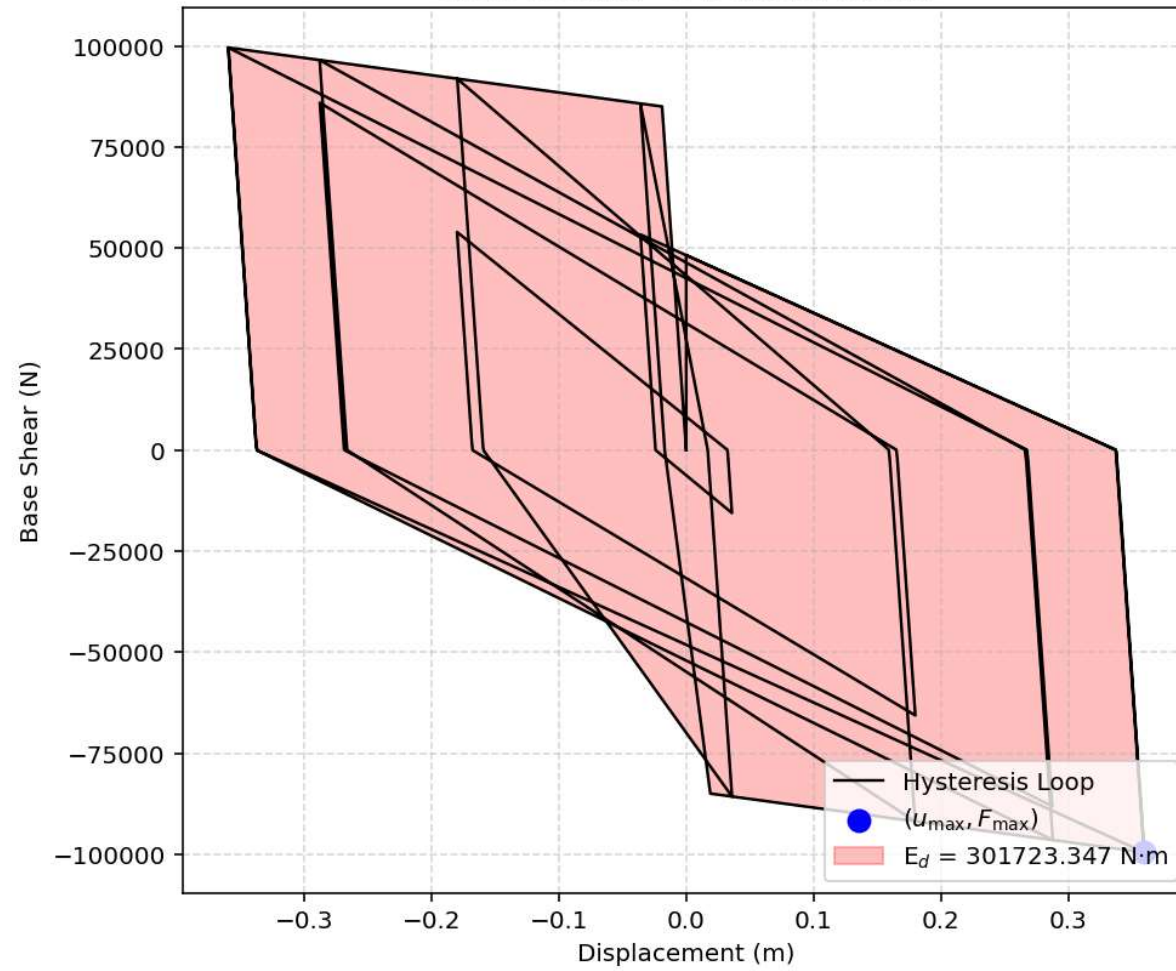








Cyclic Loading - Dissipated Energy



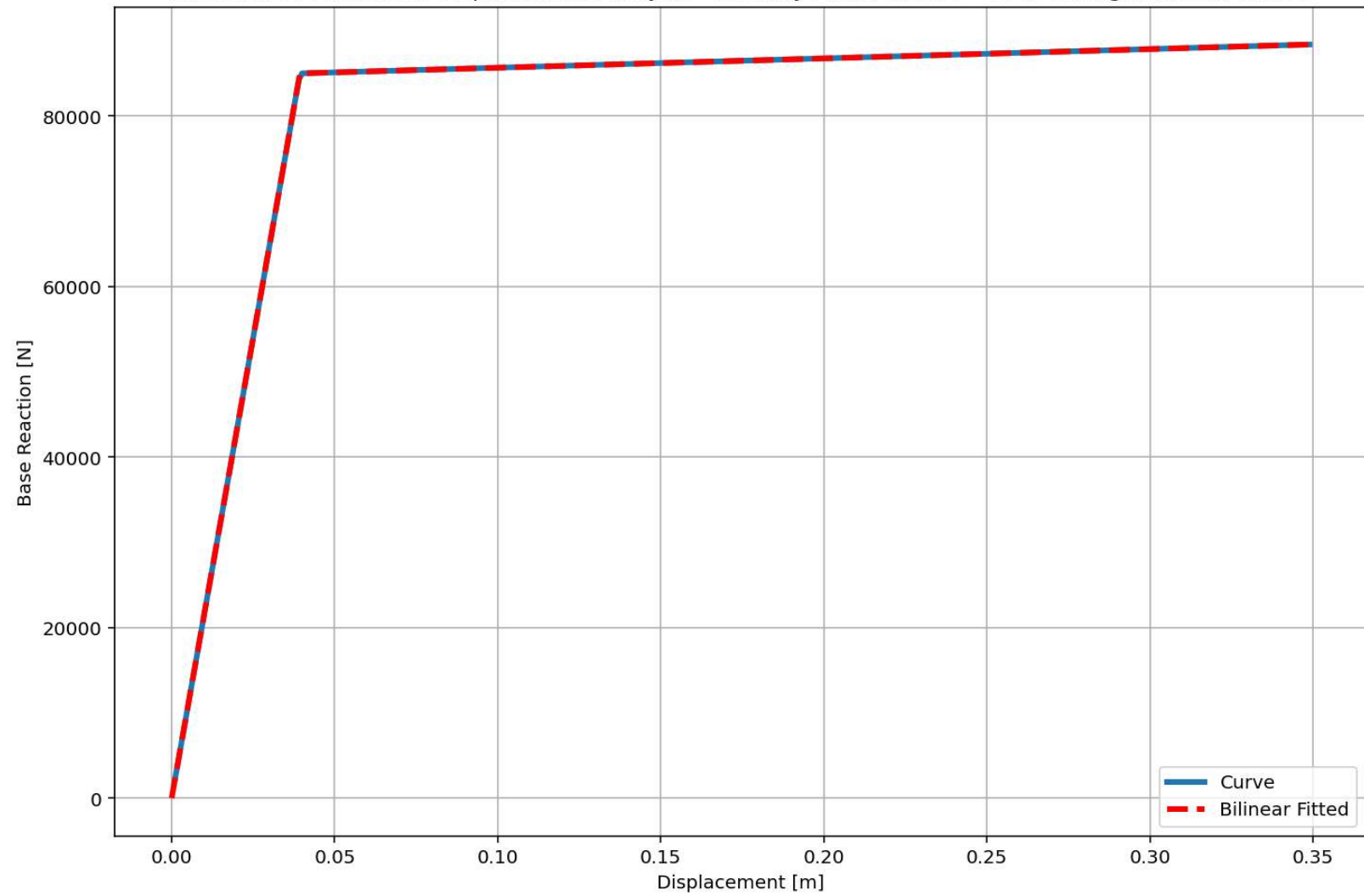
Dissipated Energy from Earthquake= 0.20 N·m

Dissipated Energy from Cyclic Displacement= 301723.35 N·m

DISSIPATED ENERGY CAPACITY INDEX: 0.000 [%]

ZONE 1: VERY MINOR DAMAGE

Last Data of BaseShear-Displacement Analysis - Ductility Ratio: 8.8696 - Over Strength Factor: 1.0399



+=====+

= Analysis curve fitted =

Disp Base Shear

[[0.00000000e+00 0.00000000e+00]

[1.88856396e-02 8.49853783e+04]

[3.60000000e-01 9.95852288e+04]]

+=====+

+-----+

Structure Elastic Stiffness : 4500000.00

Structure Plastic Stiffness : 276625.64

Structure Tangent Stiffness : 42800.46

Structure Ductility Ratio : 19.06

Structure Over Strength Factor: 1.17

+-----+

Over Strength Coefficient (Ω_0): 1.1718

Displacement Ductility Ratio (μ): 19.0621

Ductility Coefficient (R_μ): 19.0621

Structural Behavior Coefficient (R): 22.3368

Structural Ductility Damage Index in X Direction: -5.5229 (%)

Seismic Fragility Curves by Damage State from DYN. TIME-HISTORY ANA. DUCTILITY DAMAGE INDEX [%]

